

ME350

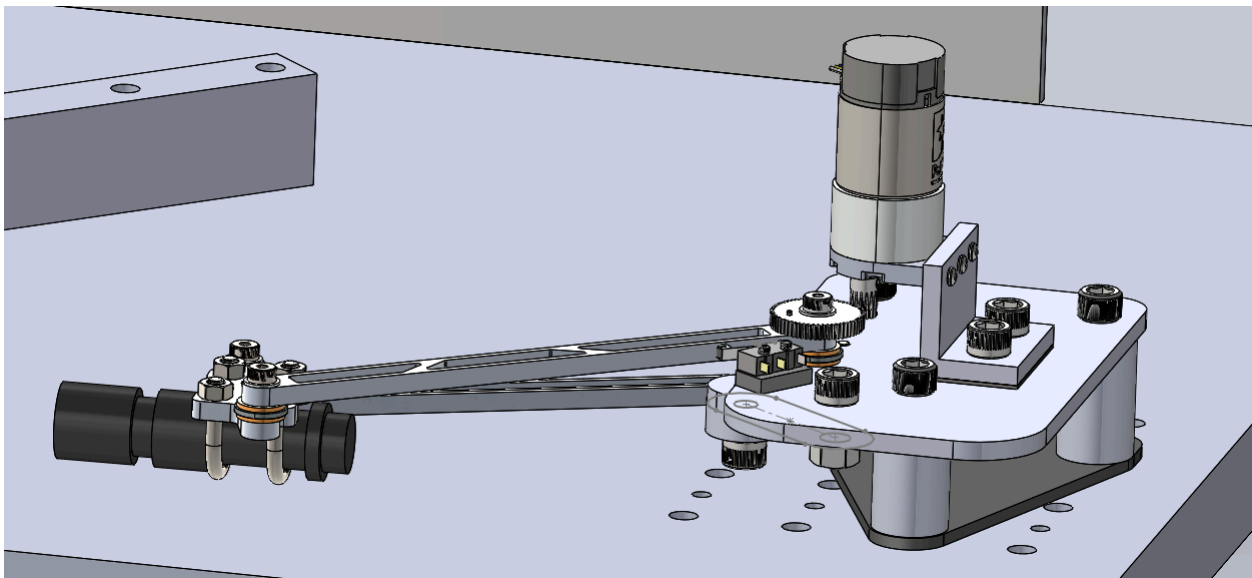
Winter 2024 Semester

FINAL REPORT

Team 51

Jay Lee
Ethan Feng
Brady Buten
Yuandi Huang
Alexander Alba Oderiz

GSI/IA: Shiwoo Lee



"We have fully abided by the University of Michigan College of Engineering Honor Code"

Jay Lee
Ethan Feng
Brady Buten
Yuandi Huang
Alexander Alba Oderiz

Section 1: Design Goals, Requirements, and Specifications

Project Goals

Requirements

Specifications

Section 2: Design Selection

Selection Criteria

Data Collection

Selection Process

Conclusion

Section 3: Final Linkage Design and Analysis

Notable Design Features:

Joint Design

Weight Estimate

ADAMS Model

Section 4: Motion Generator Revision:

Section 5: Energy Conversion Introduction:

Section 6: Transmission Ratio and Type Determination:

Inertia Matching

Section 7: Final Transmission Design:

Section 8: Torque Transfer Analysis:

Section 9: Motor Controls:

Section 10: Encoder Counts, Proximity Sensor Thresholds, and Controller Gains:

Section 11: Final Testing Results/Discussion:

Section 12: Design Critique & Evaluation:

Appendix A: Assembly Manual

Linkage Assembly

Linkage Part List:

Step 1:

Step 2:

Step 3:

Step 4:

Step 5:

Step 6:

Step 7:

Step 8:

Step 9:

Step 10:

Transmission Assembly

Transmission & Motor Assembly Part List:

Step 1:

Step 2:

Step 3:

[Step 4:](#)

[Step 5:](#)

[Step 6:](#)

[Step 7:](#)

[Step 8:](#)

[Step 9:](#)

[Step 10:](#)

[Appendix B: Bill of Materials for the Final Design](#)

[Appendix C: Approval Packages](#)

[Appendix D: Wiring Diagram, Arduino Code Changes](#)

Section 1: Design Goals, Requirements, and Specifications

Project Goals

The objective of this project is to design, test, and manufacture a powered linkage mechanism capable of detecting targets which activate sporadically, as well as shine a flashlight onto each target's corresponding photosensor. Five targets and various obstacles are placed according to a defined playing field, which the designed mechanism must mount to. The designed mechanism will be a four-bar linkage powered by a DC motor operating on a closed-loop control system, and the mechanism must be capable of orienting itself to hit all 5 targets.

Requirements

Specific engineering parameters must be set in order to properly define the design space. These target requirements are also necessary in order to design not only a working linkage mechanism, but also a competitive one. Here are some high level requirements for our linkage mechanism.

1. The flashlight must be able to rotate
 2. The combined weight must be relatively light
 3. The average flashlight to sensor distance must be relatively small
 - ~~4. The linkage must be sturdy enough to withstand multiple uses~~
 - ~~5. The linkage must be safe to assemble and to operate~~
 - ~~6. The linkage must have predictable motion~~
-
4. The linkage must maintain a relatively low transmission angle deviation.
 5. The linkage must have relatively short link lengths

Specifications

The most important parameters, in order of importance, are the flashlight clearance, weight, average transmission angle deviation, maximum transmission angle deviation, input and follower length, coupler length, and lastly, average flashlight-sensor distance. These values will largely determine the performance of our linkage mechanism. Thus, the specific parameters are stated below.

1. The flashlight's full range of motion must not be obstructed by any obstacle on the playing field
2. The combined weight of all rotating links must be 10 ± 4 ounces
3. The average flashlight-sensor distance must not exceed 16 ± 4 inches (for targets 4 and 5, this distance is measured from the flashlight head to the point of incidence on the mirror) (see Figure 1.2 and 1.3 for visual definition)
4. The average transmission angle deviation must be 16 ± 5 degrees (for visuals on transmission angle, see Figure 1.1)
5. The maximum transmission angle deviation must be 25 ± 10 degrees
6. The input and follower link lengths must not exceed 9 ± 2 inches
7. The coupler link length must not exceed 3.4 ± 1 inches

Explanations of How Requirements Connect to the Goals of the Project:

Specification 1: For our project, we need to be capable of aiming at all five photosensors. In order to be able to do that, we require our linkage to not be obstructed by any obstacle on the playing field. In addition, it must be capable of moving its full range of motion.

Specification 2, 6, 7: Part of our grading criteria in this class is maintaining a low linkage weight. Hence, we put this requirement on this project to help maximize the points we gain in the weight category of the project. In addition, a low weight correlates with a lower inertia which allows for faster angular rotations and will lead to a better performance in the overall competition.

Specification 3: Another part of our grading criteria is the intensity of light detected by the photosensors. By having our linkage be closer to those sensors, we will be able to gain more points in the intensity category. In addition, the photosensors will be triggered faster the more intense our light is. So, we put this requirement on this project to better maximize points in the intensity/performance category of the project.

Specification 4, 5: A third part of our grading criteria is sustaining a low transmission deviation. This enables our linkage to better apply torque to our coupler, which enables faster motion and a better project performance. Thus, we put an average and maximum transmission angle deviation for this project.

Table 1.1, below, serves as a summary of the above requirements.

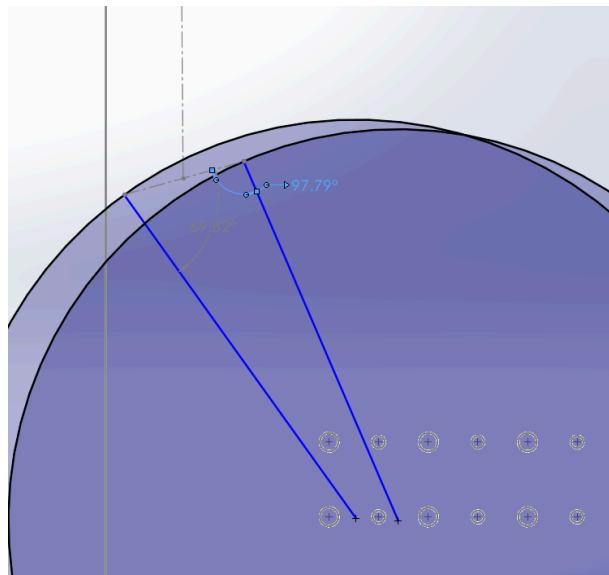


Figure 1.1: The transmission angle as shown above is the angle the coupler makes with the follower link. The transmission angle deviation is the absolute value of 90° minus the transmission angle. The desired value is to be as close to 0° as possible.

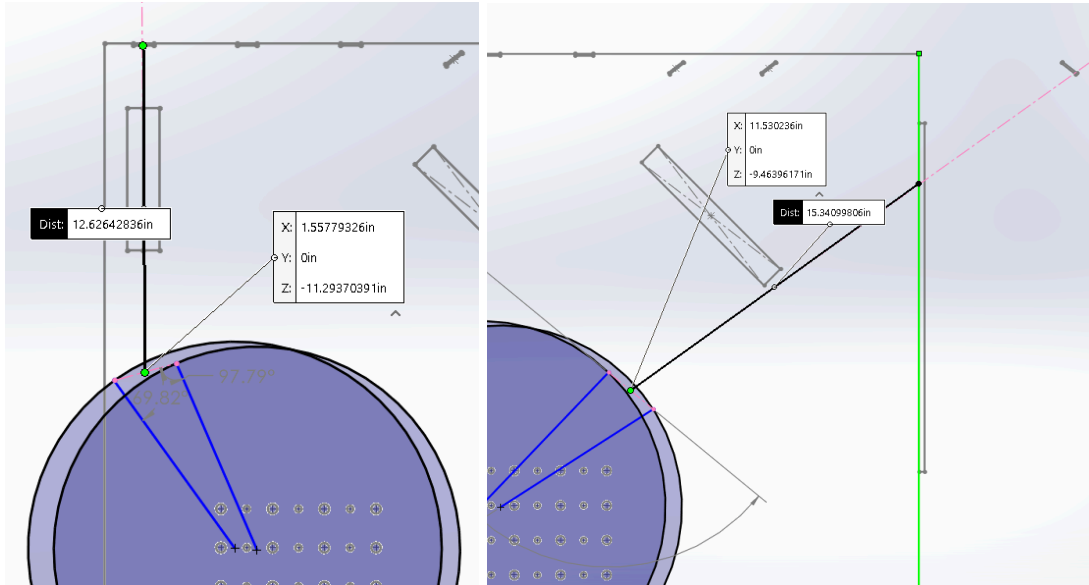


Figure 1.2 and 1.3: Definitions for flashlight-sensor distance is shown above, for target #1 and target #5, respectively.

Table 1.1: Determined target specifications and measurement techniques given design requirements

Requirement	Description	Team 51 Target Specification	Method of Measurement
Flashlight Clearance	Binary requirement (yes/no)-- measures whether or not the flashlight has full clearance in its range of motion	Yes	Test (yes/no)
Weight	Combined weight of rotating links (input, follower, coupler)	10±4oz.	Scale
Average Transmission Angle Deviation	$ Transmission\ Angle - 90^\circ $ (average of all five targets)	16±5°	CAD Measurement / Machinist Protractor
Maximum Transmission Angle Deviation	$ Transmission\ Angle - 90^\circ $ (Maximum)	25±10°	CAD Measurement/Machinist Protractor
Input Length	Pin-to-pin length of input link	9±2 inches	CAD Measurement/Machinist Ruler
Follower Length	Pin-to-pin length of follower link	9±2 inches	CAD Measurement/Machinist Ruler
Coupler Length	Pin-to-pin length of coupler link	3.4±1 inches	CAD Measurement/Machinist Ruler
Average Flashlight-Sensor Distance	Average distance between flashlight head and photosensor	16±4 inches	CAD Measurement/Machinist Ruler

Section 2: Design Selection

Selection Criteria

Our team's approach to evaluating linkages will be to record each option's specifications against our chosen requirements, and then make comparisons to determine each design's relative score in each category. These scores will be evaluated in a Pugh chart with our chosen criteria to determine the best design. We believe that this method of numerically comparing the specifications of each design and weighing them in a Pugh chart is the most unbiased approach for us. With this approach, the only input we have in comparing each design should be the relative importance of each specification, as the relative performance of each design within each specification is determined by quantitative methods based on measurements.

Each requirement will be given a weight of between 1-5. Our chosen criteria include:

- Average Transmission Angle Deviation (4)
 - Average transmission angle deviation will be determined by taking the angle deviation of the linkage when oriented towards each of the five targets. This category receives a weight of 4 because transmission angle is not only a specification we are graded on, but also a factor which determines the efficiency of our design.
- Maximum Transmission Angle Deviation (3)
 - Maximum transmission angle deviation is defined as the greatest deviation from a 90 degree transmission angle found throughout a design's range of motion. This category was included to account for links binding in a design, but is weighted less than average deviation because this is only the worst case scenario.
- Total Linkage Length / Weight (5)
 - Total linkage length is determined by the combined joint-to-joint length of the input, coupler and follower links. This specification accounts for the weight of our linkage, which is one of our grading criteria, by not considering the individual weight-saving methods used in each design, which can be optimized later. It is weighted lower than specifications concerning the functional performance of a design, because raw weight is not as impactful as moment of inertia.
- Average Flashlight-Sensor Distance (2)
 - Average flashlight-sensor distance is determined by the distance between the light ray location on the coupler link and a target, for all five targets. This is a criteria we included because we believe it will have an impact on our linkage's speed in hitting targets. However, it is given a low weight of 2 as the true effect of this specification is untested.
- Manufacturability/Assemblability (5)
 - Assemblability and manufacturability are likewise given a weight of 5 to account for the fact that our linkage must be easily modifiable and disassembled for inspection during the testing phase of this project. This category will depend on the time required to manufacture each component, which depends on the complexity of our links. The configuration of links within this design will also affect assemblability, such as if links will overlap each other throughout motion. Note: this is the only criteria with no easily-quantifiable metric, so scoring for category was determined using intuition

- Power Required (4)
 - Power consumption is defined as the maximum power required throughout the linkage's movement, with all designs standardized to the same duration of time to move from target 1 to target 5.
- Moment of Inertia (3)
 - Moment of Inertia is an important metric as a higher moment of inertia will greatly reduce the possible acceleration our linkage can achieve. Hence, keeping moment of inertia as a characteristic when deciding our final design is important, however not of utmost importance due to it not being a hard requirement.
- Friction (1)
 - Too much friction in a linkage design will inhibit performance of the linkage. However, because of the unified joint design, the impact of a relative high or low friction will be evened out as a result. Hence the relative weight of friction is kept at 1.

Data Collection

The above criteria provide us metrics to collect. To proceed with our analysis, we have measured data from all five of our original designs as well as our newly modified one. The results are displayed in Table 2.1.

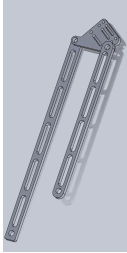
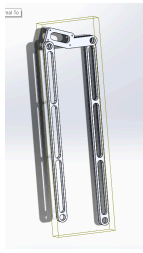
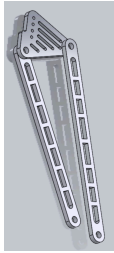
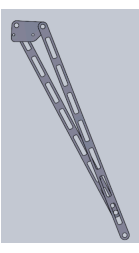
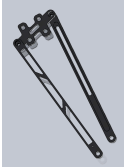
Table 2.1: Measured linkage data for all team members

	Alex	Brady	Ethan	Jay	Yuandi	New Revision
Average Deviation	24.91 deg	1.532 deg	40.514 deg	17.44 deg	23.25 deg	10.10 deg
Abs. Max Deviation	34.25 deg	2.52 deg	78.56 deg	34.63 deg	29.92 deg	20.57 deg
Input Length	8.36 in	14.92 in	9.31in	8.21 in	14.275 in	7.90 in
Coupler Length	2.14 in	2.20 in	2.5 in	2.6 in	1.77 in	2.50 in
Follower Length	11.16 in	15.21in	8.8205 in	9.09 in	15.078 in	7.90 in
Total Length	21.66 in	32.33 in	20.6305 in	19.9 in	31.123 in	18.30 in
Power (ADAMS)	0.95W	0.93W	0.96W	0.99W	0.92W	0.95W
Time of Motion (ADAMS)	0.73 s	0.82 s	0.70 s	0.60 s	0.80 s	0.47 s
Moment of Inertia	129461 gcm ²	166827 gcm ²	95123 gcm ²	87923 gcm ²	145648 gcm ²	67071 gcm ²
Average Flashlight to Sensor Distance	10.45 in	9.52 in	13.87 in	14.54 in	11.63 in	14.75 in

Selection Process

With the above data, we were able to quantify each design's relative performance in each of the 9 categories. Scores were assigned by selecting the design with median performance as reference with score of 0, and basing each of the other designs' relative performance against the median design. Alex's design has been chosen as a median design. The ranks of the pugh chart are decided by the measured values in the Table 2.1 above.

Table 2.2: Pugh Chart

Table 2.2: Pugh Chart		Design					
Sketch Thumbnails→							
Criteria	Weight	Alex	Brady	Ethan	Jay	Yuandi	New Rev.
Average Transmission Angle Deviation	4	0	+1	-1	+1	0	+1
Maximum Transmission Angle Deviation	3	0	+1	-1	0	0	+1
Total Linkage Length / Weight	5	0	-1	0	+1	-1	+1
Average Flashlight-Sensor Distance	2	0	+1	0	-1	+1	-1
Manufacturability	5	0	-1	+1	-1	+1	0
Power (ADAMS)	4	0	0	-1	+1	0	+1
Time of Motion (ADAMS)	4	0	-1	0	+1	-1	+1
Moment of Inertia	3	0	+1	0	0	-1	0
Friction	1	0	-1	0	+1	+1	0
Total	20	0	2	-2	+7	-4	+14

According to the Pugh chart, Jay's design has the highest score, but our team has decided to take Brady's linkage to reduce transmission deviation angle. Brady's linkages are constructed using two circular paths with the same radii. However, as shown in the Pugh chart, the downside of Brady's design is the total linkage length and the weight. Both input and follower linkages are longer than the other designs, which might result in the

obstacle obstructing the flashlight. Jay's design has a good amount of transmission angle deviation, and the total length is very short.

Our team has decided to integrate both designs to make a new design that has very low deviation angle and short link lengths. This new design has been introduced to the same Pugh chart above under "New Rev.", and is better than our median design for most of the criteria. A sketch of the linkage kinematics is shown below in Figure 2.1.

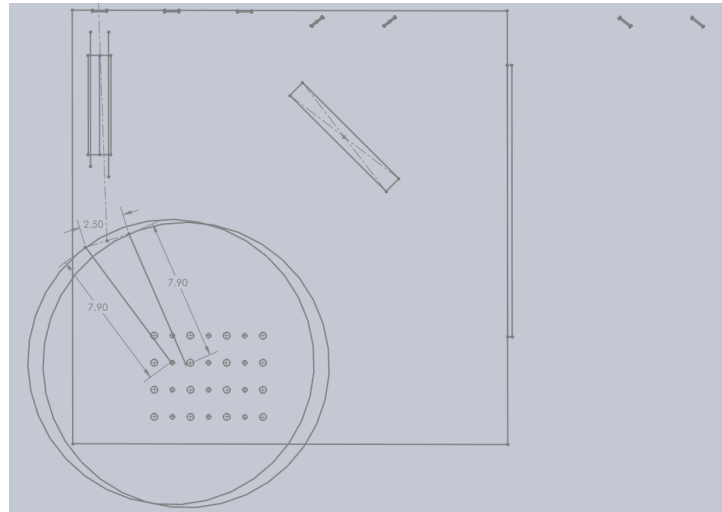


Figure 2.1: Playing field of the New Revision linkage that has both input and follower path the radius of 7.9 in

The new revision design also has two equivalent circular paths for input and follower links, as following Brady's calculations. Our team has successfully shortened the input and the follower linkages after, while maintaining the low linkage lengths. The minimum linkage lengths that we could achieve without losing large deviation angles was 7.9 inches in length for both linkages, as in Figure 2.1 above.



Figure 2.2: Trapezoid shaped slots that are middle on the input link design in order to reduce mass

Weight is another important feature that we needed to consider, so our team has improved the linkage design to reduce the mass and possible failures. All of the initial linkage design by everyone has rectangular slots in the middle, as in Figure 2.2 above. Ethan's linkage has larger slots than the other linkages, which has lower

weight. After several research, our team has decided to have trapezoid slots instead of rectangular ones, which will support the sides of the input link, and have light weight.



Figure 2.3: I-beam shaped follower link design to prevent failure by the stopper and reduce moment of inertia

Furthermore, to have more clearance around the linkages, we have decided to attach stoppers below the base plate. Since the follower link will be hit by the stoppers often and with significant force, it should be more rigid and have less possibility of failure. Our team has chosen an I-beam cross-section design for the follower link as in Figure 2.3, which features two bosses (one on either side of the linkage), rather than lightening holes. These bosses are essentially slots that do not cut all the way through the linkage and extend the entire length of the linkage. In addition, there will be one lightening slot (thru slot) further away from the ground, which will allow the link to be further light-weighted. The location of the slot was chosen because less material is needed in this portion of the link, and the mass reduction farther from the ground pivot is further beneficial to reducing the moment of inertia. In general, the I-beam also has lower moment of inertia compared to other links with similar lengths.

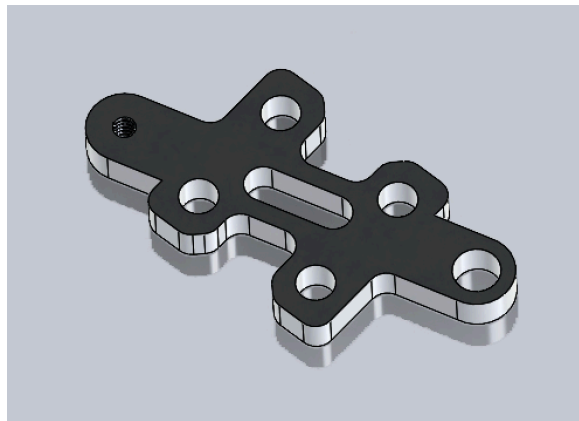


Figure 2.4: Coupler design with angled bars to mount the flashlight efficiently

The shape of the coupler has changed as well. The updated coupler design is in Figure 2.4 above. Since the flashlight is not perpendicular to the coupler. Ethan and Brady's coupler designs are thinner than others, which will have lower weight than other coupler designs. However, we have to mount the flashlight rigidly to the

coupler, we created the design with one bar connected to two bars around the flashlight in angle. This will be easily manufactured and the design is relatively simple while reducing the mass.

Conclusion

Our team has conducted a new revision design, based on Jay's linkage design, but taking Brady's transmission angle calculations. The new revision design has input and output linkages that are both 7.9 in length, and the coupler linkage is 2.5 in. The power output by this linkage is 0.95 W, which is simulated in ADAMS. This value is less than the maximum target power, 1W. The average deviation angle is 10.10 degrees, and the maximum angle is 20.57 degrees. The moment of inertia is 67076 gcm², which is the lowest we achieved out of all designs.

Section 3: Final Linkage Design and Analysis

Our chosen design from section 2 is the new revision linkage. Given our requirements, the performance of this design against each specification is shown as follows in Table 3.1. The location of our ground pivots is also shown in Figure 3.1

Table 3.1: Performance of Chosen Design According To Critical Numerical Parameters

Power Output (ADAMS)	0.95 W
Time of Motion (ADAMS)	0.47 s
Moment of Inertia	67071 gcm ²
Average Flashlight to Sensor Distance	14.75 in
Angle Deviation at Target 1	6.3 deg
Angle Deviation at Target 2	2.41 deg
Angle Deviation at Target 3	2.11 deg
Angle Deviation at Target 4	19.13 deg
Angle Deviation at Target 5	20.57 deg
Average Deviation	10.10 deg

Input Length	7.9 in
Coupler Length	2.5 in
Follower Length	7.9 in
Total Length	18.3 in

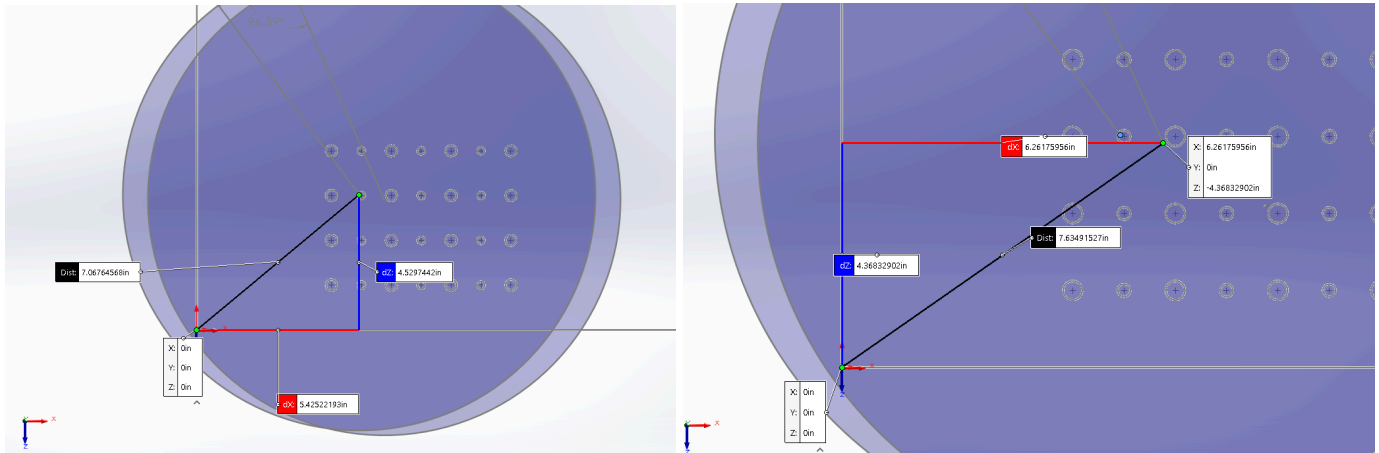


Figure 3.1: Pivot Locations of Chosen Design

The maximum transmission angle deviation of this design is 20.57 degrees, when the mechanism is pointed at target 5. This deviation falls below the maximum deviation angle limit, set by the project description, of 60 degrees. A visualization of the design across its range of motion is shown below in Figures 3.2 to 3.6.

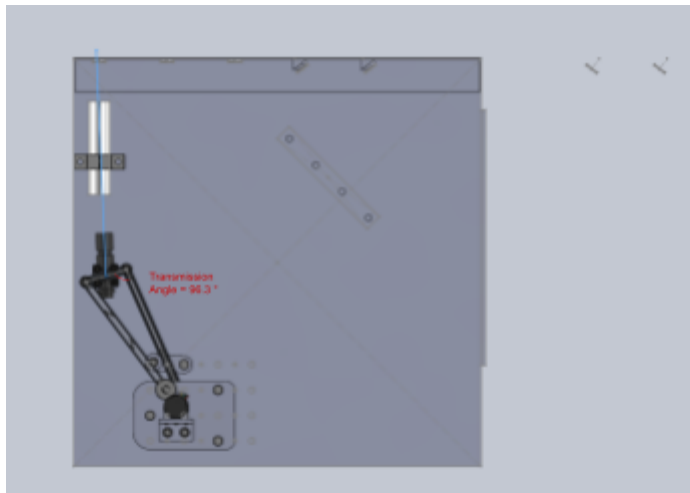


Figure 3.2: Design pointed at Target 1 where the measured transmission angle is 96.3 degrees

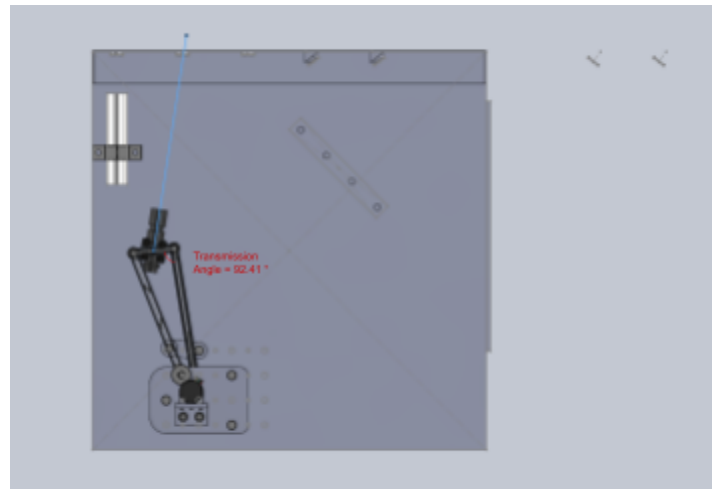


Figure 3.3: Design pointed at Target 2 where the measured transmission angle is 92.41 degrees

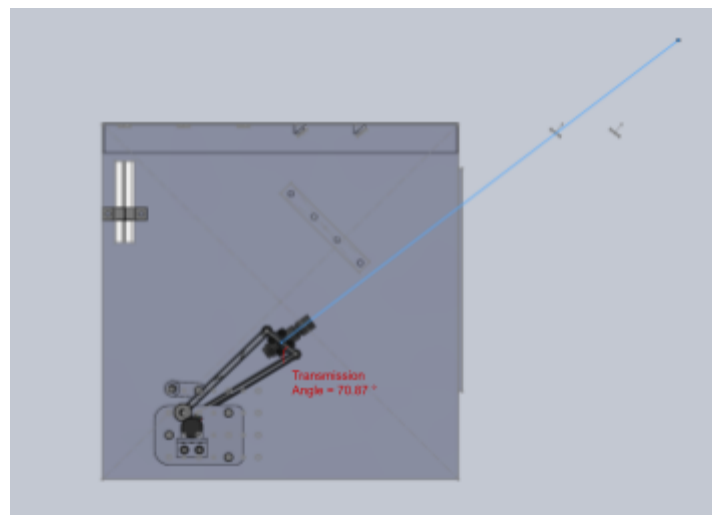
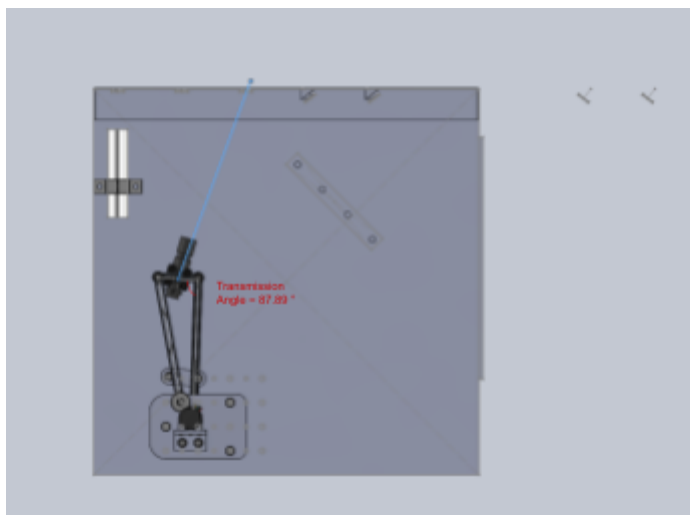


Figure 3.4: Design pointed at Target 3 where the measured transmission angle is 87.89 degrees

Figure 3.5: Design pointed at Target 4 where the measured transmission angle is 70.87 degrees

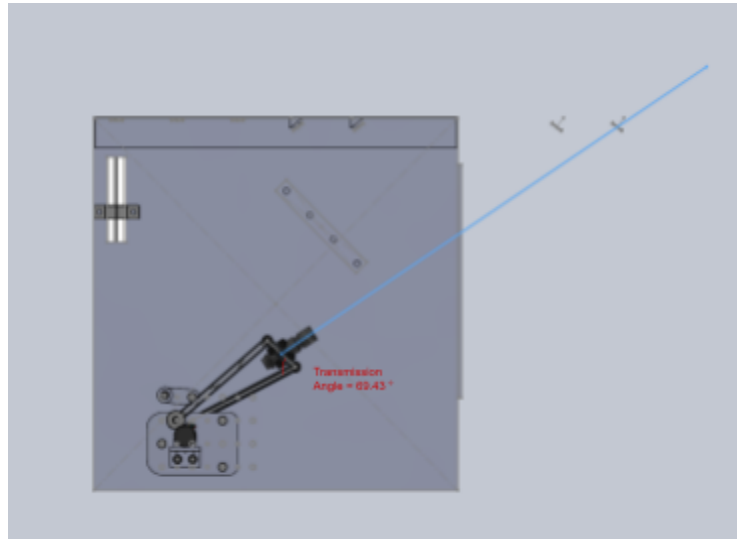


Figure 3.6: Design pointed at Target 5 where the transmission angle is 69.43 degrees

Our design incorporates adjustable hardstops to control our mechanism's range of motion. Figures 3.7 and 3.8 below show the orientation of our linkage against these hardstops, and Figure 3.9 displays an overall view of our entire assembly.

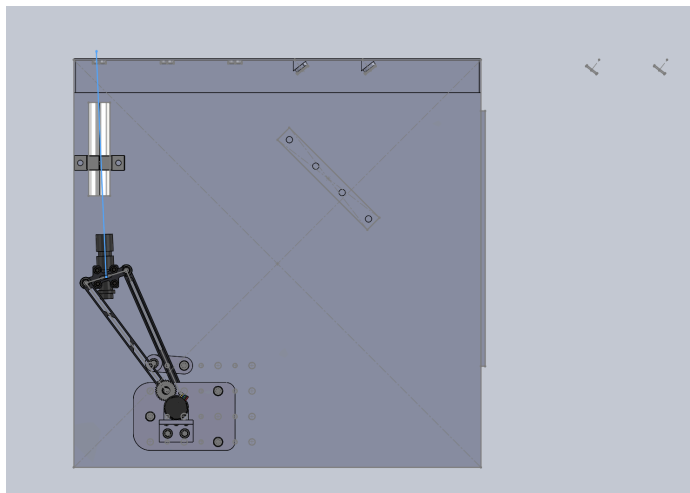


Figure 3.7: Linkage Against Leftmost Hard Stop

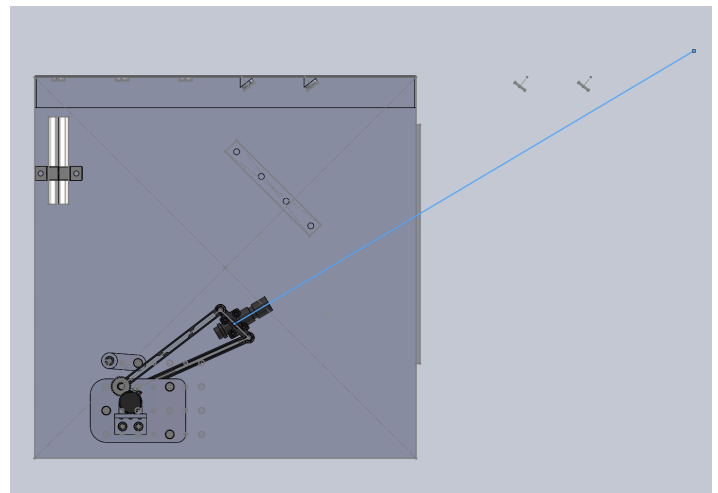


Figure 3.8: Linkage Against Rightmost Hard Stop

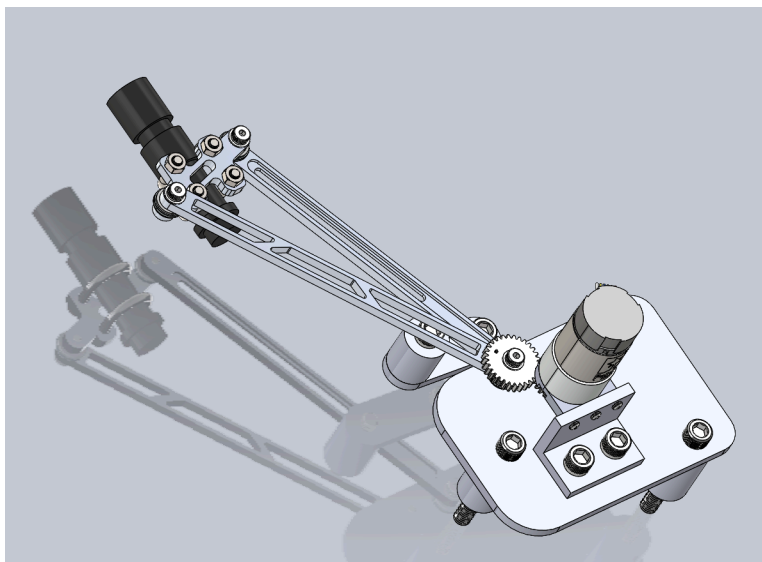


Figure 3.9: Overall Perspective View of Linkage When Mounted To Playing Field

The following are cross-sectional images of each of the linkage's four joints as well as the cross section of the follower's "I-beam" design.

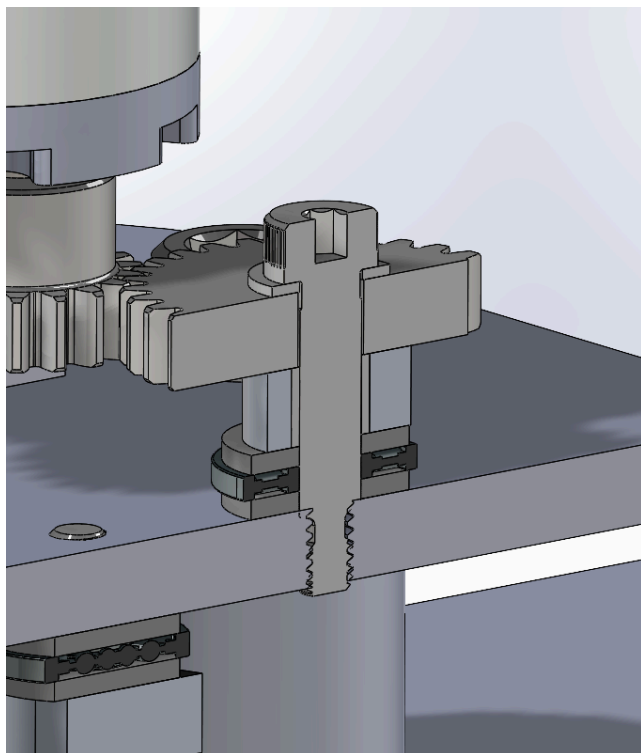


Figure 3.10: Joint between ground and input

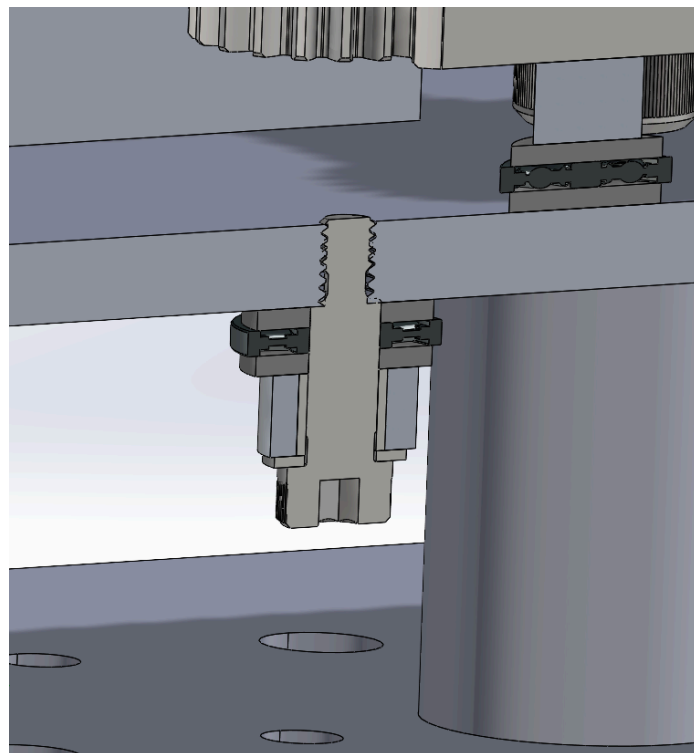


Figure 3.11: Joint between ground and follower

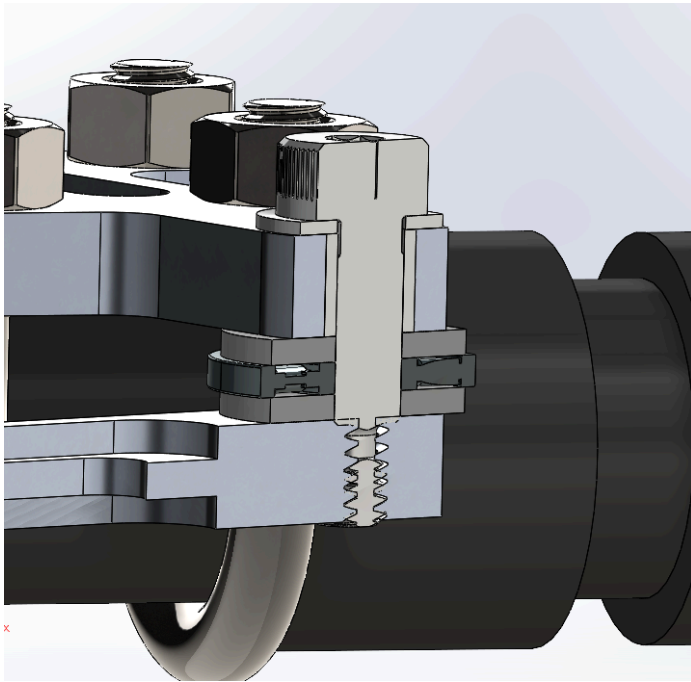


Figure 3.12: Joint between coupler and follower

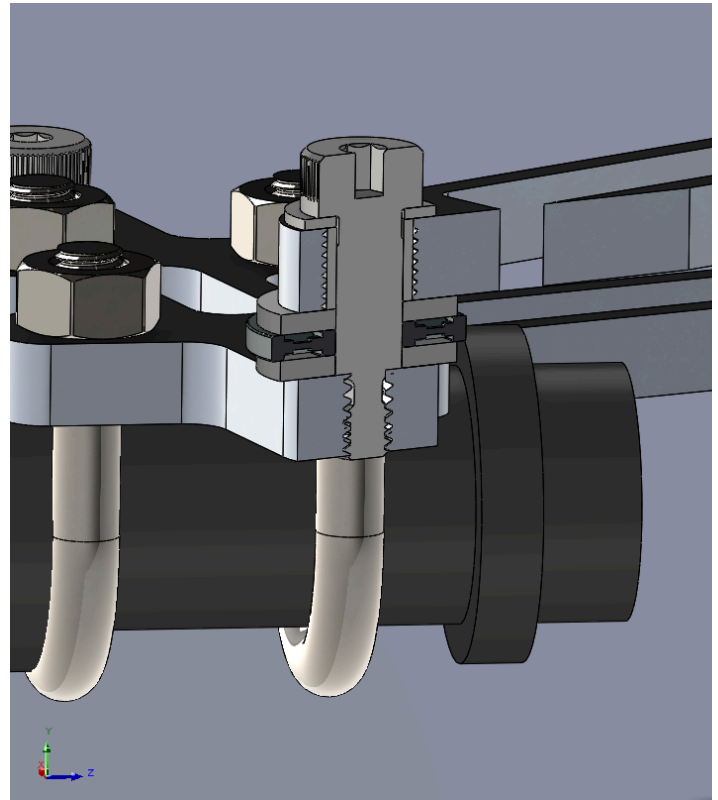


Figure 3.13: Joint between coupler and input

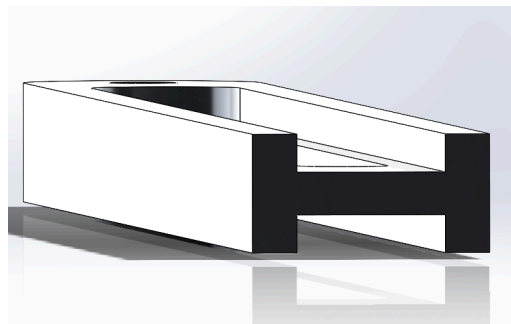


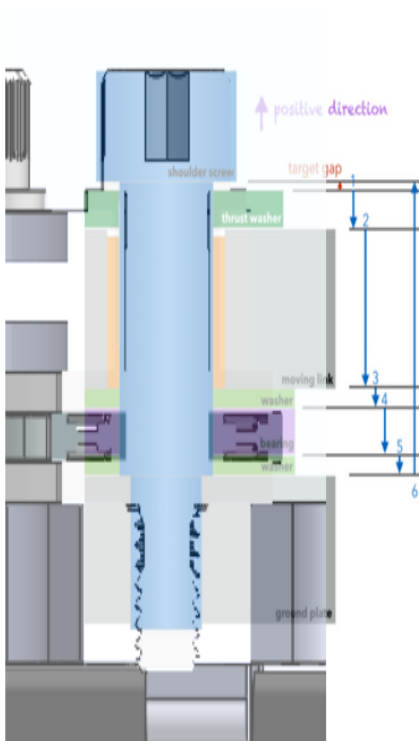
Figure 3.14: Cross section of follower link I-beam shape

Notable Design Features:

- **Hardstops:** Our design uses adjustable hardstops, allowing us to precisely fine tune our starting link position. **The hardstops consist of a small cylinder with an off center hole milled through it. This hole is connected directly to the bolts that are connected to our baseplate. If we want to adjust our hardstops, all we need to do is loosen the bolt slightly, move our hardstop to the desired area, and then re-tighten the bolt.**
- **Material:** Our design uses Aluminum 6061 for all of the links, hardstops, supports, mounts, as well as the baseplate.
- **Coupler + U-bolt mount:** Our design utilizes U-bolts to secure the flashlight to the coupler. This will enable us to constrain the flashlight in all degrees of freedom, allowing our flashlight to be closer to the targets without compromising the orientation of the flashlight.
- **Transmissions:** Our transmission is designed to maximize the angular acceleration of the links. This will enable us to minimize the time required for our link to travel between sensors, allowing us to maximize time spent at each sensor.
- **Linkage design:** Our boss linkage is designed to greatly increase the strength to withstand collisions with hardstops. From the I-beam profile, we can reliably contact the hardstops at faster speeds without risk of failure.

Joint Design

Our joint design is relatively standard and consists of a shoulder bolt, a bolt washer, a sleeve bushing press fit into our link, a needle roller bearing sandwiched in between two oil-embedded washers, and a tapped hole at the bottom of the joint. Through this design, the bushing and shoulder bolt allow the links to rotate freely, and over tightening will not prohibit this rotation due to our tolerance stackup shown below.

Table 3.2: Tolerance stack-up of joint design

Step	Item	Dir	Nom	-Tol	+Tol
1	Military Washer	-1	0.032	0.001	0.001
2	Plate	-1	0.250	0.0012	0.0012
3	Oil Washer	-1	0.032	0.002	0.000
4	Needle Roller Bearing	-1	0.078125	0.0004	0.000
5	Oil Washer	-1	0.032	0.002	0.000
6	Shoulder	+1	0.500	0.005	0.005
Total Gap			0.0785875	0.092475	0.0713875

Weight Estimate

Our weight estimate includes the input link, follower link, coupler link, flashlight, flashlight mounting hardware, rotational joint hardware (including those on the ground plate), transmission hardware attached to the input link, and any other parts of your mechanism that move. [Table 3.3 below](#) details a comprehensive breakdown of our mass estimate for the entire assembly.

Table 3.3: Mass breakdown of entire moving assembly

Part No.	Part Title	Material	Dimension(s) (in)	Quantity	Weight Per (lbs)	Weight (lbs)
1	Input link	Aluminum	0.5 x 7.9 x 0.25	1	0.050057	0.050057
2	Coupler link	Aluminum	3 x 1 x 0.25	1	0.043048	0.043048
3	Follower link	Aluminum	0.5 x 7.9 x 0.25	1	0.057550	0.057550
12	Motor (pinion) gear	Brass	20mm x 20mm x 14mm (18 teeth)	1	0.046800	0.046800
13	Link gear	Brass	32mm x 32mm x 6mm (30 teeth)	1	0.074500	0.074500
14	Spring pin	Spring Steel	1/16 x 1/16 x 9/16	1	0.000287	0.000287
15	Set screw	Brass	M4 x 7mm	1	0.000000	0.000000
16	U bolt	Steel	M6 x 20mm x 50mm	2	0.030455	0.060910
18	Shoulder screw	Alloy Steel	10-24 screw x 0.5	4	0.013199	0.052796
22	U bolt nut	Steel	10mm x 10mm x 5mm (M6)	4	0.000000	0.000000
25	Thrust bearing	Alloy Steel	0.687 x 0.687 x 0.074 (1/4)	4	0.005163	0.020652
26	Bushing	841 Bearing Bronze	5/16 x 5/16 x 1/4 (1/4)	4	0.002114	0.008456
27	Bearing washer	841 Bearing Bronze	5/8 x 5/8 x 1/16 (1/4)	8	0.004830	0.038640
28	Screw washer	Cadmium Yellow-Chromate Plated Steel	0.468 x 0.468 x 0.032 (1/4)	4	0.001125	0.004500
29	Battery	Lithium	AA size	1	0.000000	0.000000
30	Flashlight	-	1 x 1 x 3.8	1	0.140000	0.140000

Weight Estimate:

0.598196 lbs

ADAMS Model

Given the finalized geometry of our linkage, we are able to construct a preliminary model in ADAMS to verify the dynamic behavior of our design. Figure 3.15 below shows the setup we used for our ADAMS model.

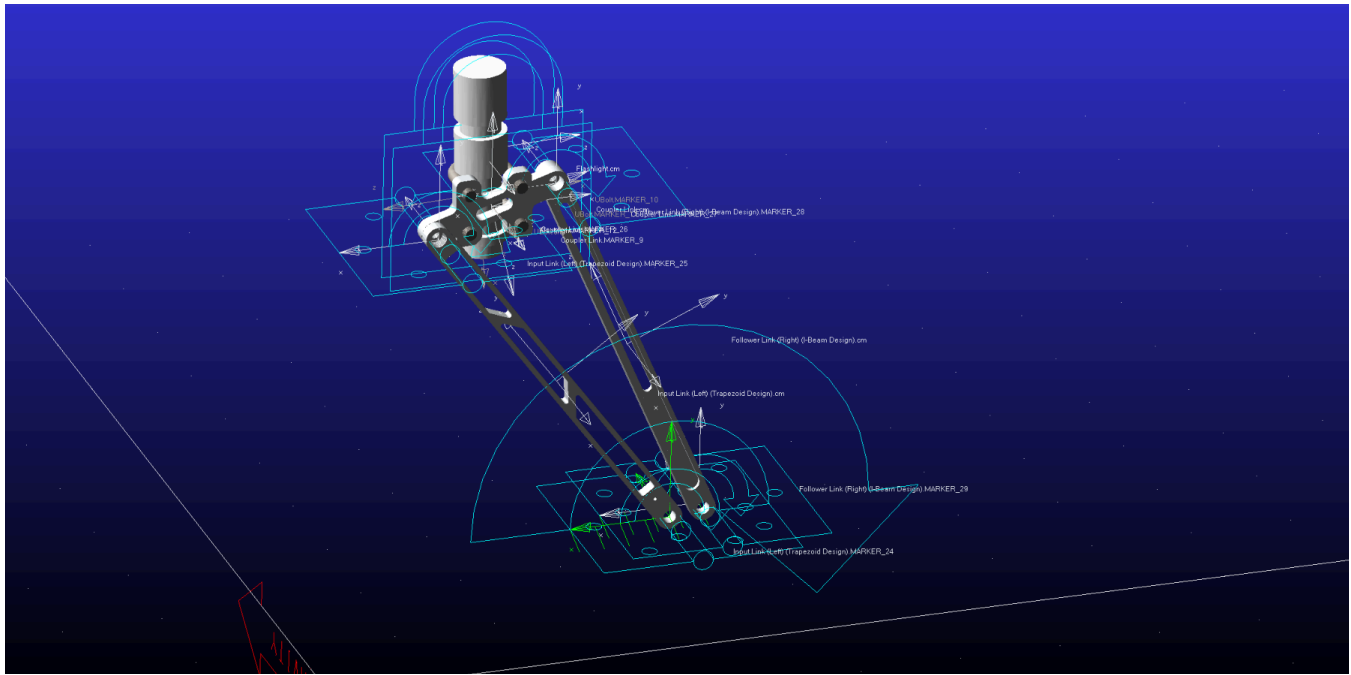


Figure 3.15: ADAMS Model

To model our design's behavior, our chosen acceleration profile simulates the 1549 degrees/sec² corresponding to a simulation time of 0.47 seconds. This simulation yields the following plots for angular position, angular velocity, power and torque on the input (left most) link, as in Figure 3.15 above.

$$\alpha = \frac{4\theta}{t^2}$$

The angular acceleration is calculated based on the equation above. The angle of deviation is measured in the SOLIDWORKS CAD model, and it is from the angle movement by the input link from the positions that aim target 1 to target 5. The measured angle from the cad is 85.5 degrees. The time used is optimized based on the maximum input link power, given as 1 W. After several simulations from ADAMS, time 0.47 results in the highest possible power below 1 W.

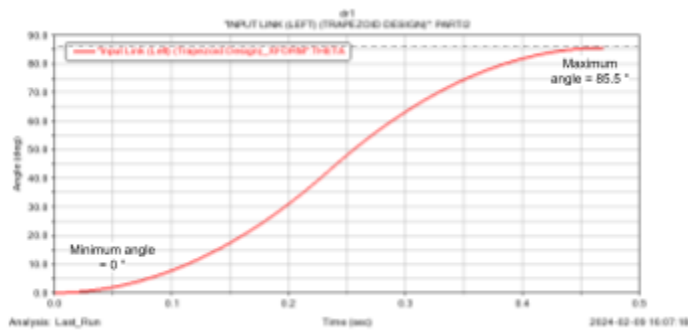


Figure 3.16: Input Link Angular Position; our design requires the input link to travel 85.5 degrees to sweep from target 1 to target 5. We expect our angles to oscillate upon reaching the final endpoint as we cannot precisely reach 85.5 degrees while having no angular velocity.

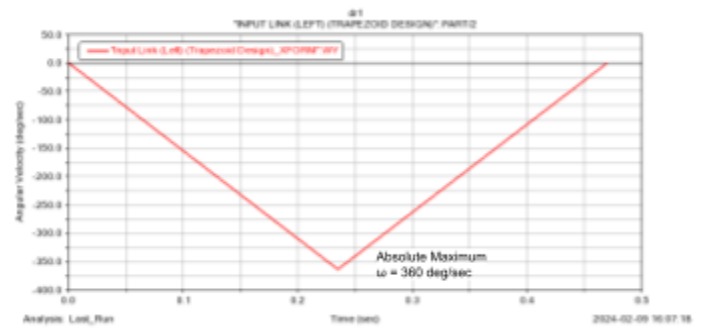


Figure 3.17: Input Link Angular Velocity; from this model, our input link reaches a maximum angular velocity of 360 degrees/second. We expect that the input will not model a linear line but more quadratic in real life due to the behavior of the motor.

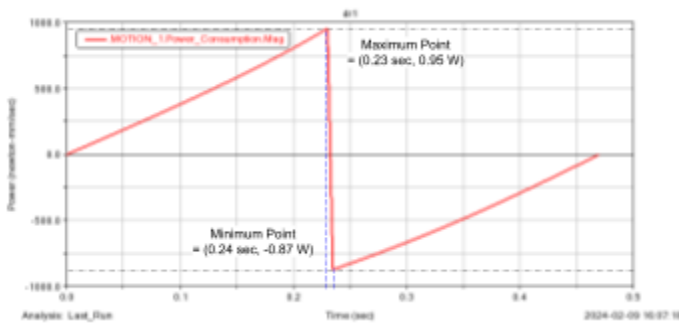


Figure 3.18: Input Link Power; from this model, our input link reaches a max at 0.95 W. The sharp decline observed in the diagram is our motor changing directions. The minimum point is -0.87 W. We expect that to obtain the desired acceleration that we want, we will need to run our motors at a higher power to overcome forces of friction and overshooting targets.

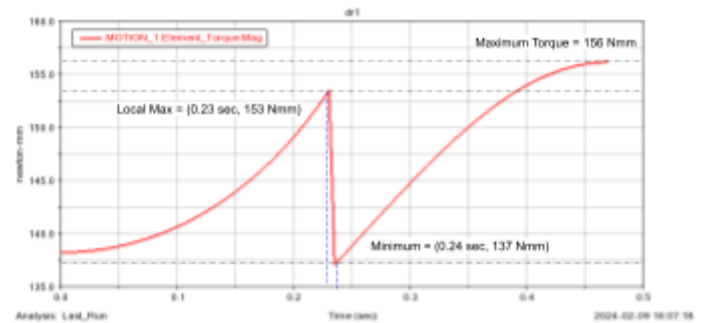


Figure 3.19: Input Link Torque; from this model, our input link reaches a local max at 153 Nmm and an absolute max at 156 Nmm. This spike indicates our motor changing directions, hence the direction of the torque output is changing. We expect that in real life we may have a period of oscillating torques to help stop our linkage at the desired stopping point.

The results of these graphs show that the input linkage will operate with a linearly changing velocity for the first half of the motion. Once half the distance has been traveled (after 0.235 seconds), the linkage will operate in a different but again linearly changing velocity. Corresponding to each half of this motion is a constant angular acceleration. So, the first half of the motion has a constant angular acceleration. The second half has the same magnitude of angular acceleration just in the opposite direction. The maximum angular position of the input link is 85.5 degrees, and the maximum angular velocity is 360 degrees/sec (absolute value of the minimum velocity in Figure 3.17).

Notable behavior includes the sudden decrease in the power and torque on the input link. The power increases at a constant rate overall, except for 0.23 to 0.24 seconds, where the power spikes from 0.95 W to -0.87 W. This spike is the motor reversing its motion rapidly to decelerate the linkage to a stop. The input link torque has also

dropped from 0.153 Nm to 0.137 Nm during this time as well. ~~In real life, these spikes will not happen, and the power and torque will continue to increase.~~

One major difference between the ADAMS simulation and real life is friction. We neglected the friction of our linkage which we will most certainly experience and have to account accordingly. While our model assumes the linkage will be perfectly stopped at the desired location, from this friction and other idealizations made in our model that occur in real life, we will need to apply PID controls to help dampen the motion of the linkage so that it is in the proper position.

Section 4: Motion Generator Revision:

After Gate Review 1, our team has changed the input link design in order to reduce mass of the total linkage. Since the input link will be stopped by the limit switch instead of the hard stop, we have changed the I-beam design to the design with trapezoidal slots. The weight of the input link is reduced from 0.0576 lbs to 0.0501 lbs. The link lengths were unchanged after Gate Review 1.

Another change is the stopper locations. Our previous linkage design did not consider the gear sizes and positions, so having stoppers at the top of the base plate will not give enough space for the gears. This made us mount the stoppers at the bottom of the base plate, instead of the top.

These changes are included in Design Review 1 contents, so the ADAMS simulation results will be the same.

First, in order to conduct a power analysis, the total inertia should be calculated, including the motor and the gear. These are the inertia of each component measured in SOLIDWORKS.

Table 4.1 Inertia of rotational parts that is measured on SOLIDWORKS

Part Name	Inertia (g cm ²)
Input link	993.35
Coupler link	941.09
Follower link	1164.01
Link gear	3.92
Pinion gear	0.07

Using parallel axis theorem, the total inertia of input link and the link gear can be calculated as:

$$\begin{aligned}
 J_{total\ input} &= J_{input} + J_{link\ gear} + m_{link\ gear} (r_{input\ to\ gear\ cg})^2 \\
 &= 993.35 + 3.92 + 3.2581 (0.3175)^2 = 997.59\ g\ cm^2
 \end{aligned}$$

All linkage calculations are done as in [section 6](#). Choosing the largest inertia out of all 5 target positions, the final inertia of the linkages will be:

$$J_{linkages} = 67075.37\ g\ cm^2$$

The inertia of the pinion gear is very low, so it can be ignored. The motor inertia is approximated to 25000 g cm² in this calculation.

Adding up all the inertia,

$$\begin{aligned}
 J_{total} &= J_{linkages} + m_{linkages} (r_{pivot\ to\ motor})^2 + J_{motor} \\
 &= 67075.37 + 271.33(6.2738)^2 + 25000 = 102755.07\ g\ cm^2
 \end{aligned}$$

The next step is to get the velocity profile of the load (linkage in this case). Assuming the triangular model as below:

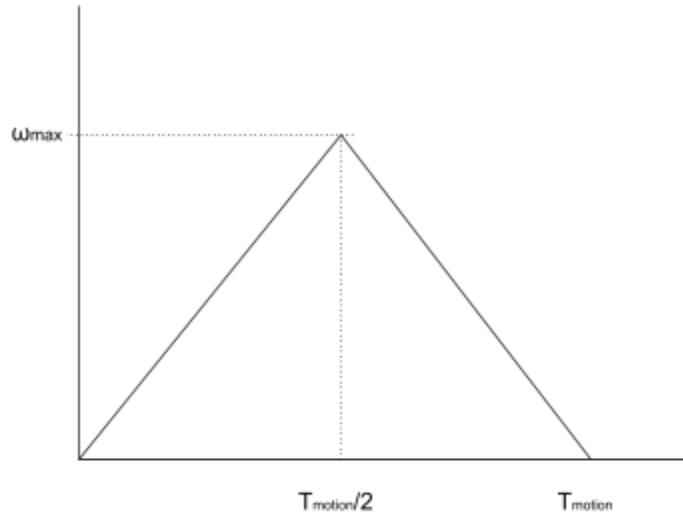


Figure 4.1 Triangular velocity profile of the linkages during a single motion

Area under the curves will be θ , which is the angle of the input link, starting from the initial position. t_{motion} is the time of motion, and $\omega_{max,load}$ is the maximum angular velocity.

$$\theta = \frac{t_{motion} \times \omega_{max,load}}{2}$$

$$\omega_{max,load} = \frac{2\theta}{t_{motion}}$$

Integrating this equation, we can get the required acceleration of the system.

$$\alpha_{max,load} = \frac{2\omega_{max,load}}{t_{motion}}$$

Then, torque on load can be calculated using this required acceleration above:

$$T_{load} = J \times \alpha_{max,load} = J \times \frac{2\omega_{max,load}}{t_{motion}} = J \times \frac{2 \times \frac{2\theta}{t_{motion}}}{t_{motion}} = \frac{4J\theta}{t_{motion}^2}$$

As was determined by the beam resolution calculation in section 5, our team chose to use the gear ratio of 4. This relates the torque on the load and torque on the motor.

$$N = \frac{T_{load}}{T_{motor}} = 4 = \frac{\omega_{motor}}{\omega_{load}}$$

This gives the velocity constraint of the motor, $\omega_{motor} = 4\omega_{load}$.

Using the torque velocity curve equation, $T_m = T_s + K\omega_{motor}$, we can relate the load torque to the load angular velocity:

$$\frac{T_{load}}{4} = T_s + K(4\omega_{load})$$

Let the stall torque at the nominal supply voltage to be T_{nom} . As the supply voltage changes, the stall torque changes to:

$$\frac{T_{s,new}}{T_{nom}} = \frac{V_{s,new}}{V_{nom}}$$

$$T_s = V_{s,new} \frac{T_{nom}}{V_{nom}}$$

This ends up as the equation below, applying the supplier's nominal voltage (12 V) and stall torque (14 kg cm = 1.373 Nm) given.

$$\frac{T_{load}}{4} = V_{s,new} \frac{T_{nom}}{V_{nom}} + 4K\omega_{load}$$

$$\frac{T_{load}}{4} = V_{s,new} \frac{1.373}{12} + k(4\omega_{load}) = 0.1144 V_{s,new} + 4K\omega_{load}$$

In the motor spec sheet, the no-load speed is 330 RPM. Using these three quantities, we can calculate the motor constant K.

$$0 = 1.373 + K(330)$$

$$K = \frac{1.373 \text{ Nm}}{330 \text{ rpm}} = -0.004161 \text{ Nm/rpm}$$

Therefore, assuming that the new input voltage is 10 V, torque of the motor can be calculated in terms of θ . For the ADAMS simulation in section 3, the time of travel $t_{motion} = 0.47$ seconds is used. With the current design, the maximum angle change from initial position is:

$$\theta = 139.68 - 52.20 = 87.48 \text{ deg} = 87.48 \times \frac{\pi}{180} = 1.5268 \text{ rad}$$

Substituting all the known values, the smallest possible t_{motion} is:

$$\frac{T_{load}}{4} = 0.1144 V_{s,new} + 4K\omega_{load} = 0.1144(10) + 4(-0.004161 \text{ Nm/rpm})\left(\frac{2\theta}{t_{motion}}\right)$$

$$\frac{4I}{t_{motion}^2} \frac{1}{4} = 1.144 \text{ Nm} - 0.016644 \text{ Nm/rpm} \left(\frac{30 \text{ s rpm}}{\pi \text{ rad}}\right) \left(\frac{2(1.5268 \text{ rad})}{t_{motion}}\right)$$

$$\frac{(1.5268)(102755.07 \text{ g cm}^2)}{t_{motion}^2} = 1.144 - 0.016644 \text{ Nm/rpm} \left(\frac{30 \text{ s rpm}}{\pi \text{ rad}}\right) \left(\frac{2(1.5268 \text{ rad})}{t_{motion}}\right)$$

$$\frac{0.015688 \text{ kg m}^2}{t_{motion}^2} = 1.144 - \frac{0.48533 \text{ Nm s}}{t_{motion}}$$

$$1.144 t_{motion}^2 - 0.48533 t_{motion} - 0.015688 = 0$$

$$t_{motion} = \frac{0.48533 \pm \sqrt{0.48533^2 - 4(1.144)(-0.015688)}}{2(1.144)} = 0.454 \text{ (since } t_{motion} > 0)$$

ADAMS simulation uses time of motion $t = 0.47$ as simulated multiple times to achieve the power right below 1 W. This discrepancy happens because there are the differences in rotational inertia of the system. The ADAMS model includes all the links, an approximation of flashlight, and U bolts. However, the calculation includes the inertia of the gears and the motor as well, which gives different approximation values than the ADAMS model. Furthermore, the calculation takes linkage inertia value at target 5, but in the calculation sheet in section 6, it shows that the inertia of linkages are different depending on which target the linkage is

targeting. On the other hand, the ADAMS model will be able to account for the change in inertia along the path, so the result is different from the calculation model.

The percentage difference between them is:

$$\% \text{ difference} = \frac{0.47 - 0.454}{0.454} \times 100\% = 3.43 \%$$

The percentage difference between simulated and theoretical values is 3.43%, which is lower than 5%. This verifies that the time frame used in ADAMS simulation is valid.

Section 5: Energy Conversion Introduction:

Why is a transmission needed: Transmissions in general are devices used to transfer mechanical power from one location to another. A transmission is needed to alter the speed and torque output, the location of motion/force, and the direction and positioning of an output with respect to an input.

What are the effects of transmissions on speed, torque, and/or other important quantities for the project:

Typically, motors spin with too much speed and not enough torque. Thus, a common usage of a transmission is to reduce the speed of the output and raise the torque using a gear ratio greater than 1. This gear ratio is the ratio of the diameter of the driven gear divided by the diameter of the driving gear. Hence a large (greater than 1) gear ratio allows the conversion of speed into torque and vice versa. In this project, we want a higher torque output, which allows a higher angular acceleration. To minimize the travel time needed between photosensors, we want to maximize the angular acceleration of the output load which comes from our choice in transmission and gear ratio.

What are the limits of any general transmission design: A common limitation of any transmission design is the input power or input torque and speed. While a transmission is useful for converting torque into speed and vice versa, it's also important to consider whether the load can be supported by the input supplied. For transmissions with DC motors, these inputs are a product of both the motor specifications and the voltage supplied to the motor. Another limitation for any general transmission is the positioning and availability of components. Gears, in order to mesh, require that their diametral pitch must be the same. Thus, choosing gears for a general transmission requires this diametral pitch factor to be accounted for, which also requires size and positioning considerations. [Another important limitation is the transmission efficiency. Transmissions are not perfectly efficient because they lose energy as they operate, and there are three main ways in which this occurs as it applies to our project. In a geared transmission, friction between the gear teeth as they mesh is a notable source of energy loss. In addition, energy can be lost as the gears are compressed into one another. A final source of transmission inefficiency comes from the inefficiencies of bearings and washers that are needed for the transmission to operate.](#)

What are the design limits for the transmissions in this project: One of the main design limitations for our transmission in this project is the gear ratio. We want to minimize the travel time between photosensors, which demands the maximum angular acceleration. Hence, one of the transmission constraints we had was to fulfill this requirement. Another limitation we had for the transmission was the DC motor supply voltage, which was held constant at 9 volts. This caps the available speed and torque we can drive our linkage at. Another limitation we needed to consider was the availability of gears. While our analysis that determined the optimal gear ratio provided us with a decimal value, we could only consider gears that had convenient ratios. Another limitation we had was the space allocated for our baseplate and linkage. We had other important elements, such as hard stops, motor mounts, etc. in our design that we needed to consider to avoid collisions.

Section 6: Transmission Ratio and Type Determination:

Inertia Matching

Methods and Validation:

- To accomplish the objectives of the project, we must be able to move our linkage with adequate control with the use of a transmission ratio. Therefore, we need to model the load that our mechanism is applying to the motor as the linkage moves throughout the playing field. As the mass and inertia distribution of the mechanism varies relative to its center of rotation, the torque required from the motor might increase or decrease as it rotates. Therefore, with the use of motor equations and Solidworks, we modeled the maximum reflected moment of inertia at any of the 5 target positions to calculate the necessary torque and transmission ratio to move the linkage at an adequate speed.
- Firstly, our entire analysis revolves around the optimal Transmission Ratio equation and the parallel axis theorem,

$$N = \sqrt{\frac{I_L}{I_m}} \quad I = I_{cm} + m \cdot r^2$$

which was derived using the start-up torque of the motor and mass moment of inertia. I_m is

approximated to be $25000 \text{ g}\cdot\text{cm}^2$ as specified in Lab 5. To calculate I_L , we worked on Solidworks to sketch and calculate the total mass of inertia of the linkage at our point of rotation in each of the 5 target positions.

- Following this equation, we used Solidworks to calculate the moment of inertia of the input link and output link. We did the same to calculate the moment of inertia of the coupler which changed according to the linkages position.
- Finally, to find the center of mass of the entire linkage at its axis of rotation, we measured the distance between the instant centers of the input and output links connected to the ground and coupler links which we called R1 and R4. We then extended the lines that connect these 4 ICs to find the axis of rotation. We then added new distance measurements from input-output-coupler joints and center of mass of the coupler to the axis of rotation point, which we labeled R2, R4 and R5 respectively. We repeated this process for the next 4 Target Positions. Moreover, we calculated the reflected moment of inertia of each of the links to each axis of rotation at each of the target positions using the Parallel Axis Theorem.

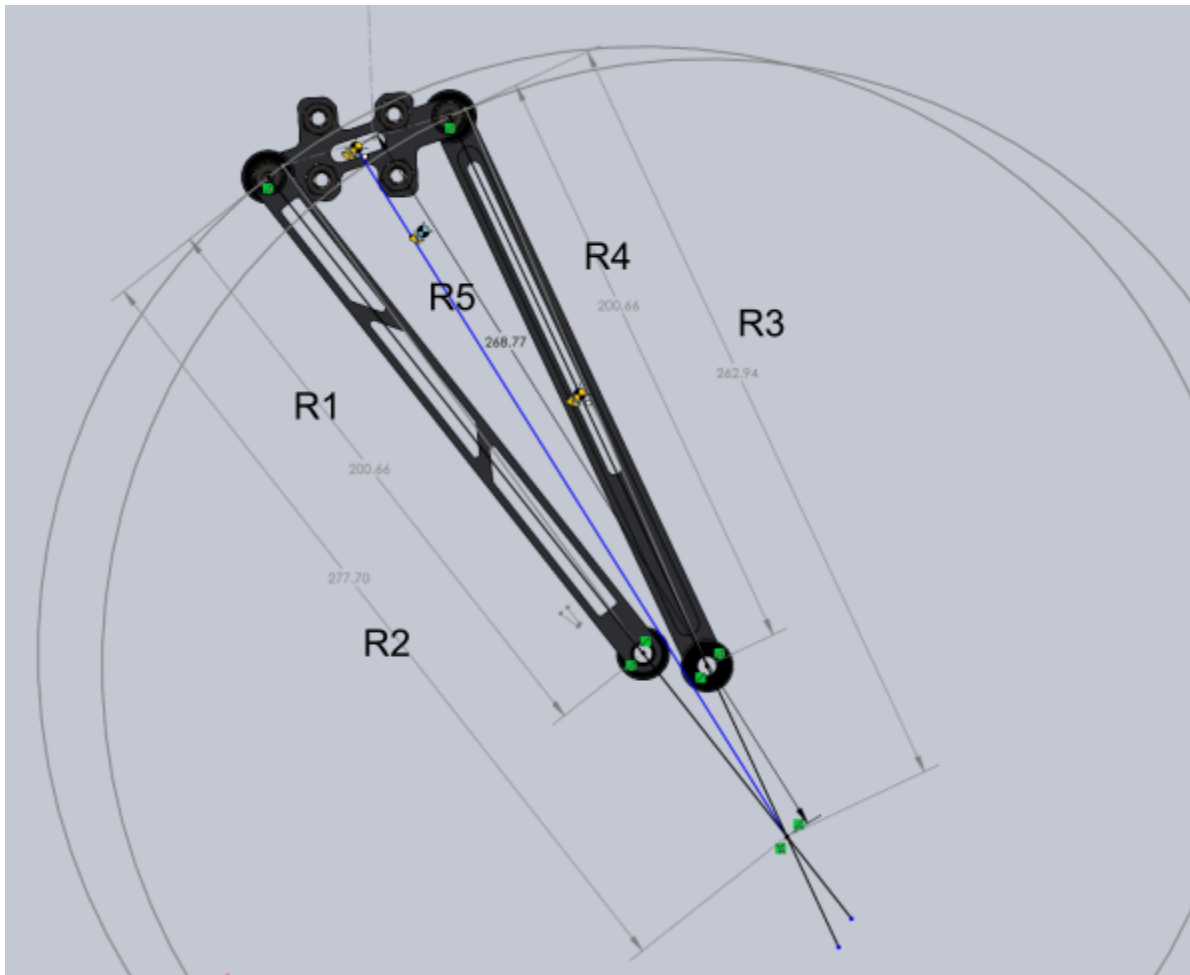


Figure 6.1: Overall sketching of R1, R2, R3, R4 and R5 measurements for Target Position 1.

- After finding all these distances, we then computed the load moment of inertia at its rotation point using the following equation:

$$\frac{1}{2}I_{total}\omega_{input}^2 = \frac{1}{2}I_{input}\omega_{input}^2 + \frac{1}{2}I_{coupler}\left(\frac{R_1\omega_{input}}{R_2}\right)^2 + \frac{1}{2}I_{output}\left(\frac{R_1R_3\omega_{input}}{R_2R_4}\right)^2$$

$$I_{total} = I_{input} + I_{coupler}\left(\frac{R_1}{R_2}\right)^2 + I_{output}\left(\frac{R_1R_3}{R_2R_4}\right)^2$$

The following table will show the results of these calculations for all the five target positions.

Table 6.1: 5 Linkage Positions Chart

Position 1	Position 2	Position 3	Position 4	Position 5																																																																																																																								
Geometric Variables <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>R1 (input link)</td><td>20.5</td><td>cm</td></tr><tr><td>R2</td><td>27.77</td><td>cm</td></tr><tr><td>R3</td><td>26.29</td><td>cm</td></tr><tr><td>R4 (output link)</td><td>2.57</td><td>cm</td></tr><tr><td>R5</td><td>26.88</td><td>cm</td></tr></table> <table><tr><th>Transmis. Angle</th><th></th><th></th></tr><tr><td></td><td>83.7</td><td>Deg</td></tr></table>	Variable	Value	Units	R1 (input link)	20.5	cm	R2	27.77	cm	R3	26.29	cm	R4 (output link)	2.57	cm	R5	26.88	cm	Transmis. Angle				83.7	Deg	Geometric Variables <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>R1</td><td>20.07</td><td>cm</td></tr><tr><td>R2</td><td>29.62</td><td>cm</td></tr><tr><td>R3</td><td>28.7</td><td>cm</td></tr><tr><td>R4</td><td>20.07</td><td>cm</td></tr><tr><td>R5</td><td>28.99</td><td>cm</td></tr></table> <table><tr><th>Transmis. Angle</th><th></th><th></th></tr><tr><td></td><td>87.6</td><td>Deg</td></tr></table>	Variable	Value	Units	R1	20.07	cm	R2	29.62	cm	R3	28.7	cm	R4	20.07	cm	R5	28.99	cm	Transmis. Angle				87.6	Deg	Geometric Variables <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>R1</td><td>20.07</td><td>cm</td></tr><tr><td>R2</td><td>30.4</td><td>cm</td></tr><tr><td>R3</td><td>30</td><td>cm</td></tr><tr><td>R4</td><td>20.07</td><td>cm</td></tr><tr><td>R5</td><td>30.03</td><td>cm</td></tr></table> <table><tr><th>Transmis. Angle</th><th></th><th></th></tr><tr><td></td><td>87.9</td><td>Deg</td></tr></table>	Variable	Value	Units	R1	20.07	cm	R2	30.4	cm	R3	30	cm	R4	20.07	cm	R5	30.03	cm	Transmis. Angle				87.9	Deg	Geometric Variables <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>R1</td><td>20.07</td><td>cm</td></tr><tr><td>R2</td><td>25.08</td><td>cm</td></tr><tr><td>R3</td><td>26.76</td><td>cm</td></tr><tr><td>R4</td><td>20.07</td><td>cm</td></tr><tr><td>R5</td><td>25.74</td><td>cm</td></tr></table> <table><tr><th>Transmis. Angle</th><th></th><th></th></tr><tr><td></td><td>70.8</td><td>Deg</td></tr></table>	Variable	Value	Units	R1	20.07	cm	R2	25.08	cm	R3	26.76	cm	R4	20.07	cm	R5	25.74	cm	Transmis. Angle				70.8	Deg	Geometric Variables <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>R1</td><td>20.07</td><td>cm</td></tr><tr><td>R2</td><td>22.86</td><td>cm</td></tr><tr><td>R3</td><td>24.82</td><td>cm</td></tr><tr><td>R4</td><td>20.07</td><td>cm</td></tr><tr><td>R5</td><td>23.65</td><td>cm</td></tr></table> <table><tr><th>Transmis. Angle</th><th></th><th></th></tr><tr><td></td><td>69.43</td><td>Deg</td></tr></table>	Variable	Value	Units	R1	20.07	cm	R2	22.86	cm	R3	24.82	cm	R4	20.07	cm	R5	23.65	cm	Transmis. Angle				69.43	Deg
Variable	Value	Units																																																																																																																										
R1 (input link)	20.5	cm																																																																																																																										
R2	27.77	cm																																																																																																																										
R3	26.29	cm																																																																																																																										
R4 (output link)	2.57	cm																																																																																																																										
R5	26.88	cm																																																																																																																										
Transmis. Angle																																																																																																																												
	83.7	Deg																																																																																																																										
Variable	Value	Units																																																																																																																										
R1	20.07	cm																																																																																																																										
R2	29.62	cm																																																																																																																										
R3	28.7	cm																																																																																																																										
R4	20.07	cm																																																																																																																										
R5	28.99	cm																																																																																																																										
Transmis. Angle																																																																																																																												
	87.6	Deg																																																																																																																										
Variable	Value	Units																																																																																																																										
R1	20.07	cm																																																																																																																										
R2	30.4	cm																																																																																																																										
R3	30	cm																																																																																																																										
R4	20.07	cm																																																																																																																										
R5	30.03	cm																																																																																																																										
Transmis. Angle																																																																																																																												
	87.9	Deg																																																																																																																										
Variable	Value	Units																																																																																																																										
R1	20.07	cm																																																																																																																										
R2	25.08	cm																																																																																																																										
R3	26.76	cm																																																																																																																										
R4	20.07	cm																																																																																																																										
R5	25.74	cm																																																																																																																										
Transmis. Angle																																																																																																																												
	70.8	Deg																																																																																																																										
Variable	Value	Units																																																																																																																										
R1	20.07	cm																																																																																																																										
R2	22.86	cm																																																																																																																										
R3	24.82	cm																																																																																																																										
R4	20.07	cm																																																																																																																										
R5	23.65	cm																																																																																																																										
Transmis. Angle																																																																																																																												
	69.43	Deg																																																																																																																										
Total Inertias of a Link (via Parallel Axis) <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>linput</td><td>3315.3 67256</td><td>(gram*cm^2)</td></tr><tr><td>loutput</td><td>3822.6 44256</td><td>(gram*cm^2)</td></tr><tr><td>lcoupler</td><td>10071 5.6616</td><td>(gram*cm^2)</td></tr></table>	Variable	Value	Units	linput	3315.3 67256	(gram*cm^2)	loutput	3822.6 44256	(gram*cm^2)	lcoupler	10071 5.6616	(gram*cm^2)	Total Inertias of a Link (via Parallel Axis) <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>linput</td><td>3315.3 67256</td><td>(gram*cm^2)</td></tr><tr><td>loutput</td><td>3822.6 44256</td><td>(gram*cm^2)</td></tr><tr><td>lcoupler</td><td>11503 3.5343</td><td>(gram*cm^2)</td></tr></table>	Variable	Value	Units	linput	3315.3 67256	(gram*cm^2)	loutput	3822.6 44256	(gram*cm^2)	lcoupler	11503 3.5343	(gram*cm^2)	Total Inertias of a Link (via Parallel Axis) <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>linput</td><td>3315.3 67256</td><td>(gram*cm^2)</td></tr><tr><td>loutput</td><td>3822.6 44256</td><td>(gram*cm^2)</td></tr><tr><td>lcoupler</td><td>12336 6.3222</td><td>(gram*cm^2)</td></tr></table>	Variable	Value	Units	linput	3315.3 67256	(gram*cm^2)	loutput	3822.6 44256	(gram*cm^2)	lcoupler	12336 6.3222	(gram*cm^2)	Total Inertias of a Link (via Parallel Axis) <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>linput</td><td>3315.3 67256</td><td>(gram*cm^2)</td></tr><tr><td>loutput</td><td>3822.6 44256</td><td>(gram*cm^2)</td></tr><tr><td>lcoupler</td><td>90886. 34453</td><td>(gram*cm^2)</td></tr></table>	Variable	Value	Units	linput	3315.3 67256	(gram*cm^2)	loutput	3822.6 44256	(gram*cm^2)	lcoupler	90886. 34453	(gram*cm^2)	Total Inertias of a Link (via Parallel Axis) <table><tr><th>Variable</th><th>Value</th><th>Units</th></tr><tr><td>linput</td><td>3315.3 67256</td><td>(gram*cm^2)</td></tr><tr><td>loutput</td><td>3822.6 44256</td><td>(gram*cm^2)</td></tr><tr><td>lcoupler</td><td>76872. 95811</td><td>(gram*cm^2)</td></tr></table>	Variable	Value	Units	linput	3315.3 67256	(gram*cm^2)	loutput	3822.6 44256	(gram*cm^2)	lcoupler	76872. 95811	(gram*cm^2)																																																												
Variable	Value	Units																																																																																																																										
linput	3315.3 67256	(gram*cm^2)																																																																																																																										
loutput	3822.6 44256	(gram*cm^2)																																																																																																																										
lcoupler	10071 5.6616	(gram*cm^2)																																																																																																																										
Variable	Value	Units																																																																																																																										
linput	3315.3 67256	(gram*cm^2)																																																																																																																										
loutput	3822.6 44256	(gram*cm^2)																																																																																																																										
lcoupler	11503 3.5343	(gram*cm^2)																																																																																																																										
Variable	Value	Units																																																																																																																										
linput	3315.3 67256	(gram*cm^2)																																																																																																																										
loutput	3822.6 44256	(gram*cm^2)																																																																																																																										
lcoupler	12336 6.3222	(gram*cm^2)																																																																																																																										
Variable	Value	Units																																																																																																																										
linput	3315.3 67256	(gram*cm^2)																																																																																																																										
loutput	3822.6 44256	(gram*cm^2)																																																																																																																										
lcoupler	90886. 34453	(gram*cm^2)																																																																																																																										
Variable	Value	Units																																																																																																																										
linput	3315.3 67256	(gram*cm^2)																																																																																																																										
loutput	3822.6 44256	(gram*cm^2)																																																																																																																										
lcoupler	76872. 95811	(gram*cm^2)																																																																																																																										
Scale Factors <table><tr><td>R1/R2</td><td>0.716 78571 43</td><td>(unitless)</td></tr><tr><td>(R1*R3)/(R2*R4)</td><td>0.948 92857 14</td><td>(unitless)</td></tr></table>	R1/R2	0.716 78571 43	(unitless)	(R1*R3)/(R2*R4)	0.948 92857 14	(unitless)	Scale Factors <table><tr><td>R1/R2</td><td>0.677 5827 144</td><td>(unitless)</td></tr><tr><td>(R1*R3)/(R2*R4)</td><td>0.968 9399 055</td><td>(unitless)</td></tr></table>	R1/R2	0.677 5827 144	(unitless)	(R1*R3)/(R2*R4)	0.968 9399 055	(unitless)	Scale Factors <table><tr><td>R1/R2</td><td>0.660 19736 84</td><td>(unitless)</td></tr><tr><td>(R1*R3)/(R2*R4)</td><td>0.986 84210 53</td><td>(unitless)</td></tr></table>	R1/R2	0.660 19736 84	(unitless)	(R1*R3)/(R2*R4)	0.986 84210 53	(unitless)	Scale Factors <table><tr><td>R1/R2</td><td>0.800 2392 344</td><td>(unitless)</td></tr><tr><td>(R1*R3)/(R2*R4)</td><td>1.066 9856 46</td><td>(unitless)</td></tr></table>	R1/R2	0.800 2392 344	(unitless)	(R1*R3)/(R2*R4)	1.066 9856 46	(unitless)	Scale Factors <table><tr><td>R1/R2</td><td>0.877 9527 559</td><td>(unitless)</td></tr><tr><td>(R1*R3)/(R2*R4)</td><td>1.085 7392 83</td><td>(unitless)</td></tr></table>	R1/R2	0.877 9527 559	(unitless)	(R1*R3)/(R2*R4)	1.085 7392 83	(unitless)																																																																																										
R1/R2	0.716 78571 43	(unitless)																																																																																																																										
(R1*R3)/(R2*R4)	0.948 92857 14	(unitless)																																																																																																																										
R1/R2	0.677 5827 144	(unitless)																																																																																																																										
(R1*R3)/(R2*R4)	0.968 9399 055	(unitless)																																																																																																																										
R1/R2	0.660 19736 84	(unitless)																																																																																																																										
(R1*R3)/(R2*R4)	0.986 84210 53	(unitless)																																																																																																																										
R1/R2	0.800 2392 344	(unitless)																																																																																																																										
(R1*R3)/(R2*R4)	1.066 9856 46	(unitless)																																																																																																																										
R1/R2	0.877 9527 559	(unitless)																																																																																																																										
(R1*R3)/(R2*R4)	1.085 7392 83	(unitless)																																																																																																																										
Total Inertia <table><tr><td>ltotal</td><td>58503. 39617</td><td>(gram*cm^2)</td></tr></table>	ltotal	58503. 39617	(gram*cm^2)	Total Inertia <table><tr><td>ltotal</td><td>59718. 24065</td><td>(gram*cm^2)</td></tr></table>	ltotal	59718. 24065	(gram*cm^2)	Total Inertia <table><tr><td>ltotal</td><td>60808. 59236</td><td>(gram*cm^2)</td></tr></table>	ltotal	60808. 59236	(gram*cm^2)	Total Inertia <table><tr><td>ltotal</td><td>65869. 34333</td><td>(gram*cm^2)</td></tr></table>	ltotal	65869. 34333	(gram*cm^2)	Total Inertia <table><tr><td>ltotal</td><td>67075. 37037</td><td>(gram*cm^2)</td></tr></table>	ltotal	67075. 37037	(gram*cm^2)																																																																																																									
ltotal	58503. 39617	(gram*cm^2)																																																																																																																										
ltotal	59718. 24065	(gram*cm^2)																																																																																																																										
ltotal	60808. 59236	(gram*cm^2)																																																																																																																										
ltotal	65869. 34333	(gram*cm^2)																																																																																																																										
ltotal	67075. 37037	(gram*cm^2)																																																																																																																										

- The maximum load inertia of 67075.37037 gram*cm ^2 occurs at Position 5. Therefore our calculated transmission ratio is:

$$N = \sqrt{\frac{I_L}{I_m}} = \sqrt{\frac{67075.37}{25000}} = 1.64$$

Light Beam Resolution

Methods and Validation:

- Besides being able to move our linkage adequately with our transmission ratio, we also want to ensure our mechanism positions itself to light the sensors effectively. Therefore, to maximize our light beam resolution, we had to perform several calculations around the recommendation that the Desired Count Range of the encoder has to be at least 30. A higher desired count range benefits us because it allows us to reach a wider range of values used to hit the targets. However, a higher Desired Count Range would make our mechanism very slow as it increases the transmission ratio. Therefore, we gauged our calculations considering light beam resolution and linkage (motor) speed.
- Our calculations and conclusions revolved around the Desired Count Range Equation:

$$\text{Desired Count Range} = \left(\frac{64 \text{ counts}}{1 \text{ rev}} \right) \cdot 30 \cdot \left(\frac{\text{smallest difference [deg]}}{\frac{360 \text{ degrees}}{1 \text{ rev}}} \right) \cdot N$$

- The equation is composed of our encoder based resolution measured in counts per resolution CPR, gearbox ratio, max-min target angle ranges, and transmission ratio. In Solidworks, we measured the max and min angles where the light beam hits the target in each of the 5 positions, and then calculated the difference between these two angles. We selected the minimum difference amongst the 5 positions and put them into the Desired Count Range equation.
- The following table shows this step of our procedure by setting 30 as our Desired Count Range.

Table 6.2: Light Beam Resolution Angle Ranges

Position	Minimum Angle [deg]	Maximum angle [deg]	Difference
1	52.2	55.2	3
2	66.94	69.84	2.9
3	81.59	84.34	2.75
4	133.27	134.97	1.7
5	138.2	139.68	1.48

$$N = (30) \left(\frac{1 \text{ rev}}{64 \text{ counts}} \right) \frac{1}{30} \left(\frac{\frac{360 \text{ degrees}}{1 \text{ rev}}}{1.48 \text{ [deg]}} \right) = 3.8$$

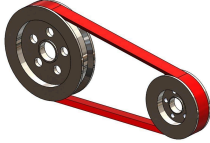
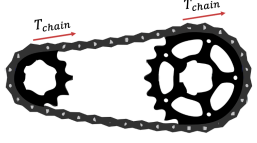
Final Transmission Ratio

- The Transmission Ratio chosen to move on with our project is 3.8. The two Transmission Ratios calculated using these methods are clearly different. Inertia Matching provides great insight as to what the minimum transmission ratio would be needed for the motor to move the linkage at an operating point and constant speed. The Light Beam Resolution calculations sacrifice speed for a higher torque and accuracy. The speed at which our linkage moves is important, however, based on our project's objectives, the speed loss with the transmission ratio of 3.8 wouldn't be detrimental for our performance as our linkage will still move very quickly between the different target positions.
- There are possible differences we can encounter when we test the transmission. Firstly, the friction generated between the joints will definitely take upon a role between the torque transmission throughout the linkage. Moreover, the effect of gravity will play a role in generating some vertical play in the coupler of our linkage as well our flashlight beam hitting the targets effectively. Some actions that we can take to mitigate these issues is to add different fasteners to our joints to create a more stable transmission assembly or to elevate the entire linkage by 1-2 inches to hit the targets.

Transmission Type

- There are three main transmission mechanisms available to us: Belt & Pulleys, Gears, and Chains & Sprockets. The Gear Transmissions allow us to change the speed or rotation with the use of different gears with different properties. They are more efficient than Belts and Chains. Belt & Pulleys work by mounting a belt onto the pulleys. They allow the designer to select the center of distance between the shafts (unlike gears). Chains & Sprockets transmit motion and force via the chain from one sprocket to another. Chain can transmit higher torques than belts can but they behave similarly when selecting the center distance between the shafts.

Table 6.3: Transmission Type Pugh Chart

		Transmission Type		
Images				
Criteria	Weight	Belt & Pulleys	Gears	Chains & Sprockets
Manufacturability	3	0	1	0
Weight	1	0	1	-1
Moment of Inertia	1	0	1	0
Friction	3	0	1	0
Assembly	2	0	1	0
Total	10	0	10	-1

(1) Chosen Selection Criteria

- We selected the following criteria considering how each transmission type would allow us to perform in the project. We considered including manufacturability and assembly as two criteria points as it would allow us to determine how easy it would be to machine and assemble the transmission. Friction, Moment of Inertia and Weight were other factors we considered as each of these different elements might affect the transmission ratio that we computed analytically.

(2) Weighting Scheme

- We considered Manufacturability and Friction as our most important elements. Our team first wanted to find an effective way to implement the transmission system into our linkage by manufacturing something in a simple and cost effective manner. Friction is also an element we considered very important to minimize in our design as it would allow our linkage to rotate perfectly effectively. In our second tier of importance, we selected how easy it is to assemble the transmission system. We're basically considering time-effort efficiency. We also included Weight and Moment of Inertia as our least important elements, as they can affect our transmission ratio calculations.

(3) Values Assigned

- From the Pugh Chart, we can determine that Gears were selected as our most efficient transmission. Gears are very easy to manufacture and assemble. We only needed to drill the spring pin hole and insert the spring pin connecting into the input link. It is correctly implemented, it greatly reduces friction. Finally, their weight and mass moment of inertia are considerably small that they don't affect our transmission ratio calculations.

Following our detailed analysis above, our final gears were purchased from McMaster-Carr. Our 16-tooth brass pinion gear can be found under the part ID 2664N407 and our 60-tooth metal gear can be found under the part ID 2664N328.

Section 7: Final Transmission Design:

Components from Final CAD Model: Our designed transmission consists of two flat spur gears attached to the input link and motor shaft. A set screw transfers torque between the motor shaft and the pinion gear, and a spring pin transfers torque from the spur gear to the input link.

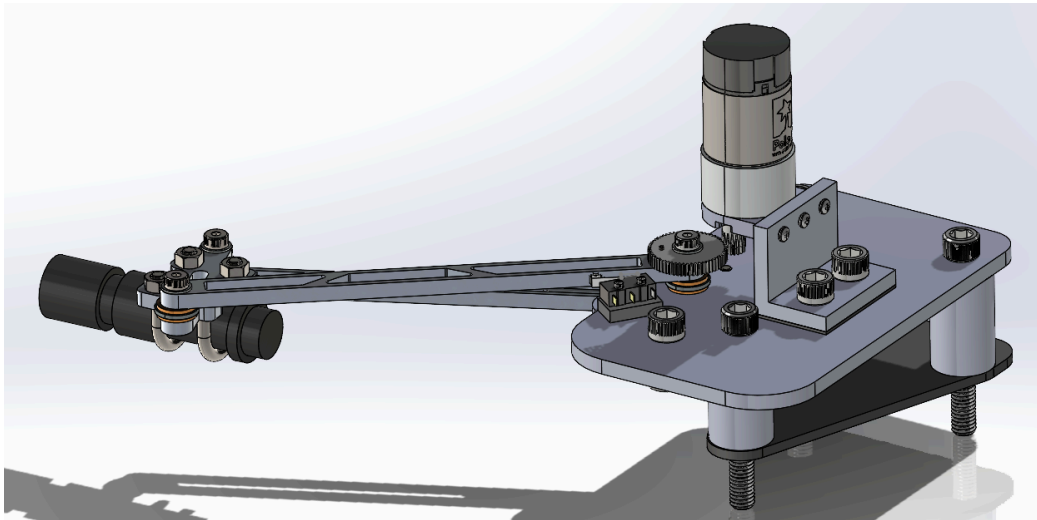


Figure 7.1: Overall linkage assembly with transmission and motor attached

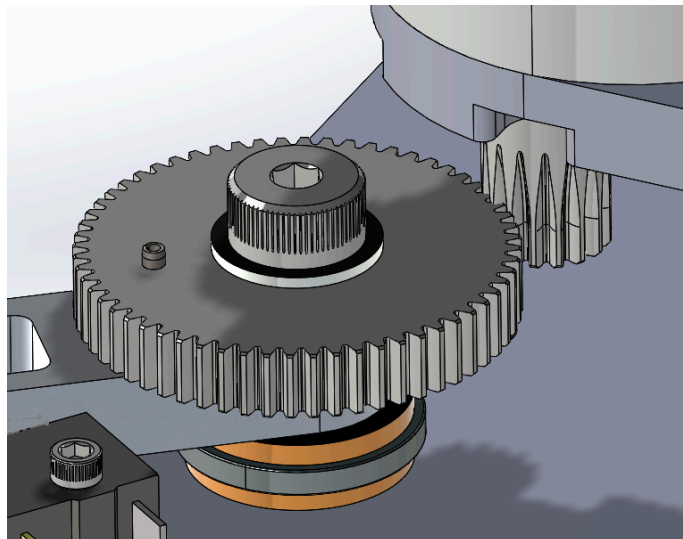


Figure 7.2: Pinion and spur gear interface

Motor Mounting: We package our spur gear above the input link as this allows better access to interface with a pinion gear. To raise our motor to the necessary height, we designed and machined a mount out of the provided angle stock. This mount attaches to the baseplate with a nut underneath, and also screws into the provided motor face bracket. The attachment points between the motor mount and baseplate are two sets of free fit holes – this allows for a degree of adjustability when installing the transmission by allowing the mount to be shifted around and rotated. Further adjustability, however, was unnecessary for our design as the gears already mesh well in CAD (meshing was also physically validated after assembly), and the clearance holes primarily serve to not overconstrain the mount.

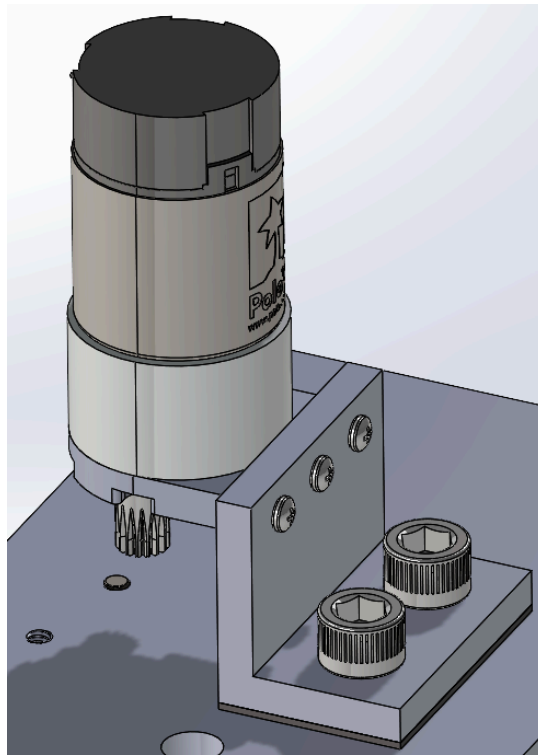


Figure 7.3: Motor mounted to base plate

To better align the pinion and spur gears vertically, we are also able to adjust the height of the motor with a 3D-printed spacer. This spacer sits between the mount and the baseplate.

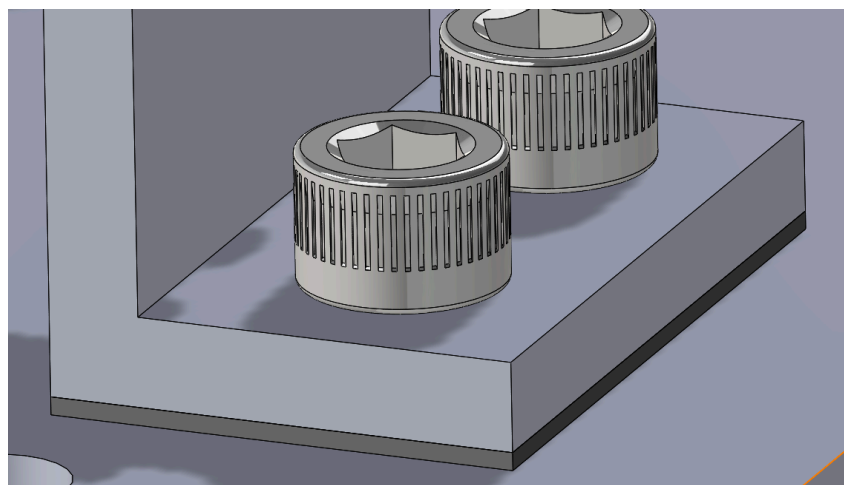


Figure 7.4: 3D-printed spacer (black) placed between motor mount and baseplate

Gear Selection: Our selected gears are one 60-tooth carbon steel gear and one 16-tooth brass gear. Metal gears were chosen over plastic gears because of their improved machinability and robustness. This was particularly important for our spur gear as we needed to drill a spring pin hole to bond this to the input link, and also because we wanted to press in a bronze bushing for a close fit with the input joint.

Overall, metal gears were the most conservative option to avoid gear failure, although at the cost of increased mass. To reduce the effects of added mass, however, and to also reduce rotational inertia in our system, we chose the smallest diameter gears available that still met our gear ratio requirement.

Torque Transferring Joints: The two torque-transferring joints within our transmission are a set screw attaching the pinion to the motor shaft, and a spring pin attaching the spur gear to the input link. The set screw was best suited for our motor shaft as our selected pinion gear by default comes with this option, and also because the output torque on the shaft would be comparatively less than the torque on the spur gear. As such, a spring pin was best suited to transmit torque to the input shaft as the input link was expected to require more torque.

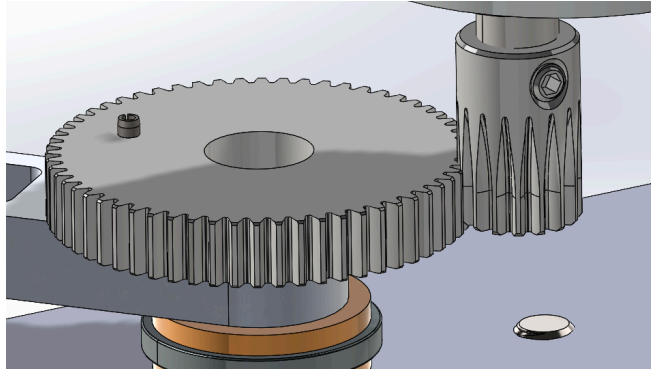


Figure 7.5: Torque-transferring joints at the pinion and spur gear

Mass Estimate: For off-the-shelf parts, we estimate mass using provided specifications from the manufacturer. For custom designed parts, we estimate mass by measuring the volume of each part's CAD and taking into account its material. The gears in our transmission are the only moving components, and as such we will only consider these mass additions for the performance of our mechanism.

Table 7.1: Transmission Mass Estimate

Part Name	Material	Weight
Pinion Gear	Brass	0.0151 lb
Spur Gear	Carbon Steel	0.0718 lb
Set Screw	Steel	0.003 lb
Spring Pin	Spring Steel	0.003 lb
Bronze Bushing	Bronze	0.003 lb

Total Mass Addition: 0.0959 lb

Section 8: Torque Transfer Analysis:

In our transmission design, there are two main joints where torque is being transferred. Torque is first transferred between the motor and transmission via set screw, which when tightened, applies a friction force that allows the pinion and the motor shaft to move together. Torque is also transferred from the transmission and onto the links via a spring pin. These two joints combined allow the linkage assembly to be driven and controlled by the motion of the motor shaft.

In order to assess the functionality and reliability of these joints, first principles analysis should be performed. If a joint were to fail, the linkage mechanism would become inoperable and thus the project goals would not be met. Therefore, validating the torque joints is necessary.

In order to assess the set screw joint, the maximum torque from the motor shaft onto the pinion can be used to find the maximum tangential friction force required for the two bodies to rotate together. We can then calculate the maximum tangential friction force available between the set screw and the motor shaft (given the static coefficient of friction and the normal force applied) and compare the values to one another. Failure would occur when the maximum tangential force required for the set screw to “stick” is **larger** than the maximum tangential friction force available.

The moment balance analysis for the motor shaft is below.

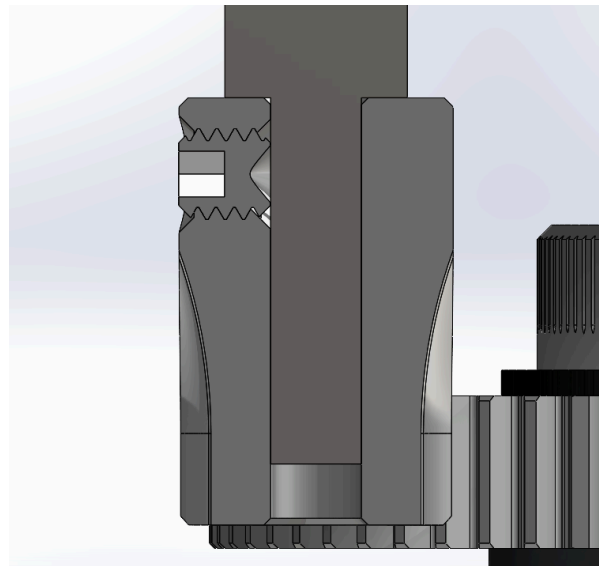


Figure 8.1: Cross sectional view of motor to transmission joint

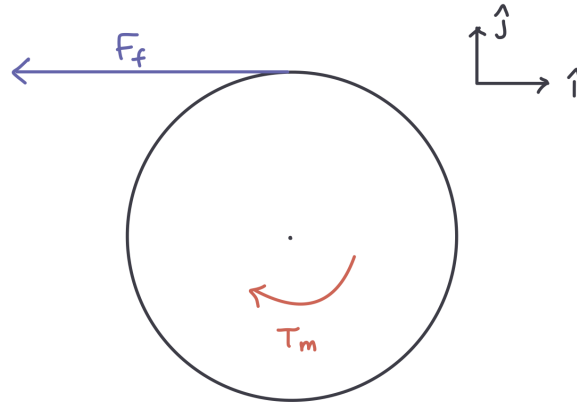


Figure 8.2: Moment diagram of motor shaft. Normal and reaction forces are neglected in this diagram since they don't contribute to the moment about the center.

In finding the maximum tangential force:

$$T_{m, max} = r_{motor shaft} \times F_{tangential}$$

$$F_{tangential} = \frac{T_{m, max}}{r_{motor shaft}}$$

The maximum torque $T_{m, max}$ is to be calculated.

From gear ratios, we know that:

$$\frac{T_m}{T_L} = \frac{\eta N_m}{N_L}$$

Plugging in gear ratio values for N_m and N_L , 156 Nmm (maximum linkage torque given in the ADAMS plot in Figure 3.19) for T_L , and assuming a 90% gear ratio efficiency η , the calculated linkage torque is shown as:

$$T_m = \frac{\eta N_L T_m}{N_L} = \frac{(0.90)(16)(156 \text{ Nmm})}{(60)} = 37.44 \text{ Nmm}$$

The radius of the motor shaft $r_{motor shaft}$ is 3 mm as it was lathed down to fit in our pinion gear. Thus, the maximum tangential force between the motor shaft and the inner diameter of the pinion gear is calculated below:

$$F_{tangential} = \frac{T_{m, max}}{r_{motor shaft}} = \frac{37.44 \text{ Nmm}}{3 \text{ mm}} = 12.48 \text{ N}$$

However, the ADAMs model is underpowered compared to reality. To account for this, a safety factor of 5 will be applied to the tangential force.

$$F_{tangential} \times SF = 62.5 \text{ N}$$

In finding the available friction force, we use the below equation:

$$F_{friction} = \mu_s N_{set\ screw}$$

Assuming that both the motor shaft and the set screw are made of steel, the coefficient of friction μ_s is 0.5-0.8 according to the Engineering Toolbox. Taking the worst case scenario, the following analysis will assume a coefficient of friction $\mu_s = 0.5$.

The normal force applied by the set screw onto the motor shaft, $N_{set\ screw}$, can be approximated using the Umbrako set screw table. Our set screw is an M3 screw, which corresponds to a value of 710 Newtons on the Umbrako metric data set.

$$F_{friction} = \mu_s N_{set\ screw} = (0.5)(710) = 355\ N$$

From this calculation, we see that the set screw friction is more than enough to secure the motor shaft to the pinion, with an *additional* safety factor of $SF = \frac{355}{62.5} = 5.69$.

We can confidently say that the set screw joint has sufficient friction to keep the motor shaft and pinion rotating in unison.

In order to assess the spring pin joint, the spring pin can be modeled as a bolt experiencing single plane shear.

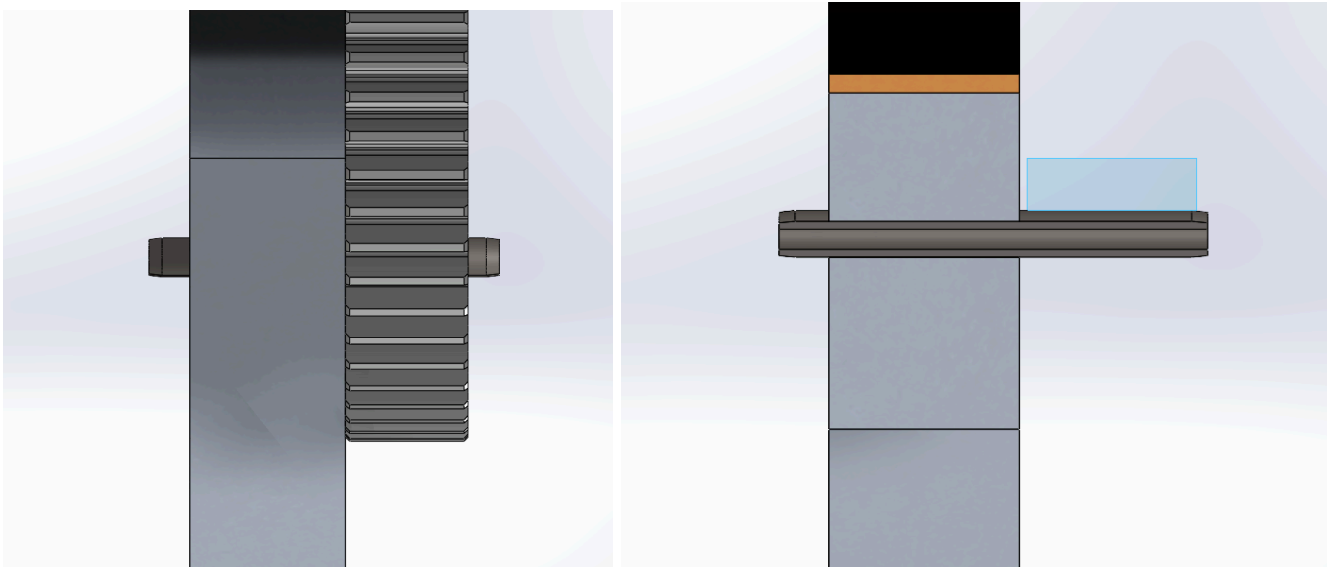


Figure 8.3 and Figure 8.4: Non-sectioned (8.3) and sectioned view (8.4) of spring pin torque joint.

As shown in the above figures, the spring pin is experiencing single shear with all of the shear stress being applied in one plane. Failure occurs when the shear stress experienced by the spring pin is larger than that of the shear yield strength with endurance being accounted for. The equation to find the spring pin shear stress is below.

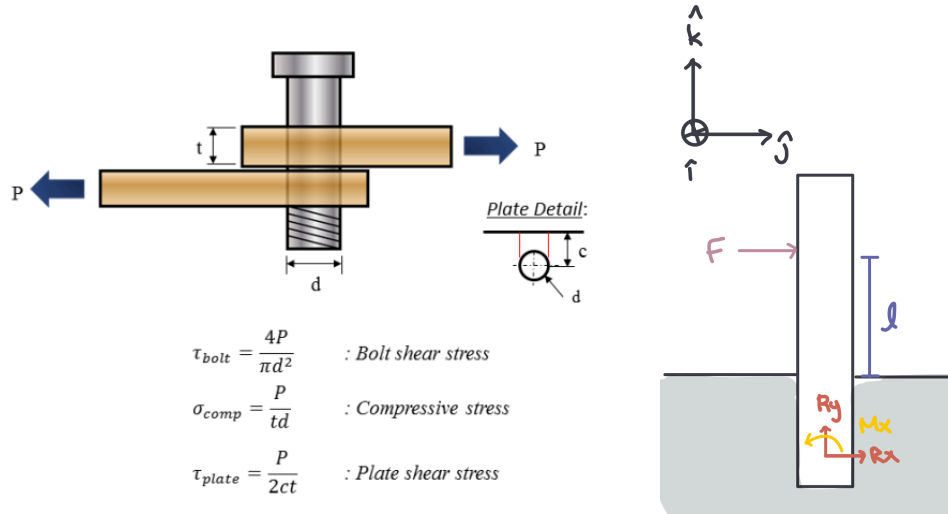


Figure 8.5 and Figure 8.6: Bolt shear stress equation for a single shear joint and FBD of spring pin. Gravity is negligible.

The bolt shear stress can be written as $\tau_{bolt} = \frac{F}{A} = \frac{F}{\pi r_o^2 - \pi r_i^2}$, where the outer radius of the spring pin r_o is 0.016 inches (0.4064 mm) and the inner radius is 0.01 inches (0.254 mm).

The magnitude of the force F is given by the equation $F = \frac{T_L}{r_{pin}}$ where T_L is the maximum torque exerted on the link and r_{pin} is the distance between the link's pivot point and the spring pin location. Again, T_L is given from our ADAMS plot in Figure 3.19 as 156 Nmm and r_{pin} is 0.4 inches (10.16 mm).

$$F = \frac{T_L}{r_{pin}} = \frac{156 \text{ Nmm}}{10.16 \text{ mm}} = 15.35433 \text{ N}$$

Revisiting the shear stress equation, we can plug in our values to determine the shear stress experienced by the spring pin.

$$\tau_{bolt} = \frac{F}{A} = \frac{F}{\pi r_o^2 - \pi r_i^2} = \frac{15.35433 \text{ N}}{\pi(0.4064)^2 - \pi(0.254)^2} = 48.56115 \text{ MPa}$$

The shear stress obtained can now be compared against the material yield strength. In order to get the approximate shear yield strength, we divide the tensile yield of 1095 steel by two.

$$\tau_Y \approx \frac{1}{2} \sigma_Y = (0.5)(570 \text{ MPa}) = 285 \text{ MPa}$$

However, this value does not take into account the multiple stress cycles of the spring pin. In order to ensure that the spring pin will withstand the multiple stress cycles, we find the endurance shear by dividing the shear yield strength by two.

$$\tau_{end} \approx \frac{1}{2}\tau_Y = 142.5 \text{ MPa}$$

Comparing the shear stress of the bolt τ_{bolt} and the yield shear stress (endurance accounted for) τ_{end} , we see that we are able to achieve a safety factor of $SF = \frac{142.5}{48.56115} = 2.9344$.

In addition, our linkage transmission includes a pinion (driving) and gear (driven). It is also necessary to evaluate the strength of the meshing teeth in order to avoid transmission failure.

The Lewis Bending Equation is below:

$$\sigma = \frac{K_v W_t}{FmY}$$

In calculating the stress experienced by the pinion:

The velocity factor is calculated by the equation $K_v = \frac{6.1+V}{6.1}$ assuming a cut/milled gear profile. The pitch-line velocity V is calculated by the equation $V = \omega_m r_{pinion}$. In the worst case scenario, we want to maximize the numerator in the Lewis Bending Equation and therefore evaluate the stress when the pinion is spinning the fastest. Since the linear velocities of the pinion and the gear are the same, we can rewrite the equation and solve, where the pitch radius of the gear is given by McMaster as 15 mm:

$$V = \omega_L r_{gear} = (2\pi \frac{rad}{s})(15 \text{ mm}) = \frac{30\pi}{1000} \text{ m/s}$$

The velocity factor can now be calculated as:

$$K_v = \frac{6.1+V}{6.1} = \frac{6.1+0.09424778}{6.1} = 1.01545$$

The tangential load experienced by the gear W_t can be found using the equation:

$$W_t = \frac{T_{L,max}}{r_{gear}} = \frac{156 \text{ Nmm}}{15} = 10.4 \text{ N}$$

The face width of the gear F is 3 mm as given by McMaster.

The module of the gear is $m = 0.5$ as given by McMaster

The Lewis Form Factor Y is determined by the number of teeth. For our pinion, 16 teeth corresponds to a Lewis Form Factor of $Y = 0.296$.

Plugging the appropriate values in, we see that our maximum gear tooth stress is:

$$\sigma_{pinion} = \frac{K_v W_t}{F m Y} = \frac{(1.01545)(10.4 \text{ N})}{(3 \text{ mm})(5 \text{ mm})(0.296)} = 2.3785 \text{ MPa}$$

The allowable stress can be calculated by taking the yield stress of the material (brass) and accounting for endurance.

$$\sigma_{end} = \frac{1}{2} \sigma_{yield} = (0.5)(34.5 \text{ MPa}) = 17.25 \text{ MPa}$$

From this value, we see that we can achieve a safety factor of $SF = \frac{17.25 \text{ MPa}}{2.3785 \text{ MPa}} = 7.25$ in regards to the pinion gear.

Repeating the same calculation for the driven gear, we see that the maximum gear tooth stress is:

$$\sigma = \frac{K_v W_t}{F m Y} = \frac{(1.01545)(10.4 \text{ N})}{(5 \text{ mm})(5 \text{ mm})(0.422)} = 1.00 \text{ MPa}$$

Once again, the allowable stress can be calculated by taking the yield stress of the material (brass) and accounting for endurance.

$$\sigma_{end} = \frac{1}{2} \sigma_{yield} = (0.5)(450 \text{ MPa}) = 225 \text{ MPa}$$

From this value, we see that we can achieve a safety factor of $SF = \frac{225 \text{ MPa}}{1 \text{ MPa}} = 225$ in regards to the linkage gear.

Section 9: Motor Controls:

Motor Controls: Motor control is necessary since there is a discrepancy between the encoder count that is written on the code and the actual encoder count. In order for the motor to move, the voltage provided to the motor should be able to overcome friction for itself. The first thing our team had to set up for the motor was the voltage, which will not move the motor to the encoder count otherwise. As the voltage increases, more power will be provided to the motor, but it will increase the speed. Furthermore, our team increased the voltage for better control, to make the linkage move faster to the targets to get a better score on the project. The difference between encoder counts is caused by the load mass and inertia provided by the linkages on the motor. It mostly results in overshooting or undershooting of the linkage's position, and it might oscillate as well. This makes the actual encoder count of the motor mismatching with the encoder count that is already set in the code. In order to remove oscillations, overshooting, and undershooting, setting up PID values of the motor is necessary (K_p , K_i , K_d values in the code in Appendix D).

Sensors: Obviously the encoder is a digital sensor that detects the position of the motor. This project uses several different sensors which are the limit switch and IR proximity sensors. The mechanisms that limit the linkage's position are the hardstops and the limit switch. Especially, limit switch measures proximity of the object by a small button that is pushed when the linkage is close enough, and unpushed when it is further away. The position of the linkage is set to 0 whenever the limit switch is pressed, so that the position of the motor starts from the same position to achieve the set encoder count relative to the initial position. Proximity sensors do not directly help control the motor's position, but it gives the encoder count that the motor needs to be moved. When the flags are up, the goal of the project is to move to the target position and shine the intensity sensor. Receiving how far the flag is up will give which target is on at the instant, so that the motor can move to the target.

Sensing Quality: The quality of encoder count affects the success rate of being able to target the intensity sensor. When the quality of the encoder count is low, then the measured encoder count when it is stationary, and the encoder count measured when the linkage is moving are different, which makes it extremely hard to find the perfect encoder count to target the correct position every time. The encoder count difference should be in the range of ± 5 for a good quality. Limit switch quality matters where the initial position starts. Since a small angle difference makes a huge difference where the flashlight targets, making sure that the limit switch is pressed at the same position every time is important. If the quality is bad, then the encoder count will vary every time since the initial positions are different. The proximity sensor quality is an important issue as well, since it decides which target is on and off. If the quality is bad, the linkage will move around and oscillate between the targets since it does not know precisely which target is on at that point.

PID: Most of the motor control implemented in our project relied on **Proportional-Integral-Derivative Control**. A PID controller constantly measures the error of our current and desired setpoints and attempts to correct it with the use K_p , K_d and K_i gains. Overshoot, rise time, settling time and steady-state errors are the main parameters that engineers use to measure the effects of the addition of K_p , K_d and K_i gains. P gain acts like a spring and increases the response speed for which a system will oscillate about the desired position. D gain adds damping to the response. It also provides great insight into future error trends. I gain eliminates steady state errors by integrating errors over time. Fundamentally, K_p corrects current error, K_i eliminates error by looking into historic cumulative error, and K_d predicts future error. Each gain has different effects on the system, hence, proper tuning and understanding of dynamic control systems is required to control a system to

reach a desired behavior. The following figures will show a PID Block Diagram and the effects of the PID in a control system.

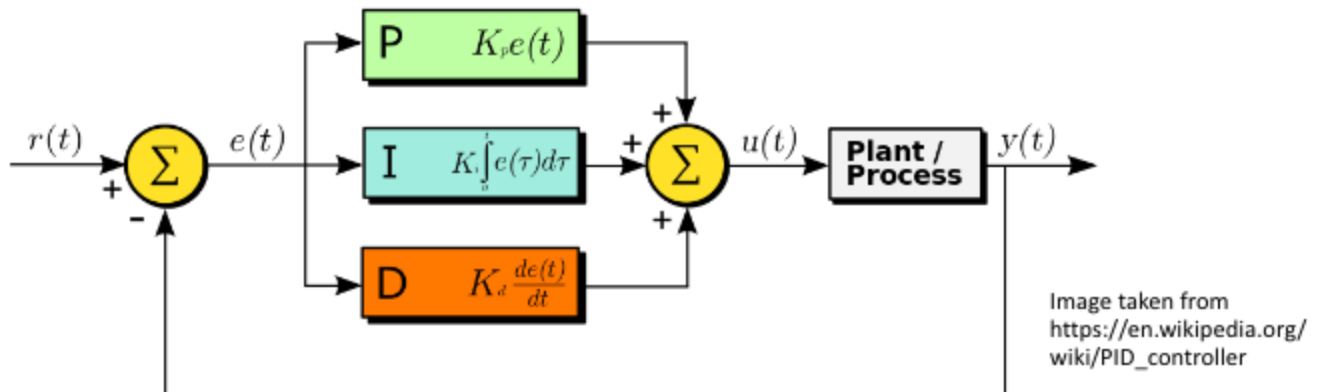


Figure 9.1: PID Block Diagram

Table 9.1: Effects of *increasing* each of the PID parameters.

Parameter	Rise Time	Overshoot	Settling Time	Steady-State Error
K_p	Decrease	Increase	Small Change	Decrease
K_i	Decrease	Increase	Increase	Eliminate
K_d	Minor Change	Decrease	Decrease	No effect in theory

PWM: Pulse Width Modulation represents a rectangular wave with a duty cycle. PWM is very useful to control the necessary amount of power or amplitude signal of a system. It is also a common way to perform Digital to Analog Conversion. DACs take discrete signals and convert them to continuous signals in time (not in value) with the use of an analog filter that smoothens the steps caused by quantization. We performed this controller mechanism when building the initial circuitry. The Circuit connects the Digital Controller (Arduino) and smoothens the digital signal into an analog signal through the PWM Amplifier (H-Bridge) and Motor (Low-Pass Filter). Unlike PID, PWM doesn't require tuning as the controller performs its functions when connecting the ARDUINO, H-Bridge, and the motor/encoder.

Proximity Sensors: These infrared proximity sensors are rigidly mounted to the playing to detect active targets. They work by shining a beam of infrared light from an LED, and measuring the intensity of the reflected light using a phototransistor. The sensors have a maximum range of about 30 cm (12 inches), hence why we need two to detect all five targets. Some limitations are that the sensors have to be cleaned for them to work properly. Moreover, proximity sensor tuning has to be made constantly as each time you assemble and remove the linkage from the playing field the sensors' readings change.

Section 10: Encoder Counts, Proximity Sensor Thresholds, and Controller Gains:

Encoder Position Tuning: During our initial tests, we determined the required encoder counts to hit each target by manually resetting the against the limit switch, and then manually adjusting the linkage's orientation so that the flashlight was centered on each target. This method was sufficient to determine under ideal conditions what the encoder count should be for our linkage to deactivate each target. While testing, however, we observed that this method of manually pointing our linkage at each target did not adequately account for factors such as play in our transmission or the inertia of the linkage as it turned towards each target. Our method of manually hitting the linkage against the limit switch likewise failed to account for the inertia of the linkage as it rotated during calibration, and even with a consistent hardstop position, this made it difficult to accurately determine the actual starting position of the linkage.

Rather than adjust the linkage by hand or perform calculations, then, our final method of tuning encoder counts was to simply let the linkage calibrate itself, then activate each target by hand and trial-and-error encoder counts until the linkage was accurate for each target. Given that our PID values were kept constant, this method proved to be far more consistent in getting our linkage on target. By using this method, we were able to adequately account for factors such as transmission play without the need of complex equations involving unknown variables. Transmission play, especially, would have been difficult to account for in calculations as this variable is difficult to quantify for our particular design. Overall, once our PID gains were determined, this single method provided a fast and easily verifiable approach to ensuring that our encoder positions were correct. The original values we determined manually, alongside the final values we determined through experimentation, are found below in table 10.1.

Proximity Sensor Tuning: The only approach we used for proximity sensor tuning was to raise each flag by hand, observe the readings for each proximity sensor in the serial monitor and identify the high and low boundaries for each sensor's reading to assign to the relevant variables. This procedure was the most convenient and straightforward as we could readily tune the proximity sensor readings for a particular playing field. Likewise, we made sure to increase the range of proximity sensor readings for each target to be as large as possible (without overlap between targets), to ensure that each target could be detected while accounting for fluctuations in proximity sensor readings. The final proximity sensor values used are found in table 10.1.

Controller Gains Tuning: Our process for tuning controller gains mostly involved gradually increasing K_p until the linkage was at a speed we were comfortable with, and then adjusting K_i and K_d to eliminate any oscillations and steady state error. First, to tune friction compensation voltage, we gradually increased the value in the tuning script until the linkage began moving. The tuning script was the simplest way to find this value as friction calculations would have required far too many unknowns and much more time. Once this was done, we ran our linkage with the base controller gains to observe its performance, before adjusting values ourselves until the linkage was at a speed we were satisfied with. One method we developed to ensure no oscillations was to record in slow motion our linkage as it traveled towards a target and check for any jolting. Overall, tuning the motor controller through this experimental process was more favorable than optimizing it through calculations. This method allowed us to visually inspect our linkage's performance and use our intuition to determine if its movement was reasonably optimized. Even if our controller values could have been further optimized for an even faster linkage, it could have presented risk to the mechanical integrity of our parts. As such, using our intuition while tuning the PID was the most optimal to mitigate risks of overtuning our motor and damaging our linkage.

Table 10.1: Summary of Variables in Arduino Code

Variable Name in Arduino Sketch File	Purpose of Variable	Device	Manually Determined Value	Value Used During Testing
TARGET_1_POSITION	Position for Target #1	Motor	157	314
TARGET_2_POSITION (WAIT_POSITION)	Position for Target #2	Motor	438	640
TARGET_3_POSITION	Position for Target #3	Motor	683	927
TARGET_4_POSITION	Position for Target #4	Motor	991	1216
TARGET_5_POSITION	Position for Target #5	Motor	1711	1940
K _p	Proportional Gain	Motor	N/A	0.09
K _i	Integral Gain	Motor	N/A	0.015
K _D	Derivative Gain	Motor	N/A	0.006
FRICTION_COMP_VOLTAGE	Friction Compensation Voltage	Motor	N/A	2 V
TARGET1_PROXIMITYSENSE1_MIN	Sensor #1 Threshold for Target #1	Proximity Sensor	N/A	430
TARGET2_PROXIMITYSENSE1_MIN	Sensor #1 Thresholds for Target #2		N/A	107
TARGET2_PROXIMITYSENSE1_MAX			N/A	173
TARGET3_PROXIMITYSENSE1_MIN	Sensor #1 Thresholds for Target #3		N/A	85
TARGET3_PROXIMITYSENSE1_MAX			N/A	106
TARGET3_PROXIMITYSENSE2_MIN	Sensor #2 Thresholds for Target #3		N/A	75
TARGET3_PROXIMITYSENSE2_MAX			N/A	90
TARGET4_PROXIMITYSENSE2_MIN	Sensor #2 Thresholds for Target #4		N/A	145
TARGET4_PROXIMITYSENSE2_MAX			N/A	160
TARGET5_PROXIMITYSENSE2_MIN	Sensor #2 Threshold for Target #5		N/A	365
TARGETNONE_PROXIMITYSENSE1_MAX	Sensor #1 Threshold for No Targets		N/A	84
TARGETNONE_PROXIMITYSENSE2_MAX	Sensor #2 Threshold for No Targets		N/A	80

Section 11: Final Testing Results/Discussion:

Testing Setup and Processes: Throughout our testing phase, we were able to gradually improve the performance of our linkage by identifying sources of inefficiency, iterating on linkage parameters, and ultimately finding a set of processes and parameters that maximized our score. Table 11.1 below illustrates the history of our linkage's performance throughout testing.

Table 11.1: Scores achieved throughout trial runs

Trial #	Score
1	3
2	21
3	23
4	26
5	28
6	27
7	28
8	30
9	27
10	30
11	29
12	27

As can be seen above, our biggest scoring improvements occurred in trials #2, #4, #7 and #8. Table 11.2 below provides a deeper explanation of what parameters we manipulated in each trial to achieve that milestone, as well as the effects those changes had on our linkage.

Table 11.2: Top 4 largest scoring improvements

Trial #	Score	Improvement on Previous Milestone	Parameters Changed	Effects on Performance
2	21	+18	Switched out flashlight battery for one with more charge	Solely by switching out our flashlight battery, we were able to increase our score by 18 points. This test occurred right after our initial trial with 3 points and made us realize the necessity of testing with a fully bright flashlight – for all future trials we made sure to check the brightness on our flashlight.
4	26	+3	Tuned PID controller for higher P gain and friction compensation voltage	We were able to increase the speed of our linkage by manipulating PID values. Specifically, we experimented with a higher P gain and tuned the rest of the PID to avoid overshoot. This increase in speed significantly improved the number of targets our linkage could drop across an entire test
7	28	+2	Wrote additional code to eliminate redundant movements observed while testing	While testing, we noticed that the provided code did not direct our linkage to follow the most efficient path. When targets 1 and 5 were both up, for example, our linkage would first move to target 1, then move to hit target 5, and then back to target 1. This was a result of the order in which targets were processed in the code, and we eliminated this by adding additional instructions for which targets should be prioritized when multiple targets are up. As a result, our linkage wasted less time and could hit more targets in the timeframe.
8	30	+2	Mechanical tuning of joints, tuning encoder values and continued testing with previously established procedure	Our final improvement came from fine checking our mechanical components. This included tightening shoulder screws and set screws to reduce play, as well as fine tuning our encoder values so that the linkage could more accurately shine on each target. Overall though, continuous testing of our linkage with the procedures set above was what brought us to our final score of 30.

Additional Testing Procedures: Beyond the four main improvements described in table 11.2, we also gradually improved the performance of our linkage through various miscellaneous ways. Our wait target, for example, was initially set at target 3, but we discovered throughout testing that we can reduce unnecessary movement in our linkage by instead setting it to target 2. This saved us some time with our linkage's movement. Other improvements mainly involved us being consistent with setting up the linkage before each trial. By the end of our testing, we had established a procedure of tightening shoulder screws, set screws, ensuring that encoder counts were accurate for each target and checking the flashlight's brightness before each trial. This, overall, made our tests less dependent on luck and with enough trials we were able to achieve our final score of 30.

Discrepancies between Results and Expectations: Overall, we were satisfied with our final score and the procedure we created to achieve it. Some of the issues we encountered were expected, such as needing to fine tune the PID, and several improvements came from simply following a rigorous procedure to ensure the reliability of the linkage. Our most unexpected issue encountered was the inefficiency in the code, which prevented our linkage from following the quickest path to deactivate targets. This required innovation on our part to create additional code so that our linkage would not waste time deactivating multiple targets.

Section 12: Design Critique & Evaluation:

1. What worked well? What didn't work well? Most importantly, explain why.

When first building our linkage, we were lucky to not have many mechanical issues with our joints or transmission as a result of having well-designed CAD. The only mechanical issues we faced included our flashlight being slightly off target in the vertical direction, and our transmission having a small amount of play. While we tried accounting for these issues in the design, such as by making the motor position slightly adjustable, they still arose mainly because of unforeseeable imperfections in both our manufactured parts and off-the-shelf parts. These issues, however, were easily improvised around by designing spacers and accounting for transmission play during tuning. Beyond this, we faced some difficulty when initially running our performance test, and this was because we had not been familiar with the provided Arduino scripts yet. After working with the Arduino for a couple sessions, however, we were able to create a process to properly tune our linkage and start iterating on performance tests.

2. What was your scoring strategy? Did it work as planned? If not, what would you improve for better results?

Our strategy focused mainly on how we set up our linkage for each test, by eliminating as many sources of lost points as possible. Our procedure to constantly replace our flashlight battery every few tests once it got dim, for example, ensured us that light intensity would not be a source of lost points as we iterated on other parameters. Beyond this, we innovated several solutions to make our linkage have optimal movement when hitting targets. This included setting our target wait position to target 2, introducing modifications to the script so that our linkage would prioritize targets in a more efficient way, and tuning encoder counts experimentally to ensure that our flashlight was on target while testing. The fact that our linkage was designed to go around the blockade to hit target 4 from the other side helped in our speed as well. Overall, our strategy worked well in that it was a solid process that could be iterated on and have its parameters gradually refined. In the end, we were able to settle down on optimal parameters that maximized our linkage's score.

3. How well did your design perform relative to your models? Explain what you think the difference was.

Our design performed slightly worse than our models. From our ADAMs model, we made a mental note of the travel time between the two farthest times and observed it to be faster than what was possible for our linkage. However, this is to be expected as we encountered friction, computational and processing time delays, inertia in our system. Additionally, in our ADAMs model, we could reliably take the most optimal path between two targets (acceleration for half and then prompt deceleration for the second half). However, as we had encountered some variability in our system, we instead needed to rely on PID control to move our linkage. As we approached the incoming target, we needed to dampen our system according to the value, derivative of, and integral of the error. Thus, our observation of our linkage moving slower than in our models is justified from both variability and from our need to use PID to overcome it.

4. How did friction influence your built device?

The objective of reducing friction ended up introducing other unforeseen problems with our linkage that we had not anticipated when creating CAD. One of these was the vertical play experienced in our linkage once it was assembled. To reduce friction as much as possible, we tried to loosen our shoulder screws to an extent where it did not apply pressure to our joint bearings, but this ended up introducing vertical play in our joints, causing our flashlight to droop slightly. This issue was resolved, however, by creating 3D-printed spacers for our linkage, so that our flashlight could be on target while also experiencing as little friction as possible. Overall, the friction in our linkage was manageable and easily accounted for with the tuning script. In the end, other factors such as target accuracy played a larger role in our performance compared to joint friction.

5. How could the control algorithm (Arduino code) be improved to make better use of the available equipment? For example, could the position be more precisely controlled? Could you make use of a feed-forward signal? How?

Using the same proximity sensor input, we can adjust the software to improve the performance of our linkage. The given software prioritized close targets when choosing the next target to go to. While good on paper, the time taken to travel between linkages is approximately the same as the time it takes for the proximity sensor value to settle. Meaning if the targets are raising/lowering, the delay before the proximity sensor reads these changes is approximately the same as our linkage travel time to anywhere on the playing field. Hence, we observed a common phenomenon where our linkage would oscillate between two targets, effectively doubling the travel time. For example, say targets 1, 2, and 5 are currently up and our linkage was at target 1. Our linkage would then observe target 5 being upwards while target 1 was lowering. As soon as target 2 was detected, our linkage would then travel back to target 2 in an attempt to locate the closest target. However, if we adjust our control to be the farthest away target rather than closest, we can significantly reduce our linkage travel time. Our current control algorithm could have benefited from a buffer when reading in our sensor data. The proximity sensors, despite being properly tuned, would sometimes experience noise. Thus, an idea to help combat this would be to store a vector of past proximity values and require the entire vector to match to proceed with any different command. We observed our linkage sometimes moving to “random” targets, hence implementing this buffer would further improve our performance. We could implement a more precise control algorithm to better leverage knowing where each target is in respect to each other. If we can better model our linkage and DC motor, we can better predict and overcome some of the variable friction and acceleration that occurs while traveling between linkages. Our current approach has us hand-tuning the PID values but with some more mathematical modeling, perhaps we can make a more informed decision on balancing rise time and settling time. This knowledge would then be implemented as a feed-forward signal and would be our input to the system.

6. Would there be a better way to use the sensors that were provided, or to use different sensors, to accomplish the same objectives?

The current orientation of sensors limits us to only seeing the two outer targets at the same time. This limits our linkage’s ability to most optimally travel between targets due to the uncertainty of hidden targets. If we had more position sensors (maybe one per target) that would individually detect each target, we could greatly improve our performance results. In addition, another idea would be to utilize a color sensor to detect when our linkage reached the target. Often our linkage would experience variability and sometimes miss the targets by a few degrees. A color sensor would help offset this error if we implemented it into our code.

7. What other lessons or unique observations did you make about your device and the process that you follow to develop it? Having completed it, would you do anything different – what and why?

One observation we made about our linkage was the tendency of our shoulder bolt on the linkage gear to loosen itself as it moved through the target sequence. A looser gear would cause our flashlight to dip vertically. Whenever we noticed our light being slightly below each target, we compensated this by barely tightening the gear to improve targeting but not so much as to greatly increase the friction of the linkage. Another observation we noticed was our linkage's tendency to skip teeth on occasion. Tightening the set screw that connected the motor shaft to the motor gear reduced this tendency. Having completed it, we would adjust our design to help be more robust against these factors. We could support our linkage both from below and above to help prevent the vertical play and we could utilize a spring pin instead of a set screw to prevent skipping of teeth.

8. On a scale of 1-5, how would you rate the safety of your mechanism? Were the provided sensors adequate or would additional sensors and guards be necessary?

We give our mechanism a 5/5 for safety, because we personally never experienced problems with our linkage for the several hours we have spent testing it. Some factors influencing this are that despite our PID being optimally tuned, our linkage still does not travel so fast such that it could seriously hurt anyone. Because our linkage did not require a large range of motion to hit all targets, this also reduced the safety risks of testing, as our linkage only travels within a reduced range of motion. This also results in a lower max acceleration and max speed, given the smaller range of motion. The provided sensors and script were enough to avoid any accidents, as the most important factor was to remember not to touch the playing field while the linkage was running.

9. Would you recommend that we give your device to someone else right now to use and operate? Why or why not?

We would not. Our linkage requires extensive tuning to get the controls just right. Everytime we began operating our device, we needed to spend time fine tuning the proximity values to account for differences in lighting and playing fields. This process has taken us significant practice to get just right. Additionally, we needed to fine tune the encoder counts for each target after we assembled our linkage on a new playing field to account for any slight deviances in assembly. Additionally, we ran into slight mechanical problems that required fine tuning as well. For example, on occasion we needed to retighten a set screw and our gear to the base plate. Overall, our linkage, while functional for our own use in testing, will prove to be rather difficult to give to someone else without significant explanation.

10. What other parts and materials would have been useful to have in your design, if you had been given them?

A useful part would be additional spacers. Something we didn't account for was the vertical play in our linkage. Thus, having readily made spacers to help compensate for this vertical error would've been nice. We did create some 3D printed ones, but it is difficult to get the height aligned with the target. Another useful part would have been more large allen keys designed for the larger shoulder screws. While some were in the mechatronics lab, there were frequent times where all of them disappeared. It made testing extremely difficult as we needed to precisely tighten our baseplate to the playing field and hand tightening the shoulder bolts proved to cause additional vibrations and made tuning our encoder counts difficult. A torque wrench could also be useful as it'll help to remove the variability in assembling our linkage on the playing field.

11. Do you think the current point distribution is fair? How could it be modified for future semesters?

The point distribution is pretty fair. It equally rewards the theoretical (weight and transmission angle deviation) and actual (game score) performance. Hence, if teams have a good design but struggle with the control, they are still rewarded for their initial efforts. In addition, if teams have a poor design but are smart with their control implementation, then they can also be rewarded. Overall, the point distribution is very fair. If it were to be modified for future semesters, one idea for a change would be to adjust the relative impact of the transmission angle deviation and weight to be equal. A lot of teams neglected transmission angle deviation completely in favor of focusing on reducing weight just because of the relative category weight of both. Making them equal could lead to a greater deviance in linkages as teams are more incentivized to analyze this tradeoff rather than being led down the reducing weight path.

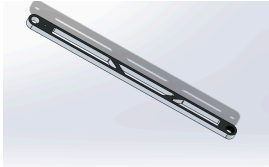
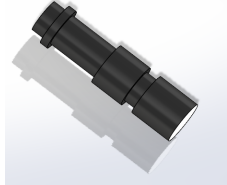
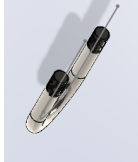
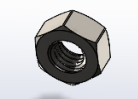
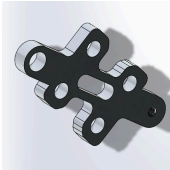
12. What would you do for a project next year in ME350?

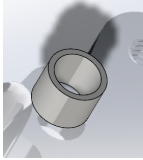
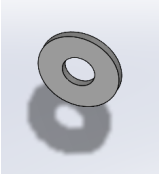
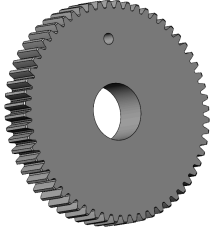
The previous project of the ball sorter seemed really interesting. The light sensors were difficult to use and varied a lot between playing fields. In addition, there was too much variability between flashlights, battery, and proximity sensor values. The ball sorter would be more complex due to the gravitational force and the range of motion acting on the same 2D plane. However, it would likely produce more consistent results once the PID is fine tuned. Hence, while being a bit more complex, we feel like the previous ball sorter project would be overall less frustrating to work with than our current project.

Appendix A: Assembly Manual

Linkage Assembly

Linkage Part List:

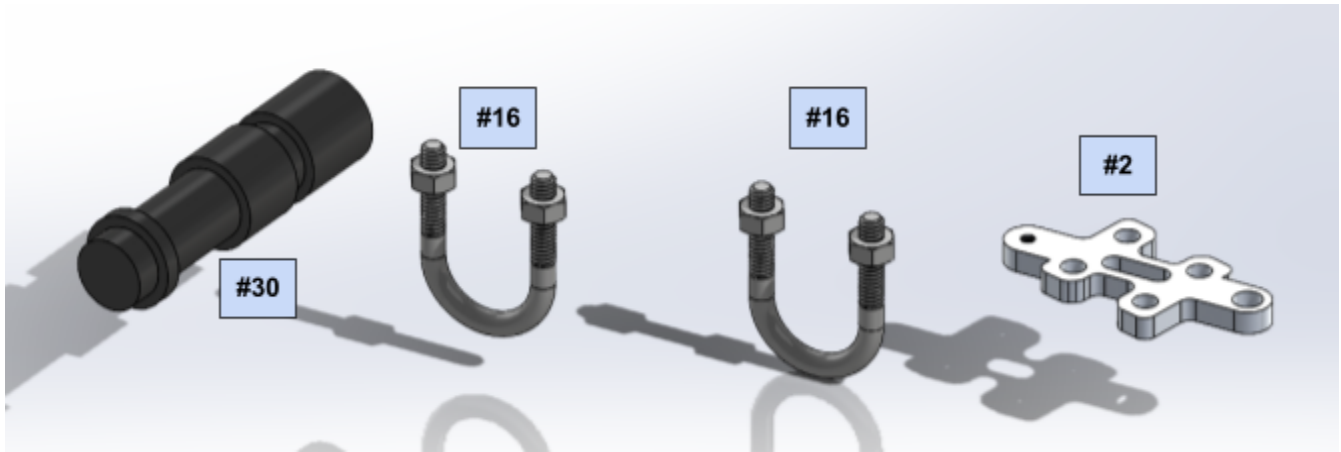
ID	Name	Quantity	Picture
#3	Follower Link	1	
#1	Input Link	1	
#30	Flashlight	1	
#16	U-bolt Screw	2	
#22	U-bolt Nut	4	
#2	Coupler	1	
#18	Shoulder screw ($\frac{1}{4}$ " Diameter x $\frac{1}{2}$ " Shoulder)	4	

#28	Washers ($\frac{3}{8}$ ")	4	
#26	Bushing	4	
#27	Bearing Washer	4	
#25	Thrust Bronze Bearing	4	
#14	Spring Pin	1	
#13	Link Gear (60 teeth)	1	

Step 1:

Materials: Flashlight, 2x U-bolt screws, coupler

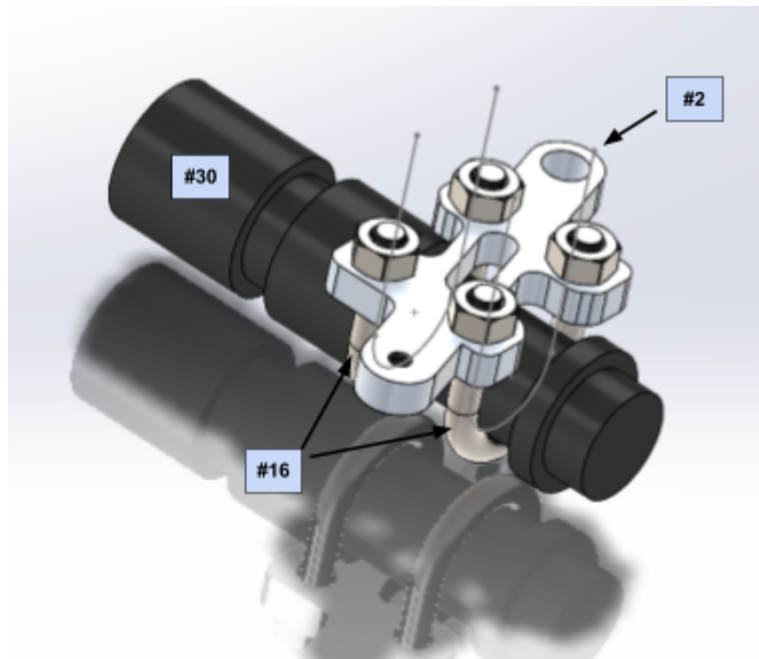
Description: Remove nuts (4x) from both U-bolt screws



Step 2:

Tools: Allen Key

Description: Secure both U-bolt screws onto the flashlight with one hand. Insert coupler on top of both U-bolt screws. Tighten nuts until secured.

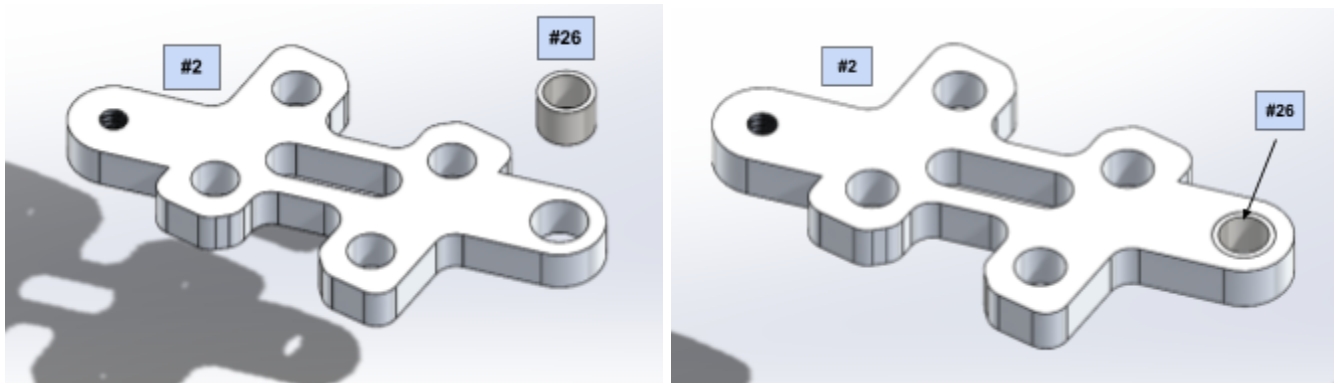


Note: For ease of visualization, flashlight, U-bolt screws and nuts will be omitted in pictures but presumed to be there.

Step 3:

Materials: Coupler and Bushing

Description: Press fit bushing into the largest hole until aligned with both faces of the coupler.



Step 4:

Materials: Shoulder screw ($\frac{1}{4}$ " Diameter x $\frac{1}{2}$ " Shoulder), Washers ($\frac{3}{8}$ "), 2 x Oil-Embedded Thrust Bronze Bearing, Needle Thrust Bearing:

Order from top to bottom:

Shoulder screw (#8)

Washer (#26)

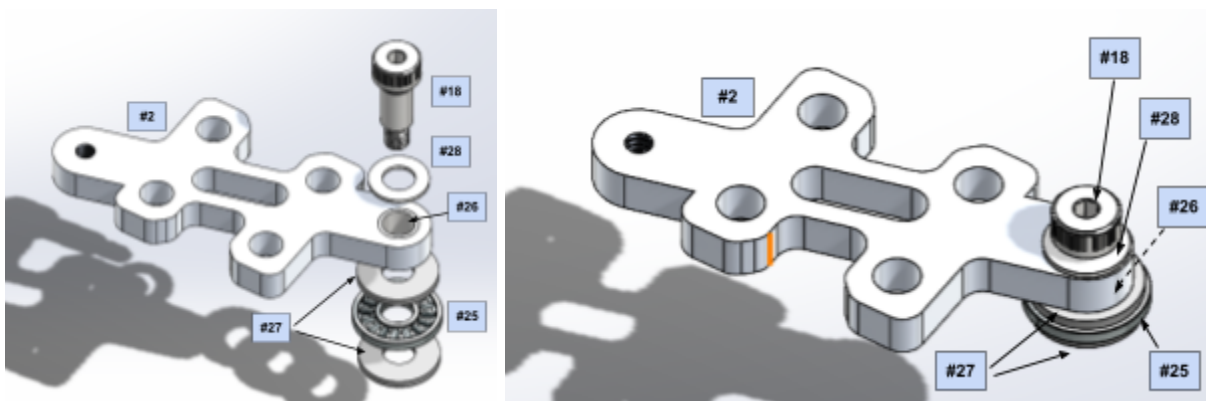
Coupler w/ Bushing (#26)

Bearing Washer (#27)

Needle Thrust Bearing (#25)

Bearing Washer (#27)

- Move washer on top of coupler
- Insert shoulder screw into both washer and coupler
- Insert remaining components in seen order

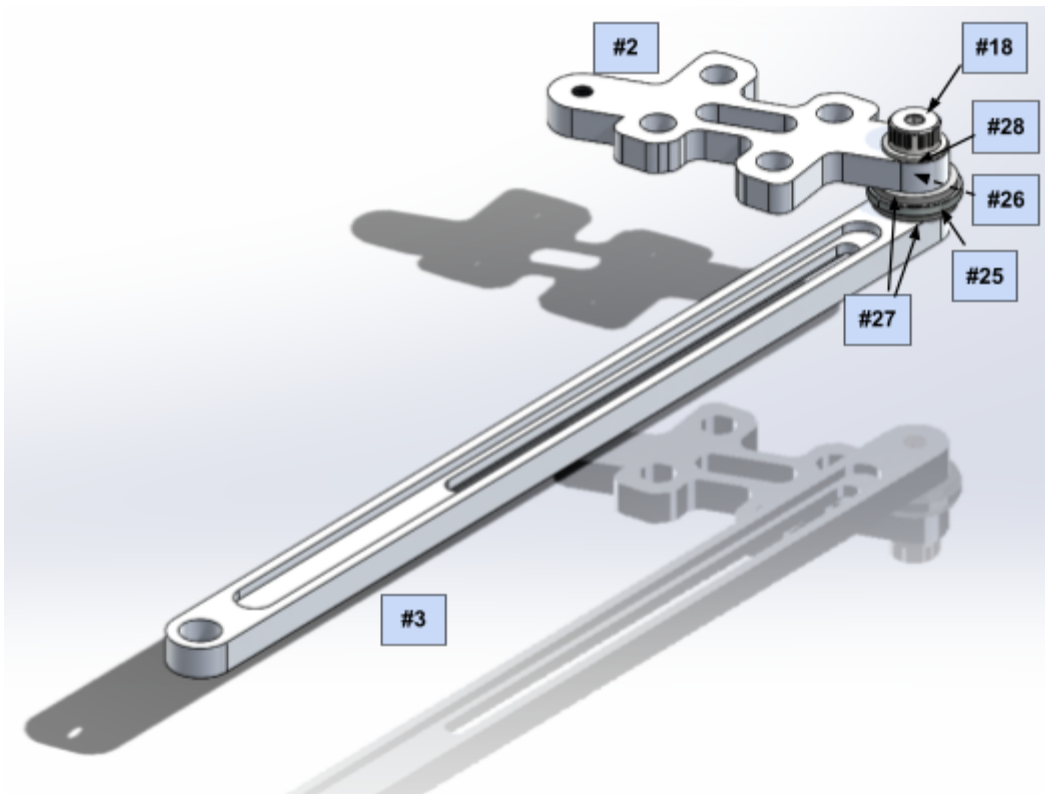


Step 5:

Materials: Boss linkage

Tools: Allen key

Description: Screw in joint configuration into boss (slotted linkage):



Step 6:

Materials: Shoulder screw ($\frac{1}{4}$ " Diameter x $\frac{1}{2}$ " Shoulder), Washers ($\frac{3}{8}$ "), 2 x Oil-Embedded Thrust Bronze Bearing, Needle Thrust Bearing:

- a. Install bearing as in step 3
- b. Flip entire part upside down
- c. Install joint same procedure as in step 4

Bottom to top:

Shoulder screw (#18)

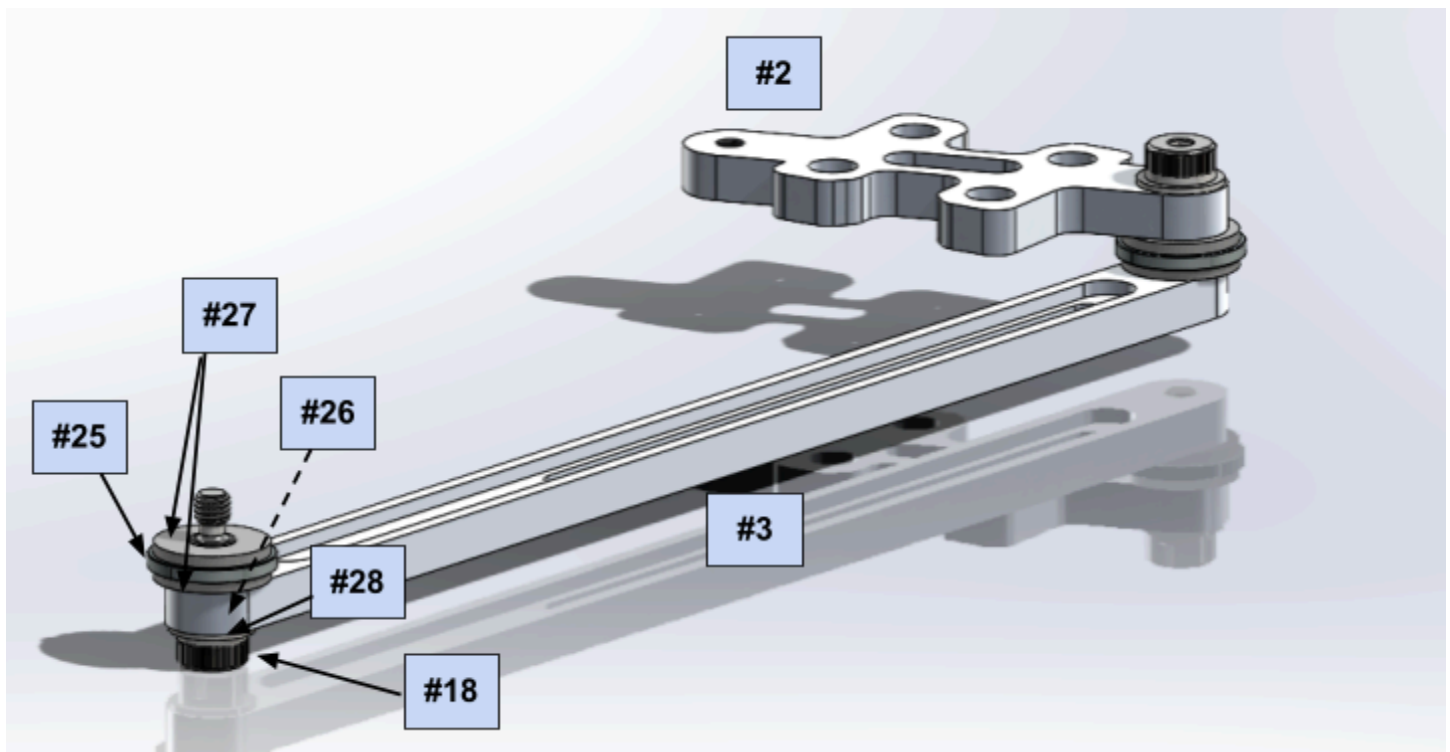
Washer (#28)

Boss w/ Bushing (#3, #26)

Bearing Washer (#27)

Needle Thrust Bearing (#25)

Bearing Washer (#27)



Step 7:

Materials: Trapezoid Linkage, Shoulder screw ($\frac{1}{4}$ " Diameter x $\frac{1}{2}$ " Shoulder), Washers ($\frac{3}{8}$ "), 2 x Oil-Embedded Thrust Bronze Bearing, Needle Thrust Bearing:

Description: Install joint on side without additional small spring pin hole.

- Install bearing as in step 3
- Flip entire part upside down
- Install joint same procedure as in step 4

Top to bottom:

Shoulder screw (#18)

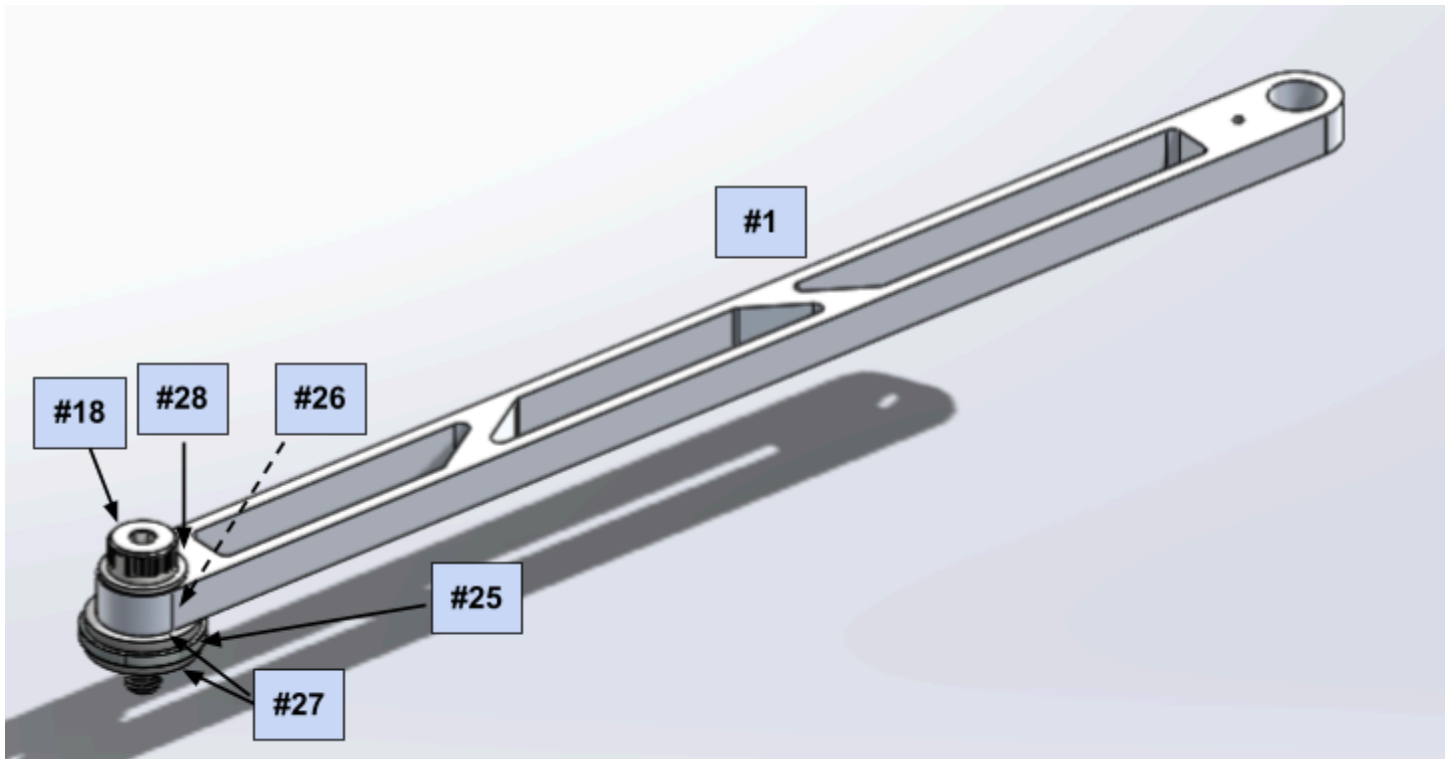
Washer (#28)

Input Link w/ Bushing (#1, #26)

Bearing Washer (#27)

Needle Thrust Bearing (#25)

Bearing Washer (#27)



Step 8:

Materials: Shoulder screw ($\frac{1}{4}$ " Diameter x $\frac{1}{2}$ " Shoulder), Washers ($\frac{3}{8}$ "), 2 x Oil-Embedded Thrust Bronze Bearing, Needle Thrust Bearing, Link Gear (30 teeth)

Description: Install joint on side with additional spring pin hole

- a. Install bearing as in step 3
- b. Flip entire part upside down
- c. Install joint similar procedure as in step 4, with extra link gear

Top to bottom:

Shoulder screw (#18)

Washer (#28)

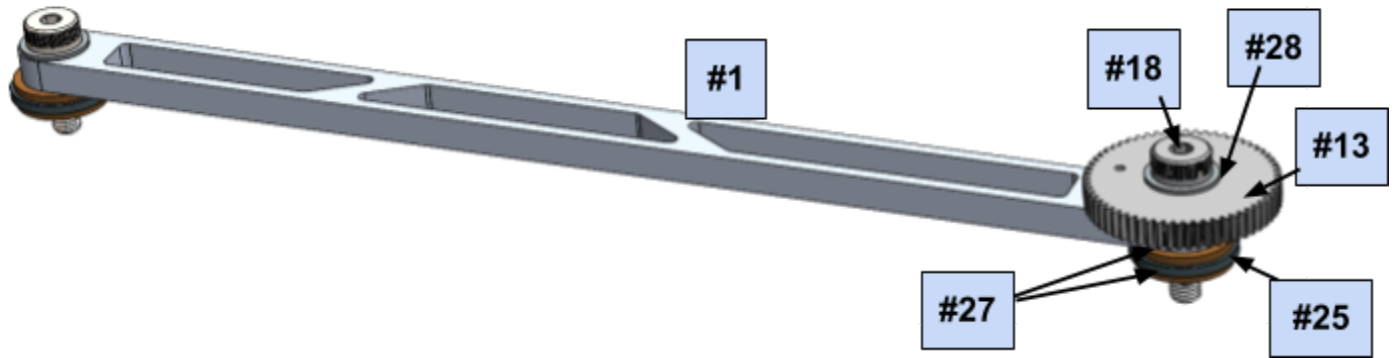
Link Gear (30 teeth) (#13)

Input Link (#1)

Bearing Washer (#27)

Needle Thrust Bearing (#25)

Bearing Washer (#27)

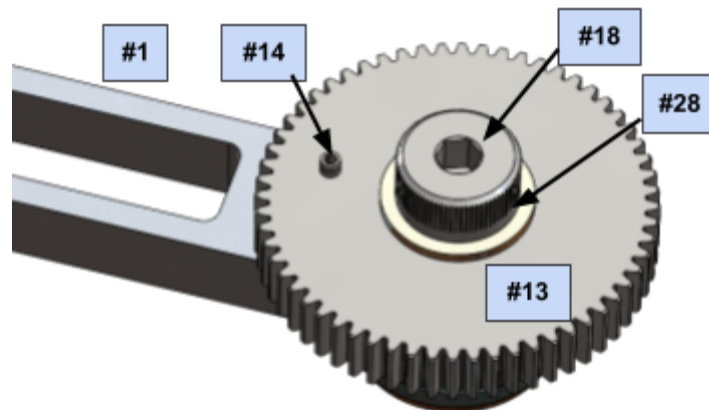
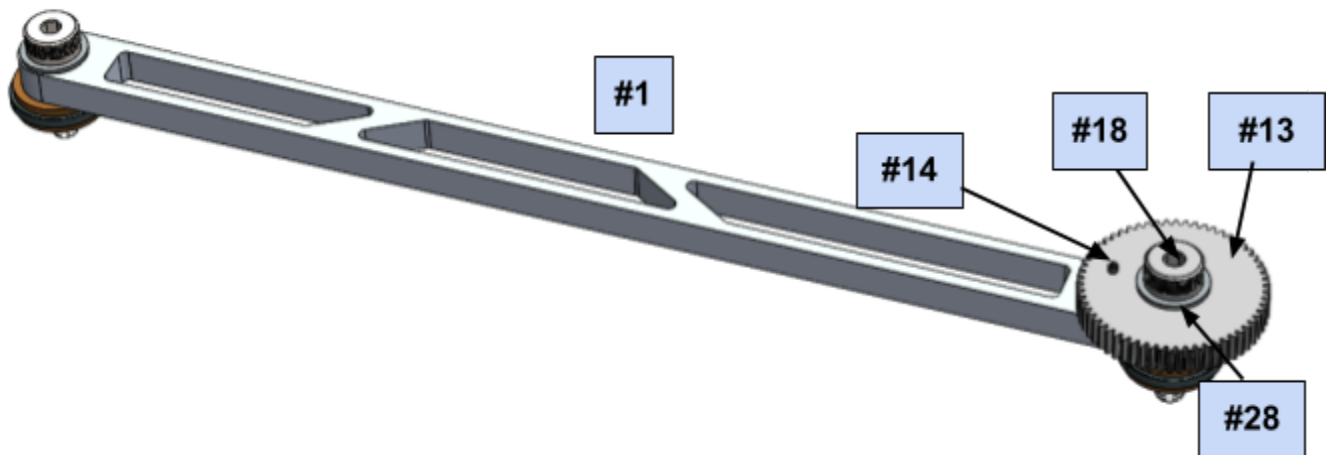


Step 9:

Materials: Spring pin

Tools: Hammer

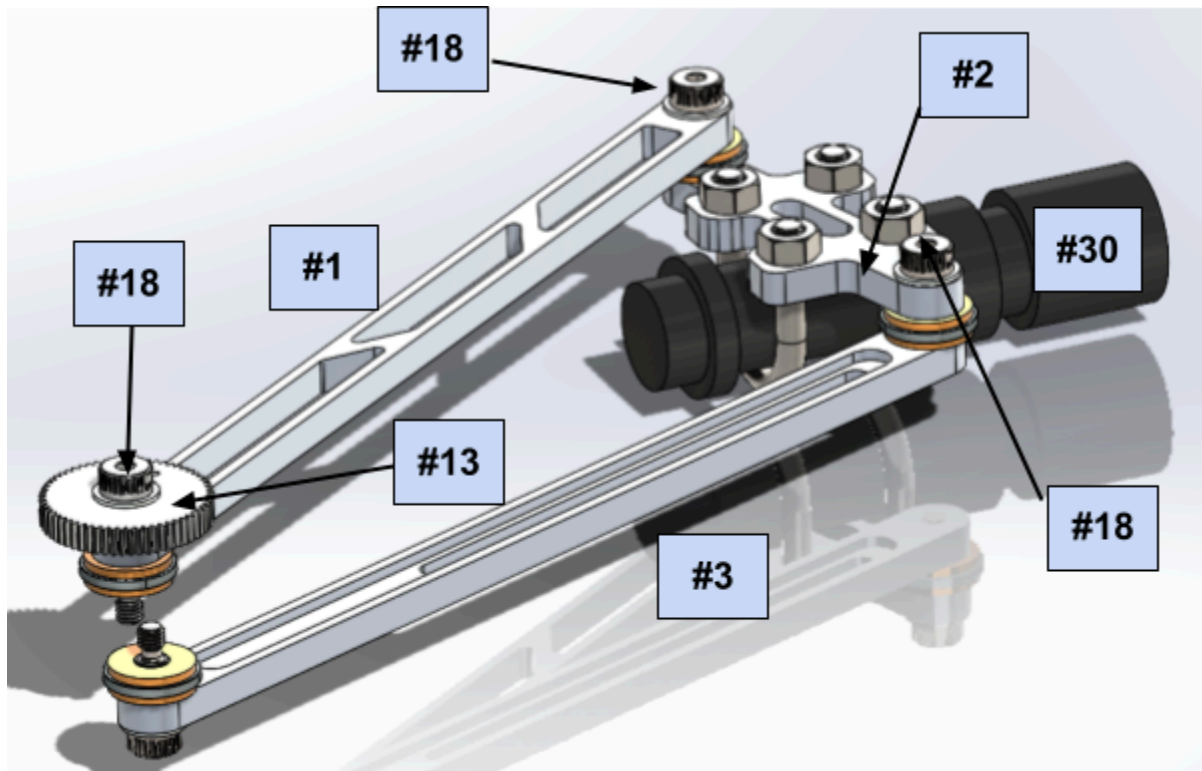
Description: Align smaller gear holes and smaller trapezoid to be concentric. Insert spring pin using a hammer



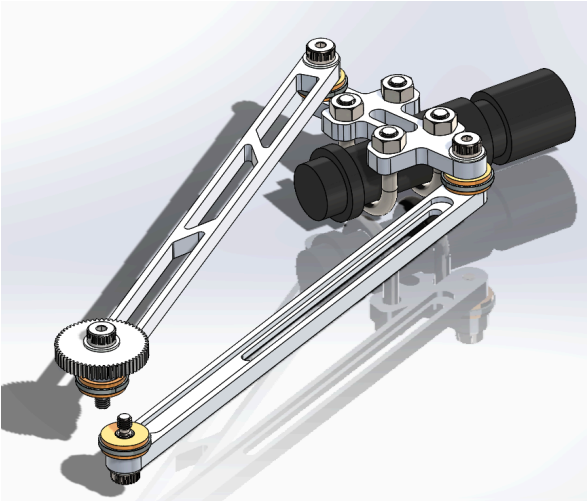
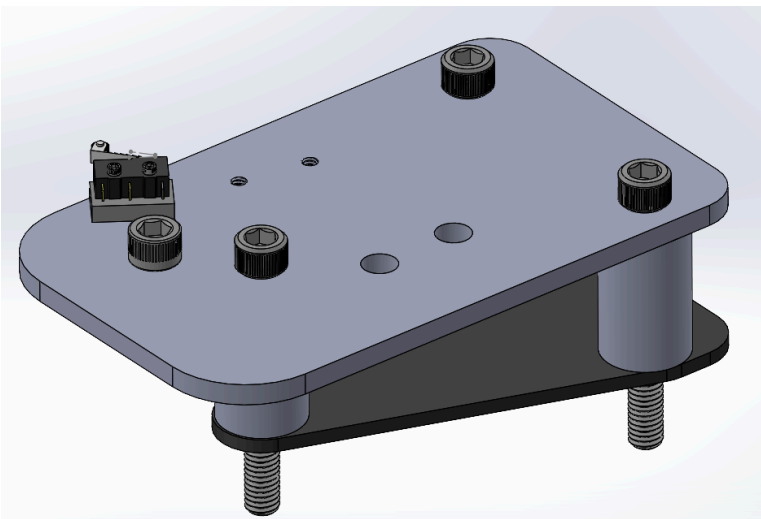
Step 10:

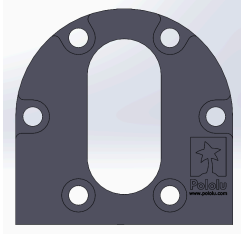
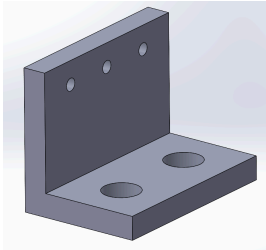
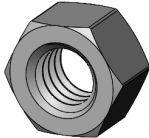
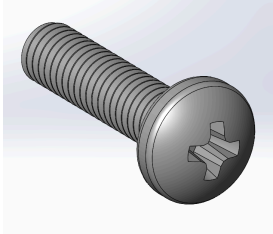
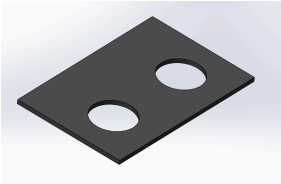
Tools: Allen key

Description: Screw in trapezoid assembly onto the coupler until secured.



Transmission AssemblyTransmission & Motor Assembly Part List:

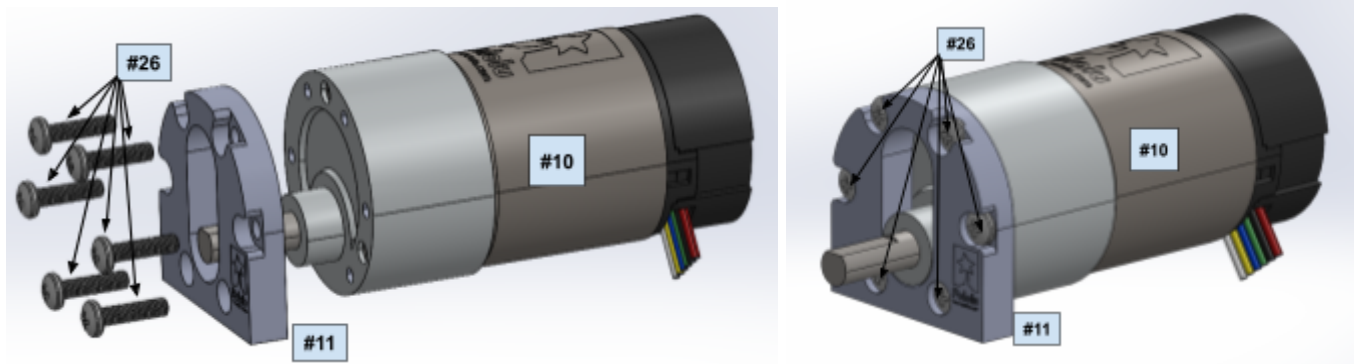
ID	Name	Quantity	Picture
A	Assembled Linkage	1	
B	Assembled Baseplate	1	
#10	Pololu Motor	1	

#11	Pololu Motor Mount	1	
#8	Designed Motor Mount	1	
#19	3/8"-16 Thread, 7/8" Long Socket Head Screw	2	
#23	3/8"-16 Nut	2	
#24	M3 Screw	3	
#12, 15	16 Tooth Pinion Gear + Set Screw	1	
#32	Motor Mount Spacer	1	

Step 1:

Materials: Pololu Motor, Pololu Motor Mount, 6x M3 Screws

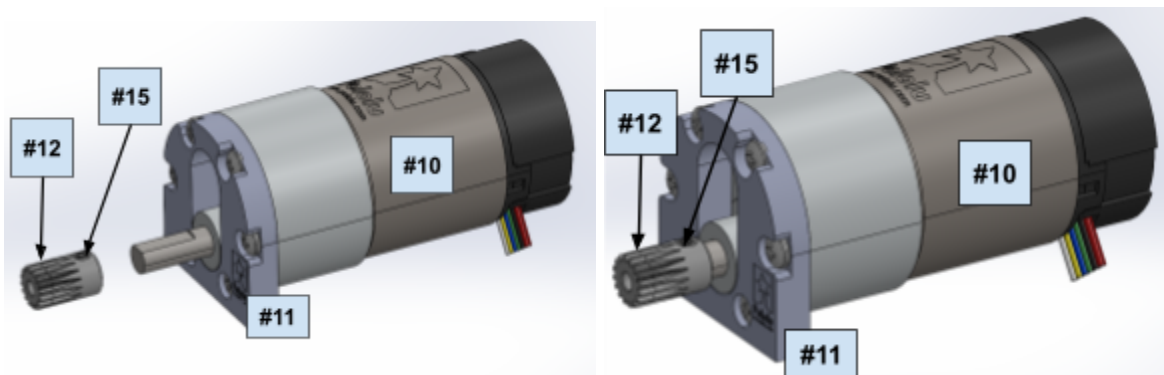
Description: Attach motor to provided bracket mount



Step 2:

Materials: Step 1 Assembly, 16 Tooth Pinion Gear & Set Screw

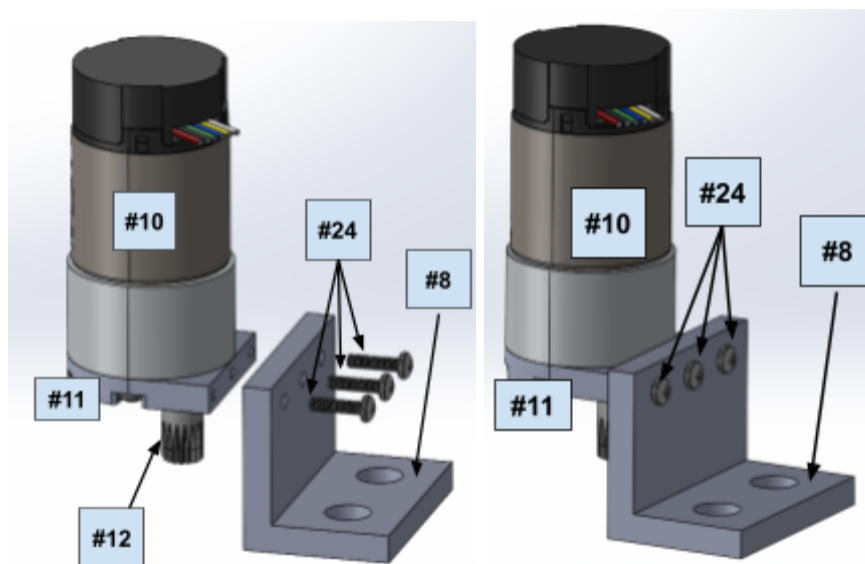
Description: Slide the pinion gear onto the motor shaft and tighten the set screw



Step 3:

Materials: Step 2 Assembly, Designed Motor Mount, M3 Screws

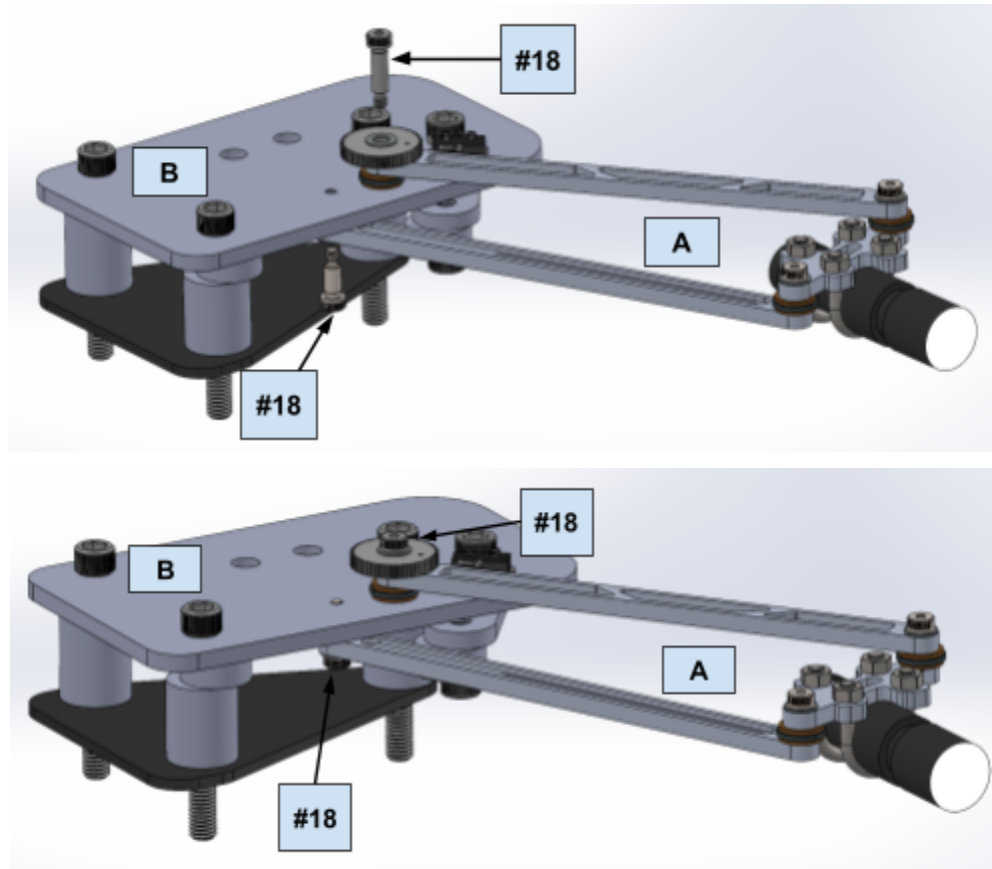
Description: Align the motor bracket with the motor mount and fasten together with screws



Step 4:

Materials: Assembled Linkage, Assembled Baseplate

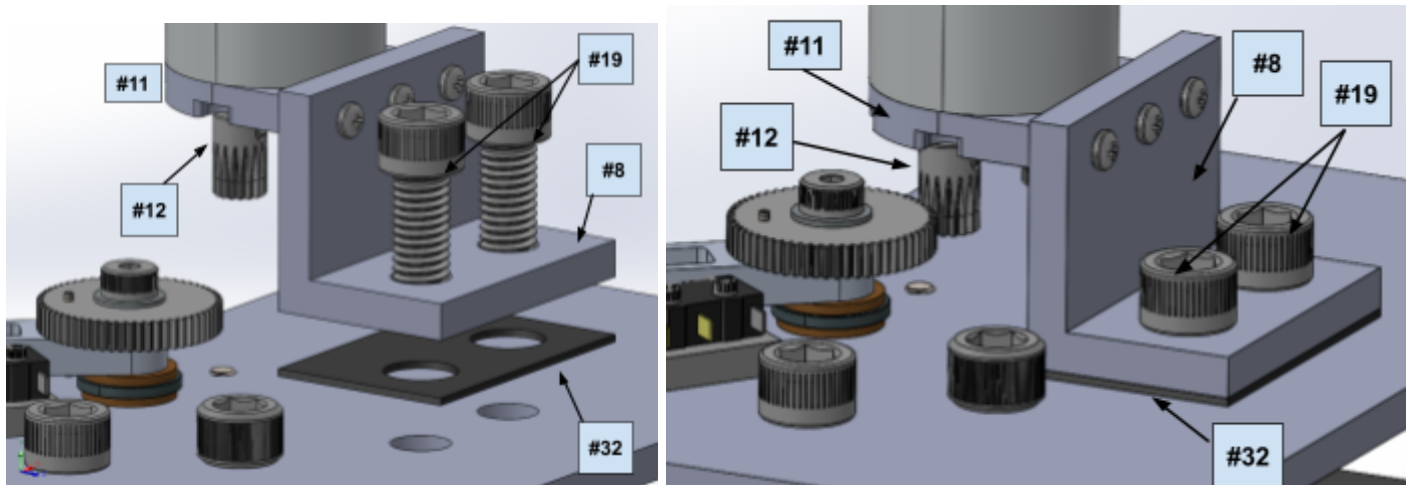
Description: Fasten the shoulder screws on the ground joints of the linkage into the assembled baseplate



Step 5:

Materials: Assembled Baseplate & Linkage, Step 3 Assembly, Motor Mount Spacer, 2x Socket Head Screws

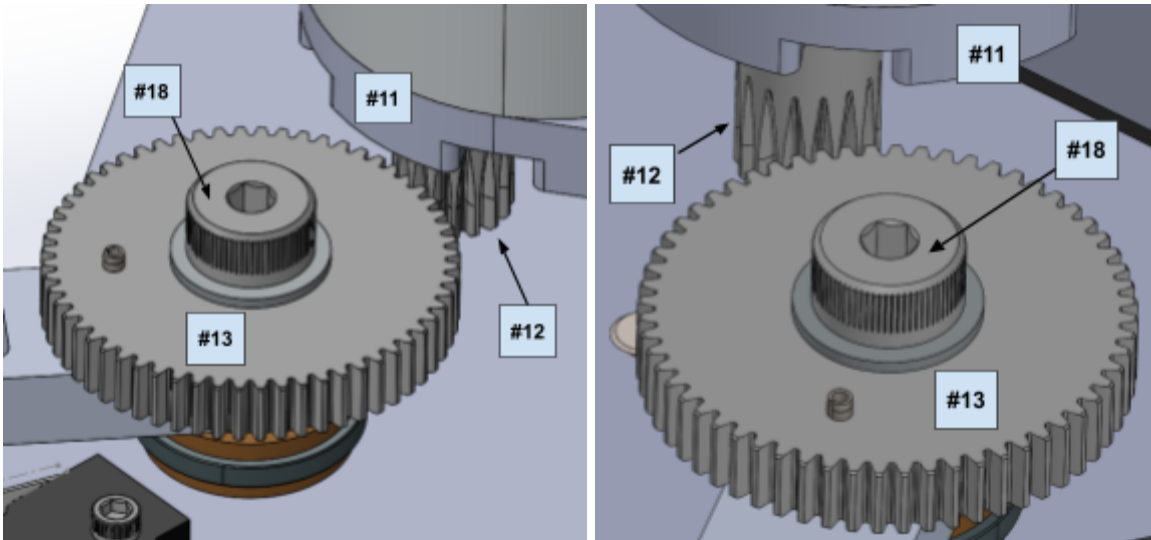
Description: Place the motor mount spacer underneath the current motor assembly and set it onto the baseplate



Step 6:

Materials: Assembled Baseplate & Linkage, Step 3 Assembly, Motor Mount Spacer, 2x Socket Head Screws

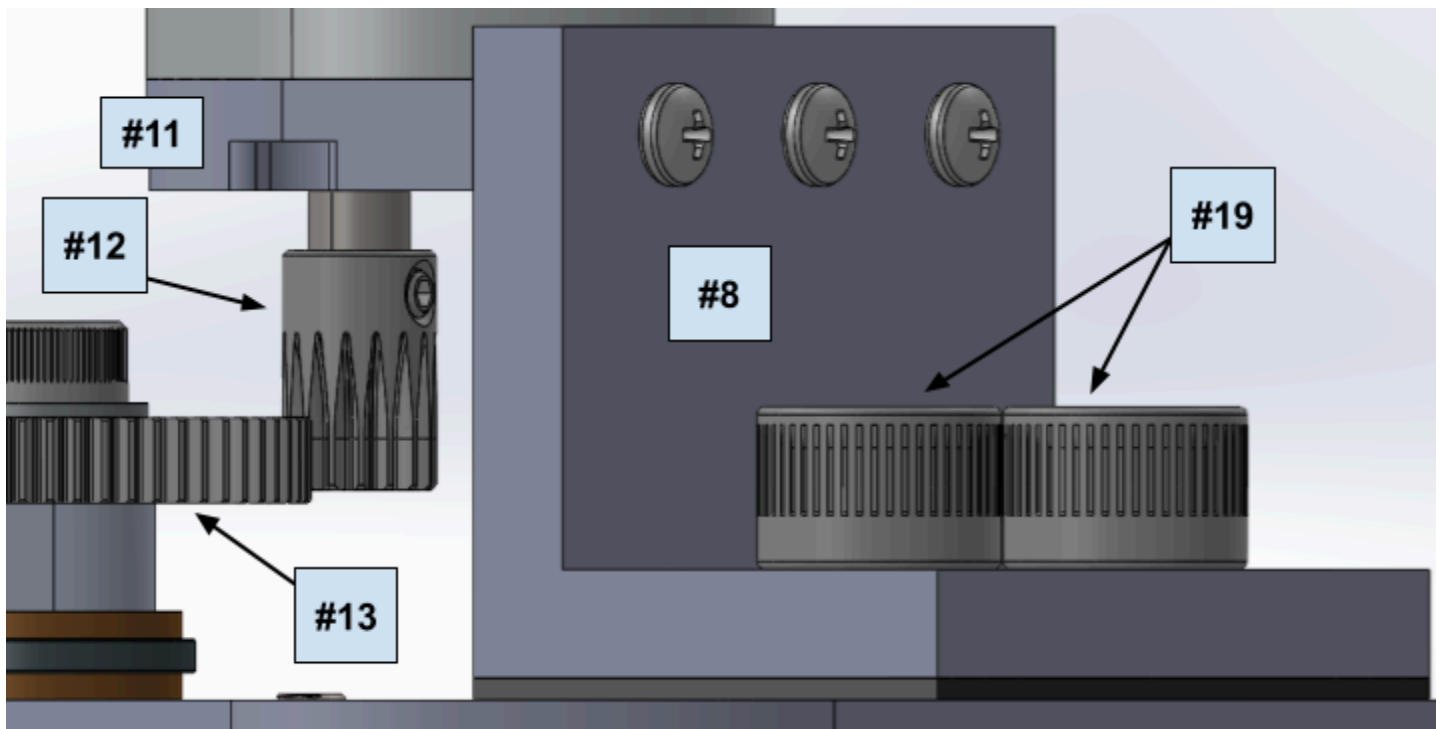
Description: With the current motor assembly on the baseplate, align the pinion and spur gears so that they mesh together and apply force to maintain their current position. Visually confirm that gears are meshing.



Step 7:

Materials: Assembled Baseplate & Linkage, Step 3 Assembly, Motor Mount Spacer, 2x Socket Head Screws

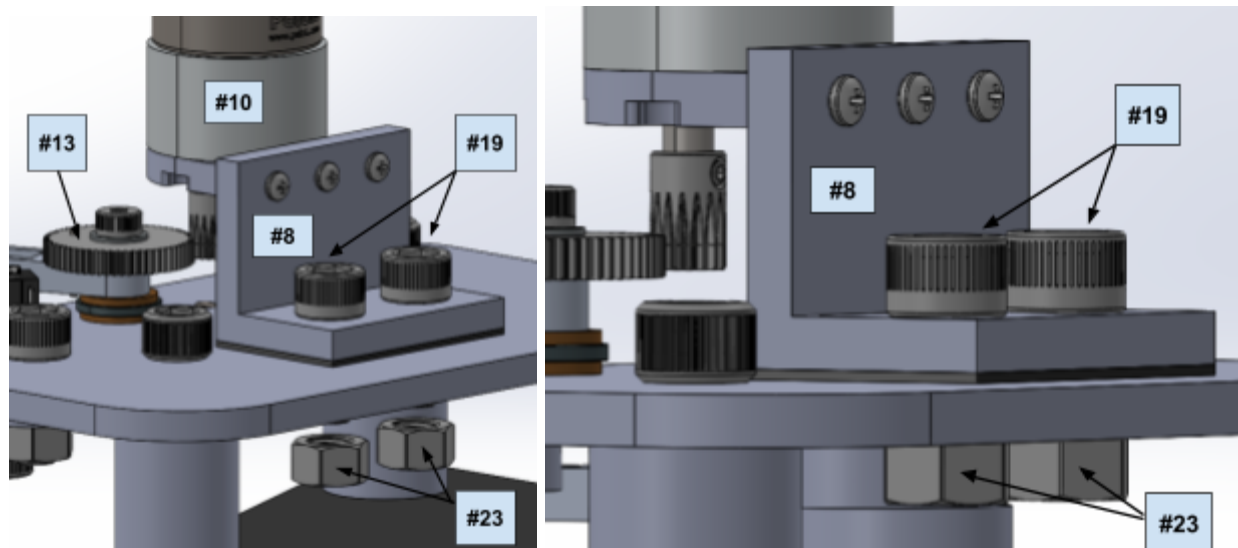
Description: Visually confirm that spur and pinion gears are properly aligned vertically and swap out motor mount spacer with another one of different thickness if necessary.



Step 8:

Materials: Step 6 Assembly, 2x $\frac{3}{8}$ " Nuts

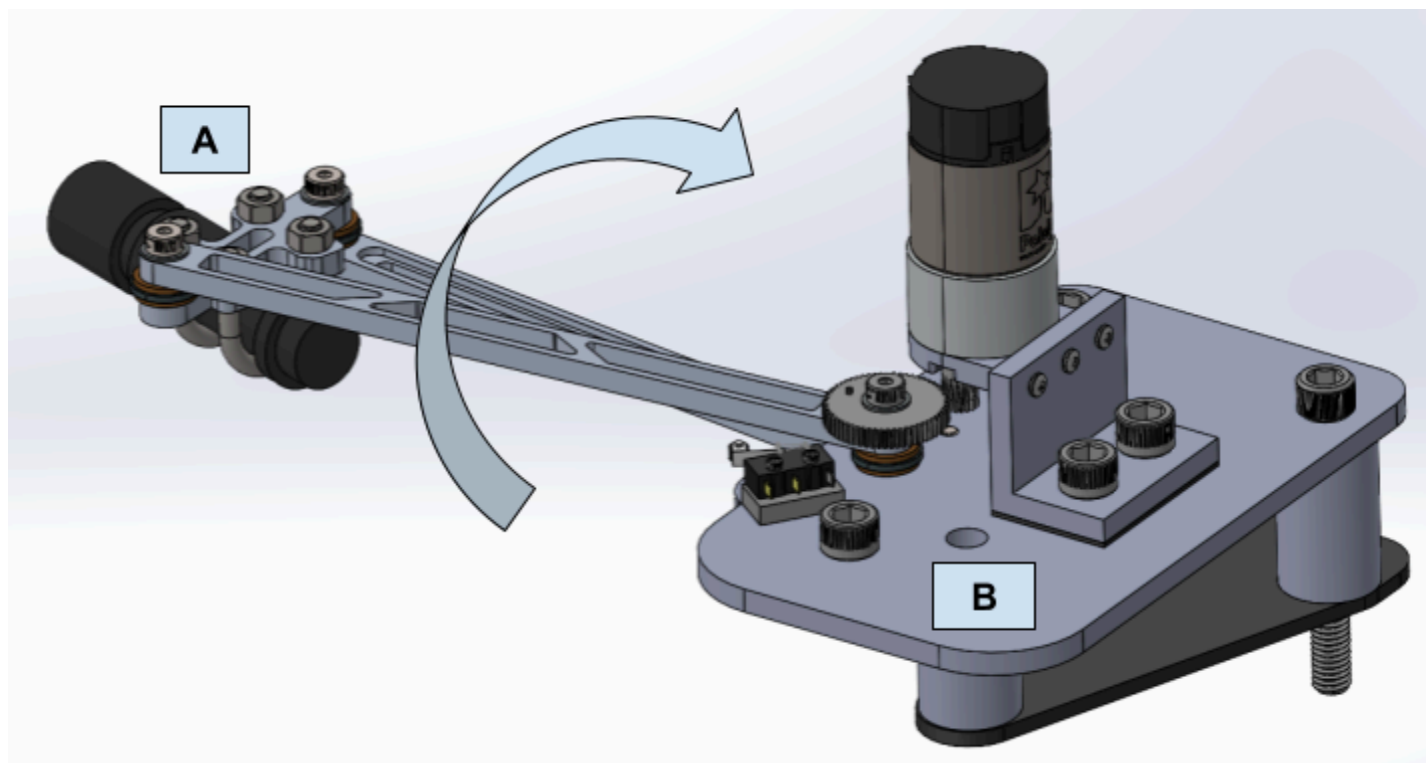
Description: With pinion and spur gear pressed together in their meshed position, thread hex nuts onto the socket head screws and tighten until the motor mount is securely fastened to the baseplate.



Step 9:

Materials: Step 8 Assembly

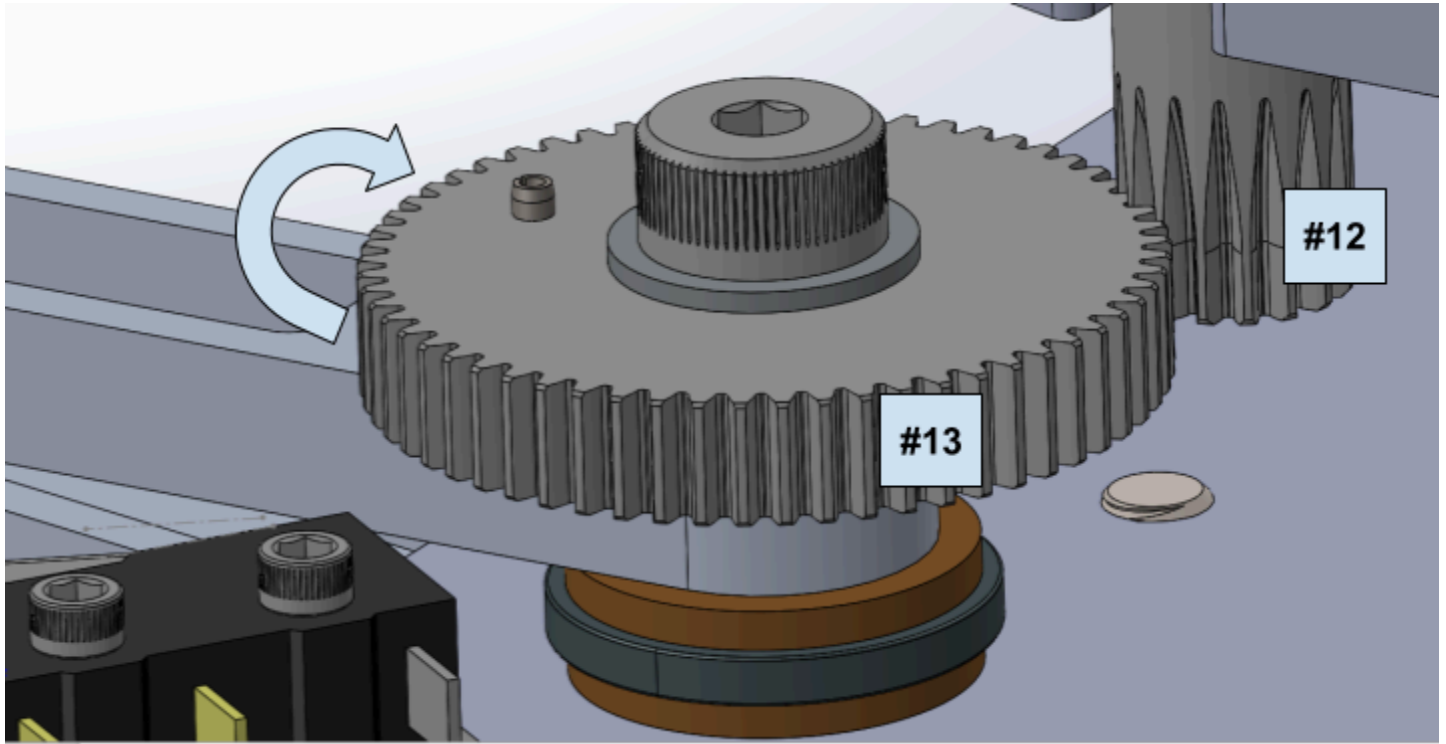
Description: Manually rotate the input link across its full range of motion to ensure that the spur and pinion gears mesh, the motor shaft is rotating, and that the linkage can perform its full range of motion



Step 10:

Materials: Step 8 Assembly

Description: Holding the pinion gear still, attempt to move the input link back and forth to check for backlash between gears. If backlash is too large, untighten the motor mount from the baseplate slightly and readjust the motor position so the gears mesh better. If backlash is sufficiently low, then transmission assembly is done.



Appendix B: Bill of Materials for the Final Design

Bill of Materials:

Part No.	Part Title	Material	Dimension(s) (in)	Supplier	Quantity	Price	Supplier Part No. (if purchased)
1	Input link	Aluminum	0.5 x 7.9 x 0.25	350 Kit	1	-	
2	Coupler link	Aluminum	3 x 1 x 0.25	350 Kit	1	-	
3	Follower link	Aluminum	0.5 x 7.9 x 0.25	350 Kit	1	-	
4	R Hardstop Spacer	Aluminum	1 x 1 x 1.15	350 Kit	2	-	
5	L Hardstop Mount	Aluminum	1 x 2.8 x 0.25	350 Kit	1	-	
6	R/L Hardstop	Aluminum	1 x 1 x 0.43	350 Kit	2	-	
7	Base Plate	Aluminum	6 x 4.5 x 0.25	350 Kit	1	-	
8	Motor mount	Aluminum	2 x 1.3 x 1.4	350 Kit	1	-	
9	Pillar Spacer	Aluminum	1 x 1 x 1.58	350 Kit	2	-	
10	Gear motor	-	37mm x 37mm x 68mm	350 Kit	1	\$51.95	
11	Pololu motor mount	Stainless Steel	1.45 x 1.45 x 0.26	350 Kit	1	\$9.95	
12	Motor (pinion) gear	Brass	14mm x 14mm x 9mm (16 teeth)	McMaster Carr	1	\$11.82	2664N407
13	Link gear	Black-Oxide Carbon Steel	31mm x 31mm x 12mm (60 teeth)	McMaster Carr	1	\$25.06	2664N328
14	Spring pin	Spring Steel	1/16 x 1/16 x 9/16	Assembly area	1	\$0.05	
15	Set screw	Brass	M3 x 2.5mm	McMaster Carr	1	\$0.00	2664N407 (included in part 12)
16	U bolt	Steel	M6 x 20mm x 50mm	Amazon	2	\$8.49	a21012900ux0085
17	Stopper screw	Black-Oxide Alloy Steel	3/8-16" screw x 3	Assembly area	3	\$3.13	
18	Shoulder screw	Alloy Steel	10-24 screw x 0.5	Assembly area	3	\$10.59	
19	Motor mount screw	Black-Oxide Alloy Steel	3/8-16" screw x 5/8	Assembly area	2	\$0.80	
20	L Hardstop screw	Black-Oxide Alloy Steel	3/8-16" screw x 1 5/8	Assembly area	1	\$1.74	
21	L Hardstop mount screw	Black-Oxide Alloy Steel	3/8-16" screw x 1/2	Assembly area	1	\$0.46	
22	U bolt nut	Steel	10mm x 10mm x 5mm (M6)	Amazon	4	\$0.00	a21012900ux0085 (included in part 20)
23	Motor mount nut	Zinc Plated Steel	9/16 x 9/16 x 0.328 (3/8-16")	Assembly area	2	\$0.32	
24	Pololu motor mount screw	Stainless Steel	M3 x 12 mm	350 Kit	3	\$0.35	
25	Thrust bearing	Alloy Steel	0.687 x 0.687 x 0.074 (1/4)	350 Kit	4	\$22.36	
26	Bushing	841 Bearing Bronze	5/16 x 5/16 x 1/4 (1/4)	350 Kit	5	\$4.15	
27	Bearing washer	841 Bearing Bronze	5/8 x 5/8 x 1/16 (1/4)	Assembly area	8	\$12.56	
28	Screw washer	Cadmium Yellow-Chromate Plated Steel	0.468 x 0.468 x 0.032 (1/4)	McMaster Carr	4	\$10.50	5360N126
29	Battery	Lithium	AA size	Amazon	1	\$12.99	L91SBP-4
30	Flashlight	-	1 x 1 x 3.8	350 Kit	1	\$5.00	B01CB2ZTYQ
31	Limit Switch Spacer	3D Printed PLA	0.5 x 0.9 x 0.2	3D Printer	1	-	
32	Motor Mount Spacer	3D Printed PLA	1 x 1.3 x 0.5	3D Printer	1	-	
33	Base Plate Spacer	3D Printed PLA	4.25 x 5 x 1.6	3D Printer	1	-	
			Shipping fee for gears	McMaster Carr	-	\$16.59	
			Sales tax for gears	McMaster Carr	-	\$2.21	
			Shipping fee for U bolts	Amazon	-	\$0.00	
			Salse tax for U bolts	Amazon	-	\$0.51	
				Total cost		\$211.57	
				Total purchased		\$88.17	

SHEET 2 OF 2



SOLIDWORKS Educational Product. For Instructional Use Only.



SOLIDWORKS Educational Product. For Instructional Use Only.

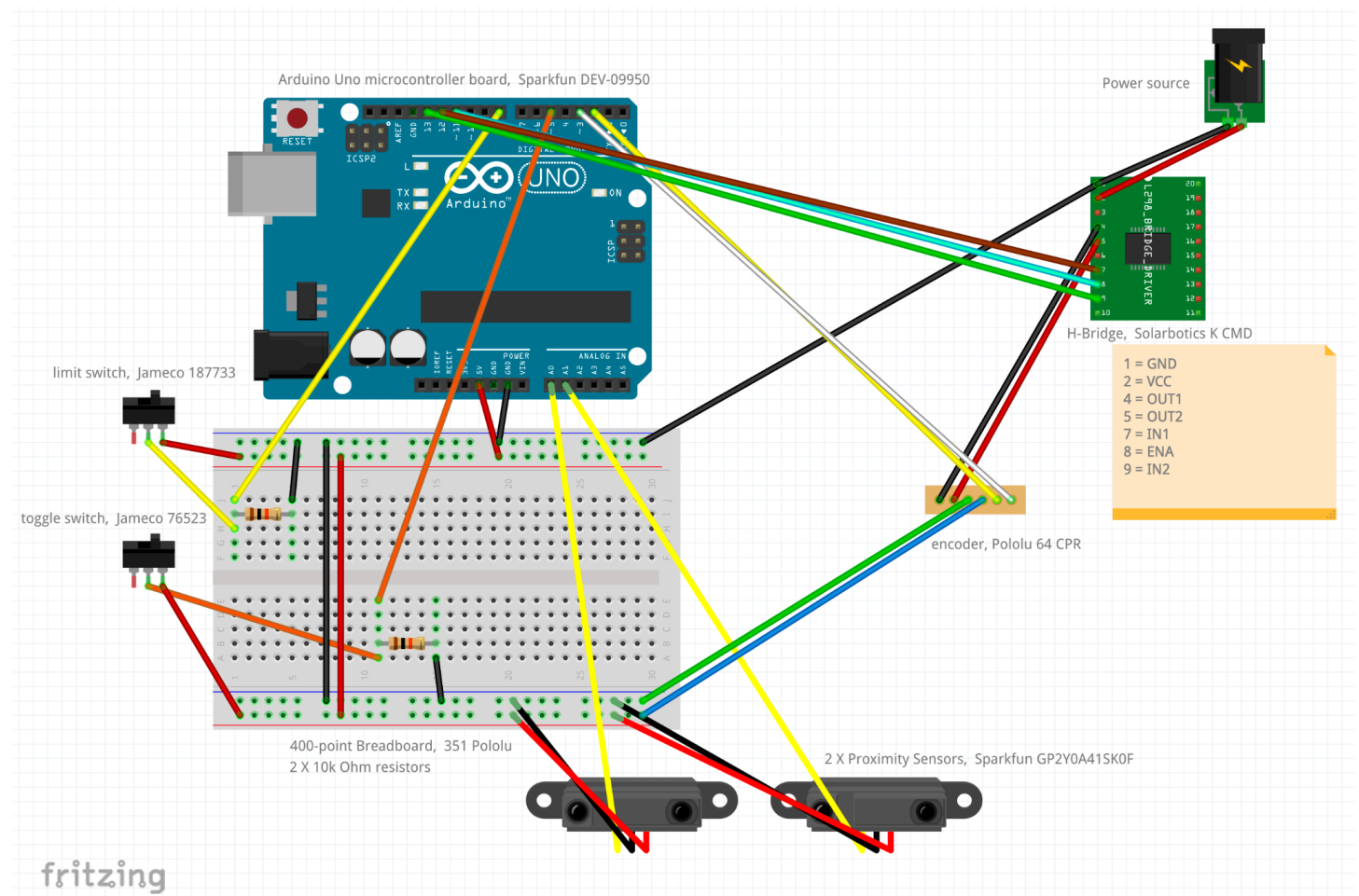
SHEET 2 OF 2



SOLIDWORKS Educational Product. For Instructional Use Only.

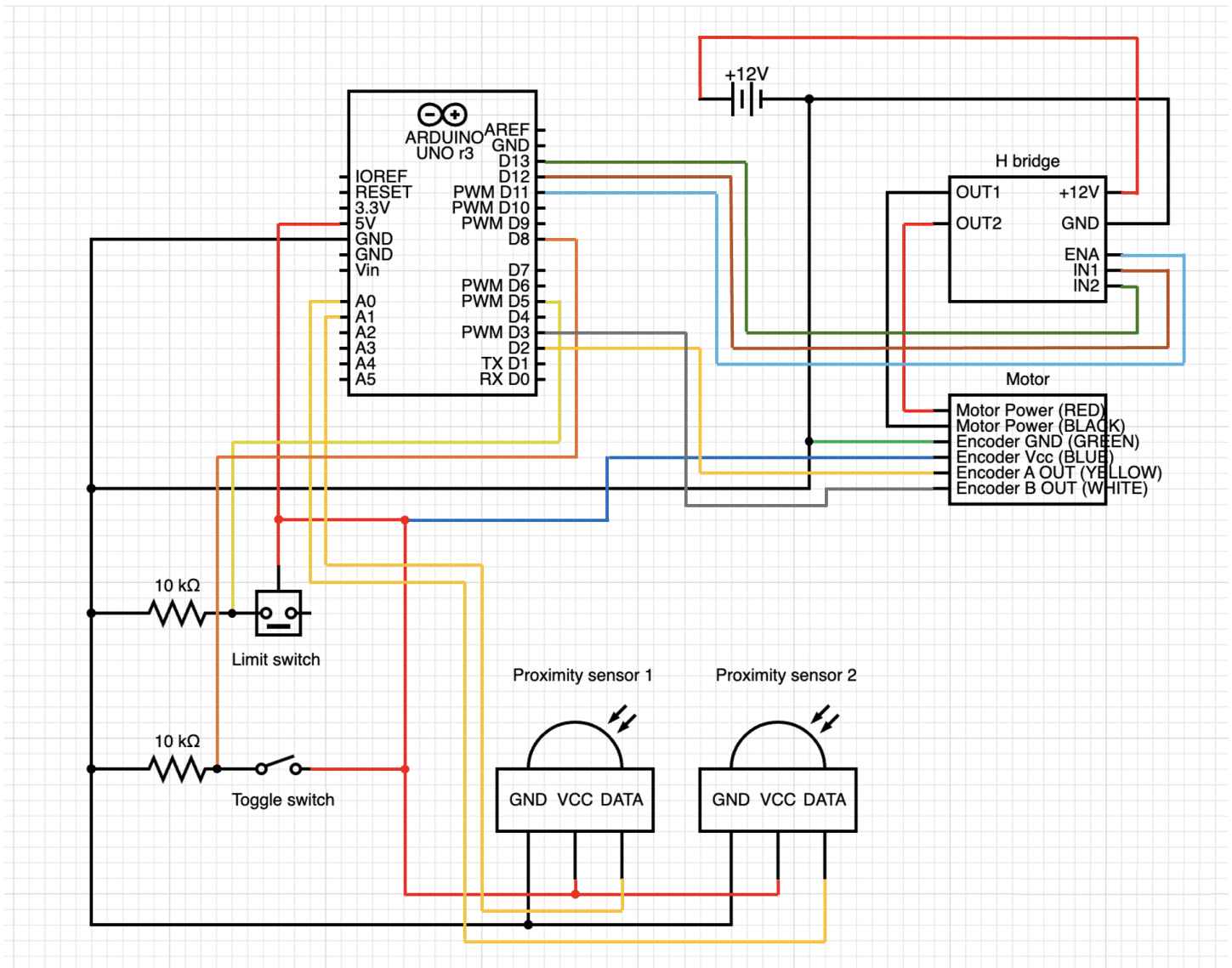
Appendix D: Wiring Diagram, Arduino Code Changes

Wiring Diagram with Breadboard:



Since Fritzing does not have H-Bridge, encoder, and limit switch, different boards and components are used in this diagram. The numbering for the H-Bridge to the board is in the diagram above.

Wiring Diagram without Breadboard:



This diagram is for further reference of the circuit schematic without a breadboard.

Overview of Changes Made to Arduino Code:

Overall, our code from a logic standpoint did not change significantly from the base template given. We updated the proximity sensor values as seen from lines 7-18, the target position values on lines 51-55, and the PID parameters as seen on lines 64-66. These values were determined from our hand-tuning.

The original code was overly verbose so we made some changes to help increase the readability of our code along with marginally increasing the performance. For our final code, we eliminated a lot of code needed for debugging and testing. For example, we eliminated the `printStateToSerial` function completely and all function calls to it in our code. This code was made to print out important parameters—incredibly useful in our testing, but once we were satisfied with our tuning, slowed down our linkage. Therefore, by removing that function, we marginally improve the computational speed of our linkage. Another removal we made was the `activeTarget` array, which was made to determine the status of each target by the encoding determined in our code (1 for active, 0 for inactive, and -1 for inactive). Again this was useful in debugging and testing, however, in our final testing, we don't utilize this array, only the variable `activeTargetPosition`. Thus, by avoiding making unnecessary assignments in our code, we again marginally improve the computational speed of our linkage.

As for the preexisting comments, they were overly verbose, so we slimmed down the amount of comments and verbiage of the comments to improve readability of our code. Many of the old comments stated verbatim what our software was doing. We adjusted our comments to give a higher overview of the purpose of major functions and control blocks such that a person without exposure to our project would better be able to get a sense of the purpose of our code without getting bogged down in unnecessary and trivial comments. We also adjusted the intending of our if statements and loops to further improve readability of our code.

Beyond slimming down the comments to improve readability and eliminating some unnecessary functions to improve computational speed, we didn't make any significant changes to the overall logic of the code. We still have three states in our state machine that function almost identically to the previous template. Our position, speed, and integral calculation are unchanged from the previous one and our pin assignments and other setup statements are identical to the previous code template. Everything else not explicitly mentioned should be verbatim from the previous code template.

Full Arduino Code w/ Line numbers:

```
1. // Definition of states in the state machine
2. const int CALIBRATE = 1;
3. const int DETERMINE_ACTIVE_TARGETS = 2;
4. const int MOVE_TO_TARGET = 3;
5.
6. // Definition of proximity sensor thresholds for each target:
7. const int TARGET1_PROXIMITYSENSE1_MIN = 430;
8. const int TARGET2_PROXIMITYSENSE1_MIN = 107;
9. const int TARGET2_PROXIMITYSENSE1_MAX = 173;
10. const int TARGET3_PROXIMITYSENSE1_MIN = 85;
11. const int TARGET3_PROXIMITYSENSE1_MAX = 106;
12. const int TARGET3_PROXIMITYSENSE2_MIN = 75;
```

```

13. const int TARGET3_PROXIMITYSENSE2_MAX = 90;
14. const int TARGET4_PROXIMITYSENSE2_MIN = 145;
15. const int TARGET4_PROXIMITYSENSE2_MAX = 160;
16. const int TARGET5_PROXIMITYSENSE2_MIN = 365;
17. const int TARGETNONE_PROXIMITYSENSE1_MAX = 84;
18. const int TARGETNONE_PROXIMITYSENSE2_MAX = 80;
19.
20. // Definition of target variables for array indexing
21. const int TARGET1 = 0;
22. const int TARGET2 = 1;
23. const int TARGET3 = 2;
24. const int TARGET4 = 3;
25. const int TARGET5 = 4;
26. const int ACTIVE = 1;
27. const int INACTIVE = 0;
28. const int UNSURE = -1;
29.
30. // Prox sensor readings:
31. int proximitySense1Values[20];
32. int proximitySense2Values[20];
33. float proximitySense1Avg = 0;
34. float proximitySense2Avg = 0;
35.
36. int activeTargetPosition = -1; // [encoder counts] Encoder count of the active
target the linkage is moving toward
37.
38. // Settings for velocity computation:
39. const int MIN_VEL_COMP_COUNT = 2; // [encoder counts] Minimal change in motor
position that must happen between two velocity measurements
40. const long MIN_VEL_COMP_TIME = 10000; // [microseconds] Minimal time that must
pass between two velocity measurements
41.
42. volatile int motorPosition = 0; // [encoder counts] Current motor position
(Declared 'volatile', since it is updated in a function called by interrupts)
43. volatile int encoderStatus = 0; // [binary] Past and Current A&B values of the
encoder (Declared 'volatile', since it is updated in a function called by
interrupts)
44.
45. float motorVelocity = 0; // [encoder counts / seconds] Current motor
velocity

```



```

46. int previousMotorPosition = 0; // [encoder counts] Motor position the last time
a velocity was computed
47. long previousVelCompTime = 0; // [microseconds] System clock value the last
time a velocity was computed
48. const int CALIBRATION_VOLTAGE = -3; // [Volt] Motor voltage used during the
calibration process
49.
50. // Target positions:
51. const int TARGET_1_POSITION = 314;
52. const int TARGET_2_POSITION = 640;
53. const int TARGET_3_POSITION = 927;
54. const int TARGET_4_POSITION = 1216; // 1933
55. const int TARGET_5_POSITION = 1940;
56.
57. // Wait position (the position where your linkage waits until a target is
detected)
58. const int WAIT_POSITION = TARGET_2_POSITION;
59. const int LOWER_BOUND = 0; // [encoder counts] Position of the left end stop
60. const int UPPER_BOUND = TARGET_5_POSITION; // [encoder counts] Position of the
right end stop
61. const int TARGET_BAND = 5; // [encoder counts] "Close enough" range when moving
towards a target.
62.
63. /** PID Controller **/
64. const float KP = 0.09; // [Volt / encoder counts] P-Gain
65. const float KI = 0.015; // 0.003; // [Volt / (encoder counts *
seconds)] I-Gain
66. const float KD = 0.006; // 0.006; // [Volt * seconds / encoder
counts] D-Gain
67. const float SUPPLY_VOLTAGE = 7; // [Volt] Supply voltage at the HBridge
68. const float FRICTION_COMP_VOLTAGE = 2; // [Volt] Voltage needed to overcome
friction
69.
70. // VARIABLES:
71. int desiredPosition = 0; // [encoder counts] desired motor position
72. float positionError = 0; // [encoder counts] Position error
73. float integralError = 0; // [encoder counts * seconds] Integrated position error
74. float velocityError = 0; // [encoder counts / seconds] Velocity error
75. float desiredVoltage = 0; // [Volt] Desired motor voltage
76. int motorCommand = 0; // [0-255] PWM signal sent to the motor

```

```

77. unsigned long executionDuration = 0; // [microseconds] Time between this and the
previous loop execution. Variable used for integrals and derivatives
78. unsigned long lastExecutionTime = 0; // [microseconds] System clock value at the
moment the loop was started the last time
79.
80. /** Pin assignment: **//
81. const int PIN_NR_ENCODER_A      = 2;  // Never change these, since the
interrupts are attached to pins 2 and 3
82. const int PIN_NR_ENCODER_B      = 3;  // Never change these, since the
interrupts are attached to pins 2 and 3
83. const int PIN_NR_ON_OFF_SWITCH  = 5;  // Connected to toggle switch (turns
mechanism on and off)
84. const int PIN_NRL_LIMIT_SWITCH  = 8;  // Connected to limit switch (mechanism
calibration)
85. const int PIN_NR_PWM_OUTPUT     = 11; // Connected to H Bridge (controls motor
speed)
86. const int PIN_NR_PWM_DIRECTION_1 = 12; // Connected to H Bridge (controls motor
direction)
87. const int PIN_NR_PWM_DIRECTION_2 = 13; // Connected to H Bridge (controls motor
direction)
88. const int PIN_PROXSENSE1       = A0; // Connected to proximity sensor 1
89. const int PIN_PROXSENSE2       = A1; // Connected to proximity sensor 2
90. const int PIN_POTENTIOMETER     = A2; // Connected to potentiometer used to
test target positions
91.
92. void setup() {
93.
94.     // Declare which digital pins are inputs and which are outputs:
95.     pinMode(PIN_NR_ENCODER_A,      INPUT_PULLUP);
96.     pinMode(PIN_NR_ENCODER_B,      INPUT_PULLUP);
97.     pinMode(PIN_NR_ON_OFF_SWITCH,   INPUT);
98.     pinMode(PIN_NRL_LIMIT_SWITCH,   INPUT);
99.     pinMode(PIN_PROXSENSE1,         INPUT);
100.    pinMode(PIN_PROXSENSE2,          INPUT);
101.    pinMode(PIN_NR_PWM_OUTPUT,        OUTPUT);
102.    pinMode(PIN_NR_PWM_DIRECTION_1,   OUTPUT);
103.    pinMode(PIN_NR_PWM_DIRECTION_2,   OUTPUT);
104.    pinMode(PIN_POTENTIOMETER,        INPUT);
105.
106.    // Turn on the pullup resistors on the encoder channels

```

```

107.   digitalWrite(PIN_NR_ENCODER_A, HIGH);
108.   digitalWrite(PIN_NR_ENCODER_B, HIGH);
109.
110.   // Activate interrupt for encoder pins.
111.   // If either of the two pins changes, the function 'updateMotorPosition' is
called:
112.   attachInterrupt(0, updateMotorPosition, CHANGE); // Interrupt 0 is always
attached to digital pin 2
113.   attachInterrupt(1, updateMotorPosition, CHANGE); // Interrupt 1 is always
attached to digital pin 3
114.
115.   // Begin serial communication for monitoring.
116.   Serial.begin(115200);
117.   Serial.println("Start Executing Program.");
118.
119.   // Set initial output to the motor to 0
120.   analogWrite(PIN_NR_PWM_OUTPUT, 0);
121.
122. }
123.
124. void loop() {
125.
126.   // Determine the duration it took to execute the last loop. This time is used
127.   // for integration and for monitoring the loop time via the serial monitor.
128.   executionDuration = micros() - lastExecutionTime;
129.   lastExecutionTime = micros();
130.
131.   // Speed Computation:
132.   if ((abs(motorPosition - previousMotorPosition) > MIN_VEL_COMP_COUNT) ||
(micros() - previousVelCompTime) > MIN_VEL_COMP_TIME){
133.       motorVelocity = (double)(motorPosition - previousMotorPosition) * 1000000 /
134.                       (micros() - previousVelCompTime);
135.
136.       previousMotorPosition = motorPosition;
137.       previousVelCompTime    = micros();
138.   }
139.
140.   // The state machine:
141.   switch (state) {
142.

```

```

143.     case CALIBRATE:
144.         if (digitalRead(PIN_NRL_LIMIT_SWITCH)==HIGH && motorVelocity==0) {
145.
146.             motorPosition = LOWER_BOUND;
147.             integralError = 0;
148.             Serial.println("State transition from CALIBRATE to
DETERMINE_ACTIVE_TARGETS");
149.             state = DETERMINE_ACTIVE_TARGETS;
150.
151.         }
152.
153.         break;
154.
155.     case DETERMINE_ACTIVE_TARGETS:
156.         for (int i = 0; i < sizeof(activeTargets)/sizeof(int); i++) {
157.             activeTargets[i] = UNSURE;
158.         }
159.
160.         activeTargetPosition = UNSURE;
161.
162.         for (int i = 0; i < sizeof(proximitySense1Values)/sizeof(int); i++) {
163.
164.             proximitySense1Values[i] = analogRead(PIN_PROXSENSE1);
165.         }
166.         for (int i = 0; i < sizeof(proximitySense2Values)/sizeof(int); i++) {
167.             proximitySense2Values[i] = analogRead(PIN_PROXSENSE2);
168.         }
169.
170.         proximitySense1Avg = average(proximitySense1Values,
sizeof(proximitySense1Values)/sizeof(int));
171.         proximitySense2Avg = average(proximitySense2Values,
sizeof(proximitySense2Values)/sizeof(int));
172.
173.         // Determine which targets active and activeTargetPosition
174.         if (proximitySense2Avg > TARGET5_PROXIMITYSENSE2_MIN) {
175.             activeTargetPosition = TARGET_5_POSITION;
176.         }
177.         else if (proximitySense2Avg > TARGET4_PROXIMITYSENSE2_MIN &&
proximitySense2Avg < TARGET4_PROXIMITYSENSE2_MAX) {
178.             activeTargetPosition = TARGET_4_POSITION;

```

```

179.
180.     }
181.     if (proximitySense1Avg > TARGET1_PROXIMITYSENSE1_MIN) {
182.         activeTargetPosition = TARGET_1_POSITION;
183.
184.     }
185.     else if (proximitySense1Avg > TARGET2_PROXIMITYSENSE1_MIN &&
proximitySense1Avg < TARGET2_PROXIMITYSENSE1_MAX) {
186.         activeTargetPosition = TARGET_2_POSITION;
187.
188.     }
189.     else if (proximitySense1Avg > TARGET3_PROXIMITYSENSE1_MIN &&
proximitySense1Avg < TARGET3_PROXIMITYSENSE1_MAX) {
190.         activeTargetPosition = TARGET_3_POSITION;
191.
192.     }
193.     if (proximitySense1Avg < TARGETNONE_PROXIMITYSENSE1_MAX &&
proximitySense2Avg < TARGETNONE_PROXIMITYSENSE2_MAX) {
194.         activeTargetPosition = WAIT_POSITION; // Make linkage go to wait
position if no targets are active
195.     }
196.
197.     if (activeTargetPosition != UNSURE) {
198.         state = MOVE_TO_TARGET;
199.     }
200.
201.     break;
202.
203.     case MOVE_TO_TARGET:
204.
205.         desiredPosition = activeTargetPosition;
206.         if (motorPosition <= activeTargetPosition + TARGET_BAND && motorPosition
>= activeTargetPosition - TARGET_BAND) {
207.             state = DETERMINE_ACTIVE_TARGETS;
208.         }
209.         break;
210.
211.     default:
212.         Serial.println("Statemachine reached at state that it cannot handle.
ABORT!!!!");

```

```
213.     Serial.print("Found the following unknown state: ");
214.     Serial.println(state);
215.     while (1);
216.     break;
217. }
218.
219. if (digitalRead(PIN_NRL_LIMIT_SWITCH)==HIGH && motorVelocity==0) {
220.
221.     motorPosition = LOWER_BOUND;
222.     integralError = 0;
223. }
224.
225. if (digitalRead(PIN_NR_ON_OFF_SWITCH)==HIGH) {
226.
227.     positionError = desiredPosition - motorPosition;
228.     integralError = integralError + positionError * (float)(executionDuration) /
1000000;
229.     velocityError = 0 - motorVelocity;
230.     desiredVoltage = KP * positionError + KI * integralError + KD *
velocityError;
231.
232.     if (positionError < -5) {
233.         desiredVoltage = desiredVoltage - FRICTION_COMP_VOLTAGE;
234.     }
235.     if (positionError > +5) {
236.         desiredVoltage = desiredVoltage + FRICTION_COMP_VOLTAGE;
237.     }
238.
239.     // Anti-Wind-Up
240.     if (abs(desiredVoltage)>SUPPLY_VOLTAGE) {
241.
242.         integralError = integralError - positionError * (float)(executionDuration)
/ 1000000;
243.     }
244.
245.     if (state==CALIBRATE) {
246.         desiredVoltage = CALIBRATION_VOLTAGE;
247.
248.     }
249. } else {
```

```

250.
251.     desiredVoltage = 0;
252.     integralError = 0;
253.     Serial.println("The toggle switch is off.  Motor Stopped.");
254. }
255.
256. motorCommand = int(abs(desiredVoltage * 255 / SUPPLY_VOLTAGE)); // Convert
from voltage to PWM cycle:
257.
258. // Clip values larger than 255
259. if (motorCommand > 255) {
260.     motorCommand = 255;
261. }
262.
263. analogWrite(PIN_NR_PWM_OUTPUT, motorCommand); // Send motor signals out
264.
265. if (desiredVoltage >= 0) {
266.
267.     digitalWrite(PIN_NR_PWM_DIRECTION_1, LOW); // rotate forward
268.     digitalWrite(PIN_NR_PWM_DIRECTION_2, HIGH); // rotate forward
269. } else {
270.
271.     digitalWrite(PIN_NR_PWM_DIRECTION_1, HIGH); // rotate backward
272.     digitalWrite(PIN_NR_PWM_DIRECTION_2, LOW); // rotate backward
273. }
274. }
275.
276. void updateMotorPosition() {
277.     // Bitwise shift left by one bit, to make room for a bit of new data:
278.     encoderStatus <<= 1;
279.     // Use a compound bitwise OR operator (|=) to read the A channel of the
encoder (pin 2)
280.     // and put that value into the rightmost bit of encoderStatus:
281.     encoderStatus |= digitalRead(2);
282.     // Bitwise shift left by one bit, to make room for a bit of new data:
283.     encoderStatus <<= 1;
284.     // Use a compound bitwise OR operator (|=) to read the B channel of the
encoder (pin 3)
285.     // and put that value into the rightmost bit of encoderStatus:
286.     encoderStatus |= digitalRead(3);

```

```
287. // encoderStatus is truncated to only contain the rightmost 4 bits by using a
288. // bitwise AND operator on mstatus and 15(=1111):
289. encoderStatus &= 15;
290. if (encoderStatus==2 || encoderStatus==4 || encoderStatus==11 ||
encoderStatus==13) {
291.     // the encoder status matches a bit pattern that requires counting up by one
292.     motorPosition++; // increase the encoder count by one
293. }
294. else if (encoderStatus == 1 || encoderStatus == 7 || encoderStatus == 8 ||
encoderStatus == 14) {
295.     // the encoder status does not match a bit pattern that requires counting up
by one.
296.     // Since this function is only called if something has changed, we have to
count downwards
297.     motorPosition--; // decrease the encoder count by one
298. }
299. }
300.
301. float average (int * array_ptr, int len) {
302.     long sum = 0L;
303.     for (int i = 0; i < len; i++) {
304.         sum += array_ptr[i];
305.     }
306.     return ((float) sum)/len;
307. }
```